

Dobývání znalostí

Učební text k okruhu státní závěrečné zkoušky

SZZ Umělá inteligence, MFF UK

srpen 2026

Text pokrývá okruh *Dobývání znalostí* magisterské SZZ oboru Umělá inteligence. Členění je záměrně přímočaré: kapitoly odpovídají **jedna ku jedné** větám oficiálního znění okruhu, takže se u zkoušky dá postupovat rovnou podle nich. Přehled dává mapovací tabulka níže.

Výklad vychází ze slidů dvou přednášek I. Mrázové na MFF UK, totiž **NDBI023 Dobývání znalostí** a **NAIL116 Sociální sítě a jejich analýza**. Tam, kde slidy samy nestačí, je doplněn o standardní látku z učebnic: Han, Kamber, Pei: *Data Mining — Concepts and Techniques*, Tan, Steinbach, Kumar: *Introduction to Data Mining* a Aggarwal: *Data Mining — The Textbook*; pro sítě pak Easley, Kleinberg: *Networks, Crowds and Markets*, Newman: *Networks* a Zafarani, Abbasi, Liu: *Social Media Mining*. Vedle definic a pseudokódů obsahuje text také odvození, číselné příklady a srovnávací tabulky. Každou kapitolu uzavírá shrnutí pro ústní zkoušku.

Rozsah textu a značení

Rozsah jednotlivých kapitol odpovídá váze látky. Nejvíce prostoru dostávají metody dobývání znalostí (kap. 3) a analýza sociálních sítí (kap. 6); právě těmito dvěma tématům se obě přednášky věnují nejpodrobněji a právě je okruh jmenuje nejkonkrétněji.

Značka [♦ nad rámec slidů] upozorňuje na látku, která **není ve slidech** ani jednoho z obou kurzů. Slidy jsou přitom k dispozici kompletní, takže nejde o mezeru v podkladech, ale o látku, která se v nich opravdu neprobírá. V textu přesto zůstává, a to ze dvou důvodů: buď ji *jmenuje oficiální znění okruhu*, nebo bez ní nedává okolní výklad smysl.

Značka je proto zároveň nabídkou ke škrtnutí: co označeno není, je ve slidech doloženo. Následuje-li za značkou dvojtečka, upřesňuje, čeho přesně se výhrada týká. Typicky jde o případ, kdy souhrnná tabulka nebo rámec jsou moje, ale jednotlivé položky v nich ze slidů pocházejí.

Ze slidů NDBI023 pochází analýza dat a shlukování, asociační pravidla, rozhodovací stromy a ensemble, SVM spolu s vyhodnocováním klasifikátorů, bayesovské modely a ELM. NAIL116 k tomu přidává základní pojmy, modelování vlivu, homofilii a afiliaci, analýzu vazeb a komplexní sítě včetně detekce komunit a predikce vazeb. Nejnápadnější mezery jsou dvě: **proces KDD a metodologie** (kap. 1) a **charakteristická a diskriminační pravidla** (kap. 4). Okruh je jmenuje výslovně, ale *ani jeden z kurzů je neprobírá*, a jsou tedy zpracovány podle Han–Kamber.

Mapování okruhu na kapitoly

Věta oficiálního znění okruhu	Kapitola v tomto textu
Základní paradigmaty dobývání znalostí.	1 Základní paradigmaty dobývání znalostí
Příprava dat; výběr atributů a metody pro analýzu jejich relevance.	2 Příprava dat, výběr atributů a analýza jejich relevance
Metody pro dobývání znalostí; asociační pravidla, přístupy založené na principu učení s učitelem a klastrová analýza.	3 Metody pro dobývání znalostí (3.1 asociační pravidla, 3.2–3.3 učení s učitelem, 3.4 klastrová analýza)
Metody pro extrakci charakteristických diskriminačních pravidel a měření jejich zajímavosti.	4 Charakteristická a diskriminační pravidla a jejich zajímavost
Reprezentace, vyhodnocování a vizualizace získaných znalostí.	5 Reprezentace, vyhodnocování a vizualizace znalostí
Modely pro analýzu sociálních sítí; míry centrality, detekce komunit.	6 Modely pro analýzu sociálních sítí, centralita a detekce komunit
Praktické využití technik pro dobývání znalostí a analýzu sociálních sítí.	7 Praktické využití technik

Poznámka k terminologii. Text používá české termíny; anglický ekvivalent je uveden kurzívou v závorce při prvním výskytu, protože obě přednášky i celá literatura jsou anglicky.

Obsah

Mapování okruhu na kapitoly	2
1 Základní paradigmaty dobývání znalostí	5
1.1 Motivace a vymezení oboru	5
1.2 Proces KDD	6
1.3 Typy úloh dobývání znalostí	6
1.4 Typy dat	7
1.5 Metodologie dobývání znalostí	8
1.6 Shrnutí ke zkoušce	9
2 Příprava dat, výběr atributů a analýza jejich relevance	10
2.1 Proč je příprava dat klíčová	10
2.2 Čištění dat: chybějící hodnoty	10
2.3 Čištění dat: šum a odlehlé hodnoty	10
2.4 Integrace dat	11
2.5 Transformace a standardizace atributů	11
2.6 Diskretizace numerických atributů	12
2.6.1 Fuzzy diskretizace	12
2.6.2 Seskupování hodnot kategoriálních atributů (KEX)	13
2.7 Redukce dat	13
2.7.1 Příliš mnoho objektů	13
2.7.2 Příliš mnoho atributů	13
2.8 Analýza relevance atributů: filtry a wrappery	14
2.8.1 Kritéria na kontingenčních tabulkách	14
2.8.2 Kritéria založená na entropii	15
2.9 Vztahy mezi atributy: regrese a korelace	16
2.10 Shrnutí ke zkoušce	17
3 Metody pro dobývání znalostí	19
3.1 Asociační pravidla a frekventované vzory	19
3.1.1 Analýza nákupního košíku	19

3.1.2	Support, confidence a lift	19
3.1.3	Volba položek, taxonomie a virtuální položky	20
3.1.4	Algoritmus Apriori	21
3.1.5	Vícenásobný minimální support: MS-Apriori	22
3.1.6	Třídní asociační pravidla (CAR)	24
3.1.7	Dolování sekvenčních vzorů	24
3.1.8	FP-Growth a FP-strom	25
3.2	Přístupy založené na principu učení s učitelem	26
3.2.1	Formulace úlohy	26
3.2.2	Rozhodovací stromy	27
3.2.3	Bayesovská klasifikace	31
3.2.4	Support Vector Machines	32
3.2.5	Logistická regrese	34
3.2.6	Diskriminační analýza	35
3.2.7	Neuronové sítě a extrémní učící stroje (ELM)	35
3.2.8	Ensemble metody	36
3.2.9	Srovnání klasifikačních metod	38
3.3	Klastrová analýza	38
3.3.1	Formulace úlohy a míry vzdálenosti	38
3.3.2	Metoda k -means	39
3.3.3	Metoda k -medoids, CLARA a CLARANS	40
3.3.4	Hierarchické shlukování	41
3.3.5	Vektorová kvantizace: algoritmus LVQ	42
3.3.6	Mřížkové a hustotní metody	42
3.4	Shrnutí ke zkoušce	43
4	Charakteristická a diskriminační pravidla a jejich zajímavost	46
4.1	Charakterizace vs. diskriminace	46
4.2	Koncepční hierarchie	46
4.3	Indukce orientovaná na atributy (AOI)	47
4.4	Míry t -weight a d -weight	47
4.5	Měření zajímavosti pravidel	49
4.6	Shrnutí ke zkoušce	49
5	Reprezentace, vyhodnocování a vizualizace znalostí	51
5.1	Formy reprezentace znalostí	51
5.2	Vyhodnocování modelů	51
5.2.1	Kritéria hodnocení klasifikačních metod	52
5.2.2	Metody odhadu chyby	52
5.2.3	Matice záměn a odvozené míry	53
5.2.4	Skórování, řazení a lift analýza	53
5.2.5	ROC křivka a AUC	54
5.3	Vizualizační techniky	55
5.4	Vyhodnocení znalostí z pohledu uživatele	55
5.5	Shrnutí ke zkoušce	56
6	Modely pro analýzu sociálních sítí, centralita a detekce komunit	57
6.1	Vymezení oboru a motivace	57
6.2	Reprezentace sítí	57
6.3	Síla vazeb	58
6.4	Klíčoví hráči: míry centrality	59
6.5	Koheze: míry na úrovni sítě	60
6.6	Model reálných komplexních sítí a jejich vlastnosti	61

6.7	Modelování vlivu	63
6.8	Vliv versus korelace	63
6.9	Homofilie, výběr a sociální vliv	65
6.10	Afiliční sítě a uzávěrové procesy	66
6.11	Schellingův model segregace	67
6.12	Pozitivní a negativní vztahy: strukturní balance	68
6.13	Analýza vazeb: huby, autority a algoritmus HITS	69
6.14	PageRank	71
6.15	Tematické komunity a jejich objevování	73
6.16	Detekce komunit	74
6.16.1	Taxonomie metod detekce komunit	77
6.17	Predikce vazeb	79
6.18	Shrnutí ke zkoušce	81
7	Praktické využití technik	84
7.1	Dobývání znalostí	84
7.2	Analýza sociálních sítí	85
7.3	Provozní a etické aspekty nasazení	86
7.4	Shrnutí ke zkoušce	86

1 Základní paradigmatata dobývání znalostí

Tato kapitola vymezuje obor, popisuje jeho proces a zavádí základní dělení úloh a dat. Je to rámec, do něhož se zasazují všechny kapitoly následující. Bez něj by nebylo zřejmé, proč si příprava dat zaslouží samostatnou kapitolu a proč se metody dělí zrovna na asociační pravidla, učení s učitelem a shlukování.

1.1 Motivace a vymezení oboru

Organizace shromažďují data v objemech, které daleko přesahují kapacitu člověka je vůbec prohlédnout. Klasické dotazování, tedy SQL a OLAP, si s tím poradí jen zčásti: umí spolehlivě odpovědět na otázku, kterou už umíme položit. *Dobývání znalostí* míří jinam: odpovídá na otázky, které jsme položit neuměli, protože hledá v datech struktury, o jejichž existenci jsme předem nevěděli.

Dobře to ilustruje jeden příklad. Systém prošel 20 milionů výkonů účtovaných zdravotní pojišťovně, 214 tisíc z nich označil za podivné a 14 tisíc nejpodezřelejších předal k ruční kontrole. Revizní lékaři jsou schopni prohlédnout řádově stovky případů; z těch, které jim systém předložil, jich okamžitě 8 tisíc označili za podvodné. Stroj tady experta nenahradil, jen *zaostřil* jeho pozornost.

Dobývání znalostí z databází

Dobývání znalostí z databází (*Knowledge Discovery in Databases*, KDD) je netriviální proces získávání implicitních, dosud neznámých a potenciálně užitečných informací z dat.

Klíčová jsou všechna čtyři slova:

- **Netriviální** vylučuje prostý dotaz i agregaci: kdyby výsledek stačilo z databáze vyčíst, o dobývání znalostí by nešlo.
- **Implicitní** znamená, že znalost není v datech nikde explicitně zapsaná, leží v nich jen jako struktura.
- **Dosud neznámá** vyřazuje závěry, které každý zná předem; potvrzená samozřejmost je bezcenná.
- **Potenciálně užitečná** požaduje, aby nález vedl k akci nebo k rozhodnutí.

Termín *dobývání dat* (*data mining*, DM) se v úzkém pojetí vztahuje pouze k jedinému kroku procesu KDD, totiž k vlastní aplikaci analytických algoritmů. Běžná praxe tento rozdíl nedodržuje a oba pojmy používá zaměnitelně. Jako samostatný obor se dobývání znalostí konstitovalo v 90. letech 20. století, a to na průsečíku tří starších disciplín:

- **Umělá inteligence** dodala především metody strojového učení, tedy indukci pravidel, stromy a neuronové sítě.
- **Databázové systémy** se starají o ukládání velkých datových souborů a efektivní přístup k nim; odtud pocházejí datové sklady i techniky vyhledávání informací.
- **Statistika** přispěla modelováním a analýzou závislostí nalezených v datech a testováním hypotéz.

Těmi třemi ale výčet nekončí. Čtvrtou složkou, v praxi ovšem rozhodující, je nutnost využít zpracovaná data k podpoře (strategického) rozhodování v podniku. Bez ní se dobývání znalostí redukuje na akademické cvičení.

Vztah k datovým skladům a OLAP. [♦ nad rámec slidů] Typickým zdrojem dat pro KDD bývá *datový sklad* (*data warehouse*), tedy integrovaná, subjektově orientovaná, časově variantní a stálá databáze konsolidovaná z provozních systémů. Data se v něm organizují do *datové kostky*, kterou tvoří *dimenze* (čas, produkt, region, ...) $\times$ *míry* (tržby, počty kusů). V relační implementaci kostce odpovídá *hvězdicové schéma*, v němž je tabulka faktů obklopena tabulkami dimenzí; fyzicky se data ukládají buď multidimenzionálně (MOLAP), nebo relačně (ROLAP).

Nad kostkou pracuje **OLAP** (*On-Line Analytical Processing*) a umožňuje agregační operace *roll-up* a *drill-down* a řezy *slice* a *dice*. I on ale odpovídá jen na otázky, které si analytik umí *předem položit*.

Dobývání znalostí jde o krok dál a hledá vzory, o nichž předem nevíme, že v datech jsou. Souvislost generalizace v OLAP s indukci orientovanou na atributy viz kap. 4.

1.2 Proces KDD

[♦ nad rámec slidů: pětifázový rámec jako celek; náplň jednotlivých fází ve slidech je] Dobývání znalostí není přímočará posloupnost kroků, ale **interaktivní a iterativní** proces: výsledky jednoho kroku běžně vedou k návratu ke krokům předchozím. Klasické schéma (Fayyad et al.) rozděluje celý postup do pěti fází:

Fáze	Vstup → výstup	Náplň
Výběr (<i>selection</i>)	původní data → vybraná data	výběr relevantních záznamů a atributů z datového skladu
Předzpracování (<i>pre-processing</i>)	vybraná data → předzpracovaná data	čištění, chybějící hodnoty, šum, odlehlé hodnoty, integrace zdrojů
Transformace (<i>transformation</i>)	předzpracovaná data → transformovaná data	normalizace, diskretizace, odvozené atributy, redukce dimenze
Vlastní dobývání (<i>data mining</i>)	transformovaná data → vzory	aplikace algoritmů (stromy, asociace, klastry, ...)
Interpretace (<i>interpretation</i>)	vzory → znalost	vyhodnocení z pohledu koncového uživatele, vizualizace, nasazení

Cílem prvních tří fází, souhrnně nazývaných *příprava dat* (kap. 2), je z dat uložených ve složité struktuře vybudovat **jednu tabulku** obsahující relevantní údaje o zkoumaných objektech, tedy o klientech banky, o zákaznících a podobně. Zní to jako pouhá administrativa, ale právě tento krok typicky spotřebuje 60–80 % celkového času projektu.

Sedm kroků očima manažera. [♦ nad rámec slidů] Na tentýž proces se lze podívat i organizačně; impulsem je pak „aktuální problém“ a výstupem „znalost k jeho řešení“. (1) Nejprve se sestaví tým odborníků, v němž jsou zastoupeni experti na oblast, na data a na metody KDD; žádná z těchto rolí není zastupitelná. (2) Pak se specifikuje problém, což znamená převést obchodní zadání do kontextu dobývání znalostí: „zvýšit zisk“ → „predikovat pravděpodobnost odchodu zákazníka“. (3) Následuje sběr všech dostupných dat včetně externích, která popisují prostředí (roční období, reklamní kampaně, počasí); tento krok může vést až k přeformulování problému. (4) Teprve potom se volí metoda či kombinace metod a k nim metody vizualizační.

(5) Data se předzpracují. (6) Vlastní dolování probíhá opakovaně, protože výsledky předchozích běhů ovlivňují parametry běhů následujících. (7) Nakonec se výsledky interpretují. Část z nich bývá nezajímavá nebo triviální, proto se hodí shrnout je do analytické zprávy; výstupem ovšem může být i provedení konkrétní akce.

1.3 Typy úloh dobývání znalostí

[♦ nad rámec slidů] Jakou znalost od dobývání vlastně čekáme, není samozřejmé: jednou má být přesná, jindy srozumitelná a potřeby především nová. Úlohy se proto dělí především podle toho, jaký vztah má nalezená znalost k cílovému konceptu:

Typ úlohy	Cíl	Charakter výsledku
Klasifikace / predikce	zařadit nové případy, resp. odhadnout budoucí vývoj entity	preferujeme <i>přesnost pokrytí</i> před jednoduchostí; hodně znalosti, obtížně interpretovatelné
Popis (charakterizace, profil)	najít dominantní strukturu či vztahy v datech	požadujeme <i>srozumitelnost</i> při plném pokrytí konceptu; méně znalosti a méně přesné
Hledání „nugetů“	najít zajímavou (novou, překvapivou) znalost	znalost <i>nemusí</i> pokrývat celý koncept; hodnotí se překvapivost

Rozdíl je nejlépe vidět na příkladech. Predikcí je předpověď počasí, ceny akcií nebo spotřeby

elektriny. Popisem je třeba profil bonitního klienta. A nugetem je neobvyklá kombinace zboží v košíku či podezřelý vzorec transakcí.

Deskriptivní vs. prediktivní úlohy

Deskriptivní (*descriptive*) úlohy charakterizují obecné vlastnosti dat. Patří k nim charakterizace a diskriminace, asociační a korelační analýza, shlukování, detekce odlehlých hodnot a analýza vývoje.

Prediktivní (*predictive*) úlohy naproti tomu provádějí nad daty inferenci za účelem predikce. Jsou to klasifikace s diskrétním cílovým atributem, regrese se spojitým cílovým atributem a predikce časových řad.

Ortogonalně k tomu se úlohy třídí podle dostupnosti cílové proměnné. Při *učení s učitelem* (*supervised*) cílová proměnná je, což je případ klasifikace a regrese. Při *učení bez učitele* (*unsupervised*) není, a sem spadá shlukování, asociační pravidla a detekce odlehlých hodnot. *Poloučené* úlohy (*semi-supervised*) leží mezi obojím: malá část dat je označovaná a velká nikoli.

Právě tímto dělením se řídí i kap. 3: okruh v ní jmenuje jednoho reprezentanta učení bez učitele s transakčními daty, totiž asociační pravidla, dále učení s učitelem a konečně učení bez učitele s metrickými daty, tedy klastrovou analýzu.

1.4 Typy dat

Metodu nelze volit bez ohledu na to, jak data vypadají. Třídí se ze dvou hledisek. Prvním z nich je **typ atributu**, tedy škála, na níž se hodnoty měří; ta určuje, jaké operace jsou nad hodnotami vůbec smysluplné, a tím i to, jaké metody lze použít:

Škála	Operace	Příklad	Statistika
Nominální (<i>nominal</i>)	$=, \neq$	barva, druh ovoce	modus, četnosti
Ordinální (<i>ordinal</i>)	$=, <$	známka, {nízký, střední, vysoký}	medián, kvantily
Intervalová (<i>interval-scaled</i>)	$+, -$	teplota ve °C, datum	průměr, směr. odchylka
Poměrová (<i>ratio-scaled</i>)	$\times, /$	příjem, hmotnost, četnost	geometrický průměr

U intervalové škály je klíčové, že *intervaly mají v celém rozsahu stejnou důležitost*: rozdíl mezi věkem 10 a 20 let je týž jako rozdíl mezi 40 a 50 lety. Poměrová škála má navíc absolutní nulu, takže u ní dává smysl i poměr dvou hodnot. Poměrové atributy s exponenciálním chováním, jako je počet mikroorganismů Ae^{Bt} , se před dalším zpracováním logaritmují a pak se s nimi zachází jako s intervalovými.

Pro numerické metody se nečíselné škály převádějí. *Nominální* atribut o v hodnotách se rozepíše na v binárních atributů, z nichž odpovídající kategorie dostane hodnotu 1 a ostatní 0. *Ordinální* atribut se běžně zpracovává jako intervalový.

Druhým hlediskem je **struktura datového souboru**, tedy to, co vlastně tvoří jeden záznam:

- **Tabulková, čili relační data** mají tvar objekty $\times$ atributy a jsou standardním vstupem většiny metod.
- **Transakční data** mají záznam proměnné délky, typickým příkladem je nákupní košík. Vstupují do asociačních pravidel.
- **Temporální data** (*temporal*) jsou třeba časové řady kurzů akcií. Typickou úlohou nad nimi je predikce budoucích hodnot a řada se kvůli ní transformuje do tabulky posuvným oknem: vstupy $y(t_0), y(t_1), y(t_2), y(t_3) \rightarrow$ výstup $y(t_4)$; $y(t_1), \dots, y(t_4) \rightarrow y(t_5)$ atd.
- **Prostorová data** (*spatial*) pocházejí z geografických informačních systémů. Vyznačují se implicitní relací sousedství, tedy tím, že hodnoty atributů sousedních objektů se od sebe příliš neliší; užitečná je zejména pro interpretaci.

- **Strukturní data** (*structural*) popisují objekt jeho stavbou; jsou to třeba chemické sloučeniny zapsané řetězcí SMILES nebo soubory faktů v Prologu. Úlohou pak bývá klasifikace látek podle struktury.
- **Textová a webová data, multimédia, grafy a sítě** vyžadují specifickou reprezentaci, ať už vektorový model, nebo grafové modely (viz kap. 6).

1.5 Metodologie dobývání znalostí

♦ **nad rámec slidů** Metodologie má poskytnout jednotný rámec a vést aplikace bez ohledu na obor podnikání, a umožnit tak, aby se zkušenosti z úspěšných projektů daly sdílet a přenášet. Jednotlivé metodologie vznikají dvěma cestami. Buď u velkých výrobců BI nástrojů, odkud pochází SEMMA od SAS Institute a 5A od SPSS, nebo jako neutrální, softwarově nezávislé standardy; zástupcem druhé cesty je CRISP-DM.

SEMMA (SAS Enterprise Miner) má pět fází. *Sample* vybere data pro modelování, tedy vzorkuje, imputuje a dělí na trénovací, testovací a validační množinu. *Explore* data vizuálně prozkoumá a zredukuje; sem patří odlehle hodnoty, vícerozměrné histogramy, shlukování a Kohonenovy mapy. *Modify* připraví objekty, hodnoty a proměnné, tedy vybírá, tvoří a transformuje. *Model* aplikuje vlastní metody: stromy, regresi a neuronové sítě. *Assess* nakonec vyhodnotí spolehlivost a užitečnost výsledků, přičemž platí, že *uživatel jim musí rozumět*.

CRISP-DM (*C*Ross-*I*ndustry *S*tandard *P*rocess for *D*ata *M*ining) je otevřený model procesu, neutrální oborově, nástrojově i aplikačně. Vývoj financovala iniciativa ESPRIT a konsorcium tvořily ISL (později SPSS, dnes IBM), Teradata, NCR, Daimler AG a pojišťovna OHRA. Fází je šest a jejich **pořadí není striktní**: výsledky jedné fáze určují volbu dalších kroků a některé kroky je nutné opakovat. Proces se proto zobrazuje jako cyklus, kolem něhož vede ještě vnější cyklus nasazení a monitoringu.

1. **Porozumění problému** (*Business Understanding*) stanoví obchodní cíle a kritéria úspěchu. Proveďte inventuru zdrojů, tedy lidí, informací, softwaru a výpočetní kapacity, sepíší se předpoklady a omezení, rizika, náklady a přínosy. Z toho se odvodí cíle dobývání znalostí a sestaví plán projektu.
2. **Porozumění datům** (*Data Understanding*) začíná sběrem počátečních dat a jejich popisem (množství, vlastnosti, formát). Následuje průzkum a vizualizace, tedy rozdělení klíčových atributů, vztahy dvojic, subpopulace a jednoduché statistiky. Fáze končí ověřením kvality dat.
3. **Příprava dat** (*Data Preparation*) zahrnuje výběr, čištění, konstrukci odvozených atributů, integraci, agregaci a formátování. Tyto kroky se provádějí iterativně a v různém pořadí.
4. **Modelování** (*Modeling*) začíná volbou techniky, a to s ohledem na její předpoklady, což si může vynutit další úpravy dat. Pak se navrhne test, sestaví model s různými parametry a model se posoudí.
5. **Vyhodnocení** (*Evaluation*) porovnává výsledky s *obchodními* kritérii úspěchu, schvaluje modely, reviduje celý proces a určuje další kroky.
6. **Nasazení** (*Deployment*) prezentuje výsledky ve formě, kterou uživatel snadno interpretuje a nasadí. Patří sem plán nasazení, monitoring a údržba a závěrečná zpráva s revizí projektu, v níž se dokumentují obtíže a slepé uličky.

ASUM-DM (IBM, 2015) reaguje na hlavní výtku vůči CRISP-DM, totiž na malou podporu projektového řízení. Jeho fáze se jmenují *Analyze*, *Design*, *Configure & Build*, *Deploy* a *Operate & Optimize*; průřezově přes ně vede *Project Management*.

Všechny tři metodologie popisují tentýž postup, liší se však původem, mírou neutrality a tím, kolik pozornosti věnují obchodní stránce věci a řízení projektu:

	SEMMA	CRISP-DM	ASUM-DM
Původ	SAS Institute	ESPRIT / konsorcium	IBM (2015)
Neutralita	vázáno na SAS EM	nástrojově neutrální	vázáno na IBM
Byznys fáze	chybí	ano (fáze 1 a 5)	ano
Projektové řízení	ne	slabé	explicitní, průřezové
Počet fází	5	6	5 + PM

1.6 Shrnutí ke zkoušce

- **KDD** je netriviální proces získávání implicitních, dosud neznámých a potenciálně užitečných informací z dat; probíhá interaktivně a iterativně.
- **Pět fází KDD** jde za sebou takto: výběr → předzpracování → transformace → dobývání → interpretace. Samotná příprava dat spotřebuje 60–80 % času.
- **Kořeny oboru** leží v umělé inteligenci (strojové učení), v databázích a ve statistice; čtvrtou složkou je potřeba podpořit rozhodování. **OLAP** odpovídá na otázky položené předem, kdežto KDD hledá vzory, o nichž nevíme.
- **Tři typy úloh** se liší tím, co se od výsledku žádá: klasifikace a predikce staví přesnost nad jednoduchost, popis požaduje srozumitelnost při plném pokrytí a hledání nugetů oceňuje překvapivost, i když nález koncept nepokrývá.
- Úlohy se dělí na **deskriptivní a prediktivní**, a nezávisle na tom podle dostupnosti cílové proměnné na **učení s učitelem, bez učitele a poloučené**.
- **Škály atributů** — nominální, ordinální, intervalová a poměrová — určují, jaké operace jsou přípustné, a tím i volbu metody. Nominální atribut se převádí na *v* binárních atributů, poměrový s exponenciálním chováním se logaritmuje.
- Z **metodologií** se čeká znalost SEMMA (Sample, Explore, Modify, Model, Assess), CRISP-DM (šest fází, jejichž pořadí není striktní, proces je cyklický) a ASUM-DM, což je CRISP-DM doplněný o projektové řízení.
- Nejčastější doplňující otázka zní: *proč pořadí fází CRISP-DM není striktní?* Protože výsledky fáze určují další kroky; zjištěná kvalita dat například vede zpět k přeformulování cíle.

2 Příprava dat, výběr atributů a analýza jejich relevance

Věta okruhu spojuje do jedné kapitoly dvě věci, které spolu souvisejí těsněji, než se na první pohled zdá. Tou první je *příprava dat*, tedy vše, co je nutné udělat, aby data byla vůbec zpracovatelná. Tou druhou je *výběr atributů s analýzou jejich relevance*, tedy otázka, jak z mnoha atributů vybrat právě ty, které nesou informaci. Druhá část je ve skutečnosti speciálním případem té první, protože i ona redukuje dimenzi dat. Okruh ji přesto jmenuje zvlášť, a to z praktického důvodu: právě u ní se očekává znalost konkrétních kritérií a schopnost je spočítat.

2.1 Proč je příprava dat klíčová

Předzpracování má *klíčový význam pro úspěch aplikace*. Zároveň je obtížné a časově náročné, takže se vyplatí přizvat doménového experta, který o datech ví to, co ze samotných čísel vyčíst nelze. Práce má tři cíle:

- z dostupných dat vybrat, případně teprve vytvořit, ta, která jsou pro řešenou úlohu relevantní;
- reprezentovat je ve formě vhodné pro zvolený algoritmus;
- dospět do stavu, kdy je výsledkem (*jedna*) datová tabulka obsahující hodnoty atributů objektů.

[♦ nad rámec slidů] Kvalitu dat posuzujeme podle několika rozměrů: *přesnost (accuracy)*, *úplnost (completeness)*, *konzistence*, *aktuálnost (timeliness)*, *důvěryhodnost* a *interpretovatelnost*. Zásada „garbage in, garbage out“ přitom platí bez výjimky. Sebelepší algoritmus na špatných datech vytvoří jen sebejistý nesmysl.

2.2 Čištění dat: chybějící hodnoty

Pro chybějící hodnoty se nabízejí čtyři základní strategie, seřazené zde od nejjednodušší k nejpřesnější.

1. **Ignorovat objekty** s libovolnou chybějící hodnotou. Postup je jednoduchý, ale při mnoha attributech tak ztratíme většinu dat. Navíc zavádí zkreslení všude tam, kde chybějící hodnoty nejsou náhodné.
2. **Nahradit novou hodnotou** „nevím“ (*I don't know*). Chybějící hodnota se tím stane plnohodnotnou kategorií, což se hodí tehdy, když už sama absence nese informaci: typickým případem je nevyplněný příjem u žádosti o úvěr.
3. **Nahradit existující hodnotou atributu**. Nabízí se nejčastější hodnota (modus, u numerických atributů průměr či medián), volba úměrná poměru všech hodnot, tedy náhodné losování podle empirického rozdělení, anebo prostě libovolná hodnota.
4. **Doplnit hodnotu pomocí modelu**. Naučíme prediktivní model (regrese, rozhodovací strom, *k*-NN), který chybějící atribut předpovídá z ostatních. Je to volba nejpřesnější a současně výpočetně nejnáročnější a riskuje se u ní, že mezi atributy zavlečeme umělou závislost.

[♦ nad rámec slidů] Terminologicky se rozlišují tři situace. MCAR (*missing completely at random*) znamená zcela náhodné chybení, MAR (*missing at random*) chybení závislé na pozorovaných attributech a MNAR (*missing not at random*) chybení závislé na nepozorované hodnotě samotné. Jen u MCAR je prosté vypuštění nezkreslené.

Některé metody si s chybějícími hodnotami poradí samy. C4.5 je při konstrukci stromu ignoruje, protože kritérium dělení počítá jen ze známých hodnot, a při klasifikaci je odhaduje z ostatních vzorů. CART chybějící hodnoty do rozhodovacího kritéria nezapočítává a naivní Bayes umí klasifikovat i vzory popsané nekompletně (kap. 3).

2.3 Čištění dat: šum a odlehlé hodnoty

Šum (noise) je náhodná chyba nebo rozptyl v měřené veličině. Vyhlazuje se čtyřmi technikami. **Binning** rozdělí hodnoty do košů a nahradí je průměrem, mediánem nebo hranicí koše. **Regrese**

nahradí hodnoty predikcí regresní funkce. **Shlukování** označí za odlehlé ty hodnoty, které padnou mimo shluky. A **kombinace stroje a člověka** předloží podezřelé hodnoty ke kontrole.

Rozpětí, kvartily, mezikvartilové rozpětí, odlehlá hodnota

Rozpětí (*range*) množiny dat je rozdíl nejvyšší a nejnižší hodnoty.

Kvartily Q_1, Q_2, Q_3 dělí data na čtyři části: 25 % pozorování má hodnotu menší než dolní kvartil Q_1 , 50 % pod mediánem Q_2 a 75 % pod horním kvartilem Q_3 .

Mezikvartilové rozpětí (*interquartile range*): $IQR = Q_3 - Q_1$.

Odehlá hodnota (*outlier*) je extrémní hodnota ležící mimo celkový vzorec dat. Podle běžného pravidla je odehlá každá hodnota, pro kterou platí

$$x > Q_3 + 1,5 \cdot IQR \quad \text{nebo} \quad x < Q_1 - 1,5 \cdot IQR.$$

Kvartily se pro n vzestupně seřazených hodnot počítají u diskretních dat takto. Pro Q_1 vydělíme n čtyřmi: vyjde-li $n/4$ jako celé číslo, bereme střed mezi odpovídajícím a následujícím členem, a nevyjde-li, zaokrouhlíme nahoru a vezmeme odpovídající člen. Pro Q_3 postupujeme analogicky s podílem $3n/4$. U spojitých dat se místo zaokrouhlení interpoluje.

Příklad — krabicový graf a odlehlé hodnoty

Hladina glukózy v krvi u 30 žen (mmol/l), stonkový diagram (klíč: 2 | 1 znamená 2,1):

2 | 2 2 3 3 5 7 (6)

3 | 1 2 6 7 7 7 8 8 8 8 9 9 9 (13)

4 | 0 0 0 0 4 5 6 7 8 (9)

5 | 1 5 (2)

Zde je $n = 30$, takže $30/4 = 7,5$ zaokrouhlíme na 8; osmé číslo je 3,2, a tedy $Q_1 = 3,2$. Medián vychází $Q_2 = 3,8$. Pro Q_3 počítáme $3 \cdot 30/4 = 22,5$, zaokrouhlíme na 23, a protože 23. číslo je 4,0, dostáváme $Q_3 = 4,0$.

Odtud $IQR = 4,0 - 3,2 = 0,8$, dolní mez $= 3,2 - 1,5 \cdot 0,8 = 2,0$ a horní mez $= 4,0 + 1,5 \cdot 0,8 = 5,2$.

Odehlá hodnota je 5,5. Nejvyšší hodnota, která odehlá není, je 5,1: v krabicovém grafu (*box plot*) k ní vede vous, zatímco odehlá hodnota se vyznačí křížkem.

Násobek 1,5 je pouhá konvence; s násobkem 1 dostaneme kritérium přísnější. Jak se meze počítají v praxi, ukazují dva soubory měření. Pro želví samce s $Q_1 = 400$ g a $Q_3 = 580$ g je $IQR = 180$ a meze vycházejí $400 - 180 = 220$ a $580 + 180 = 760$, takže žádná z hodnot 400, 260, 550 a 640 g není odehlá a největší hmotnost, kterou ještě za outlier neoznačíme, je 760 g. U samic je $Q_1 = 260$, $Q_3 = 340$ a $IQR = 80$, meze tedy vycházejí 180 a 420 a odehlé jsou hodnoty 170 g a 440 g.

2.4 Integrace dat

♦ **nad rámec slidů** Při *integraci* (*data integration*) spojujeme více zdrojů do jedné tabulky a narazíme přitom na čtyři typické problémy. **Problém identity entity** nastává, když atribut `cust_id` v jedné databázi odpovídá atributu `customer_number` v druhé; k jeho řešení jsou potřeba metadata a párování záznamů. **Redundance** znamená, že jeden atribut je odvoditelný z jiných, například roční příjem = 12 × měsíční; u numerických atributů ji odhalí korelační analýza, u kategoriálních χ^2 test (viz odd. 2.8). **Konflikty hodnot** vznikají z různých jednotek, škál a kódování. A **duplicita** znamenají, že tentýž objekt je v datech vícekrát, pokaždé s drobně odlišnými hodnotami.

2.5 Transformace a standardizace atributů

V euklidovském prostoru se standardizace atributů doporučuje proto, aby všechny atributy mohly mít na výpočet vzdáleností *stejný vliv*. Bez ní rozhoduje ten atribut, který má náhodou největší rozsah. Pro body $x_i = (0,1; 20)$ a $x_j = (0,9; 720)$ vyjde

$$\text{dist}(x_i, x_j) = \sqrt{(0,9 - 0,1)^2 + (720 - 20)^2} = 700,000457,$$

takže vzdálenost je zcela ovládnuta rozdílem $720 - 20 = 700$ ve druhém atributu a první atribut na ni nemá prakticky žádný vliv. Standardizace nutí atributy do společného rozsahu hodnot.

Základní standardizační transformace

Nechť f je atribut a x_{if} jeho hodnota u i -tého objektu.

Desítkové škálování (*decimal scaling*) do $[-1, 1]$: dělíme hodnoty nejmenší mocninou 10, která udrží všechny transformované hodnoty v $[-1, 1]$.

Min-max normalizace (*range standardization*) do $[0, 1]$:

$$\text{range}(x_{if}) = \frac{x_{if} - \min(f)}{\max(f) - \min(f)},$$

do $[-1, 1]$ pak $\text{range}_{[-1,1]}(x_{if}) = 2 \cdot \text{range}(x_{if}) - 1$.

Standardizace podle průměrné absolutní odchylky dá nulový průměr a jednotkovou střední absolutní odchylku:

$$m_f = \frac{1}{n}(x_{1f} + \dots + x_{nf}), \quad s_f = \frac{1}{n}(|x_{1f} - m_f| + \dots + |x_{nf} - m_f|), \quad z(x_{if}) = \frac{x_{if} - m_f}{s_f}.$$

Standardizace podle směrodatné odchylky (*std-score*) dá nulový průměr a jednotkovou korigovanou výběrovou směrodatnou odchylku:

$$\text{cors}_f = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{if} - m_f)^2}, \quad \text{std}(x_{if}) = \frac{x_{if} - m_f}{\text{cors}_f}.$$

Volba mezi oběma z-skóry se řídí jednoduchou úvahou. Střední absolutní odchylka je k odlehlým hodnotám *robustnější* než směrodatná, protože se odchylky neumocňují na druhou, takže odlehlé hodnoty stlačí ostatní data méně.

2.6 Diskretizace numerických atributů

Řada metod pracuje výhradně s kategoriálním vstupem: patří sem asociační pravidla, ID3, CHAID i bayesovská klasifikace s diskretními atributy. *Diskretizace* proto rozdělí rozsah numerického atributu na intervaly. Jednotlivé metody se od sebe liší ve čtyřech ohledech: *strategií* (dělí intervaly shora, nebo je slučují zdola), *počtem* intervalů, *typem* intervalů a *kritériem* kvality, kterým bývá minimální klasifikační chyba, entropie nebo χ^2 test.

- **Ekvidistantní** diskretizace (*equi-width*) rozdělí rozsah na intervaly stejné délky. Je nejjednodušší, zato citlivá na odlehlé hodnoty a na nerovnoměrné rozdělení.
- **Ekvifrekvenční** diskretizace (*equi-depth*) [♦ nad rámec slidů] vytváří intervaly se stejným počtem objektů.
- **S využitím informace o třídě** se hranice volí tak, aby intervaly vyšly co nejčistší. Kritériem je buď entropie, a pak se dělí rekurzivně jako v C4.5, nebo χ^2 , a pak se slučuje zdola jako v CHAID.

2.6.1 Fuzzy diskretizace

U běžné diskretizace jsou hranice intervalů ostré, takže dva objekty s téměř stejnou hodnotou mohou skončit v různých kategoriích. *Fuzzy diskretizace* hranice „rozostří“: jedné numerické hodnotě pak mohou odpovídat *dvě* diskretizované hodnoty, přičemž součet příslušných charakteristických funkcí je 1. Vznikají tak „fuzzy“-objekty s vahou < 1 ; váha fuzzy-objektu je dána *součinem* hodnot charakteristických funkcí všech atributů:

$$w = \prod_j \mu_j.$$

Za tuto jemnost se ale platí. Hrozí ztráta informace a *kontradikce*, tedy situace, kdy dva objekty mají stejné hodnoty diskretizovaných atributů, ale patří do různých tříd.

Procedura *Interval* (fuzzy diskretizace)

1. Má-li posloupnost hodnot stejnou třídu, vytvoř z ní interval s charakteristickou funkcí rovnou 1 na celém intervalu.
2. Pro každý interval Int_i patřící do třídy „?“:
 - patří-li sousední intervaly Int_{i-1} a Int_{i+1} do *téže* třídy, spoj je do jednoho intervalu $Int_{i-1} \cup Int_i \cup Int_{i+1}$ (charakteristická funkce = 1 na celém intervalu);
 - jinak vytvoř *dva fuzzy-intervaly*: spojením $Int_{i-1} \cup Int_i$ s charakteristickou funkcí rovnou 1 na $[L_{i-1}, U_{i-1}]$ a lineárně klesající $\frac{U_{i+1}-x}{U_{i+1}-L_{i-1}}$ na přechodové oblasti, a analogicky spojením $Int_i \cup Int_{i+1}$ s funkcí lineárně rostoucí $\frac{x-L_{i-1}}{U_{i+1}-L_{i-1}}$.
3. Zajisti spojitě pokrytí celého rozsahu (mezery mezi intervaly se chápou jako intervaly třídy „?“).

2.6.2 Seskupování hodnot kategoriálních atributů (KEX)

Má-li kategoriální atribut příliš mnoho hodnot, vyplatí se je seskupit. **Cílem** je vytvořit takové grupy $A(Grp)$, aby se podmíněná pravděpodobnost $P(Class | A(Grp))$ *významně* lišila od apriorní pravděpodobnosti $P(Class)$, a vytvořit jich tolik, kolik je tříd, plus jednu navíc pro kód „?“.

Algoritmus seskupování hodnot (KEX)

Hlavní cyklus:

1. vytvoř uspořádaný seznam hodnot uvažovaného atributu;
2. pro každou hodnotu: (a) spočítej četnosti výskytu objektů s touto hodnotou v jednotlivých třídách; (b) přiřaď hodnotě kód třídy procedurou *Assign*;
3. vytvoř grupy hodnot procedurou *Group*.

Procedura *Assign*: patří-li pro danou hodnotu všechny objekty do téže třídy, přiřaď hodnotě kód této třídy; jinak — liší-li se rozdělení tříd pro danou hodnotu *významně* (podle χ^2 testu) od apriorního rozdělení tříd — přiřaď kód nejčastější třídy, a pokud ne, přiřaď kód „?“.

Procedura *Group*: vytvoř grupu z hodnot, kterým byl přiřazen stejný kód třídy.

2.7 Redukce dat

2.7.1 Příliš mnoho objektů

Roste-li počet objektů, dávkové zpracování se stává problematickým. Řešení jsou tři. Buď se pracuje jen se **vzorkem** objektů, nebo se data uloží tak, aby k nim šlo přistupovat bez nutnosti mít je celá v operační paměti, anebo se postaví **více modelů nad podmnožinami** a ty se pak zkombinují do ensemblu (kap. 3).

Klíčovou vlastností vybraných dat je jejich **reprezentativnost**. Vzorek má zachytit povahu dat co nejlépe, a to zejména co se týče souhlasu rozdělení hodnot atributů na vzorku a na celých datech.

Vlastní kapitolou jsou **nevyvážené třídy** v původních datech, které vedou k tendenci preferovat majoritní třídu. Řešení jsou dvě: přiřadit různým typům chyb různé *váhy*, tedy použít cenově citlivé učení, nebo vybírat objekty z různých tříd s *různou pravděpodobností*, což znamená převzorkovat minoritní třídu nebo podvzorkovat majoritní.

2.7.2 Příliš mnoho atributů

Trápí-li nás naopak počet atributů, vedou z toho dvě principiálně odlišné cesty.

- **Redukce transformací** vytvoří z existujících atributů *méně nových*, typicky jako jejich lineární kombinace; hlavním zástupcem je analýza hlavních komponent (PCA). Nevýhody jsou dvě: je nutné znát hodnoty *všech* původních atributů a nové atributy nemusí mít jasnou interpretaci.
- **Redukce selekcí** (*feature selection*) z existujících atributů vybere jen ty nejdůležitější. Interpretace se tím zachovává a nepotřebné atributy se pak vůbec nemusí měřit.

[♦ nad rámec slidů] PCA hledá ortogonální směry maximálního rozptylu, tedy vlastní vektory kovarianční matice s největšími vlastními čísly. Je to transformace *nesupervizovaná*, a proto může zahodit směr, který je pro predikci klíčový, ale má malý rozptyl.

2.8 Analýza relevance atributů: filtry a wrappery

Tím se dostáváme k druhé polovině okruhu. Hledáme atributy, které nejlépe přispívají ke správné klasifikaci objektů, a existují na to dva principiálně různé přístupy.

Filtry vs. wrappery

Filtrační metody (*filter*) spočítají pro každý atribut charakteristiku odrážející jeho přínos ke klasifikaci, atributy podle ní seřadí a vyberou stanovený počet nejlepších. Charakteristika je *nezávislá* na následně použitém modelu, a metoda je proto rychlá.

Obalové metody (*wrapper*) postaví pomocí algoritmu strojového učení model nad *podmnožinou* atributů a ten model vyhodnotí; nakonec se použije nejlepší z postavených modelů i s odpovídající podmnožinou atributů. Metoda je výpočetně drahá, zato zohledňuje interakci atributů s *konkrétním* modelem.

Prostor podmnožin se prohledává dvěma směry:

- „zdola nahoru“ (*bottom-up, dopředná selekce*) začíná od modelů nad jednotlivými atributy a postupně přidává další;
- „shora dolů“ (*top-down, zpětná eliminace*) začíná od modelu nad úplnou množinou atributů a postupně odstraňuje nepotřebné.

Filtry mají **zásadní omezení**: hodnotí každý atribut *samostatně*, takže vzájemný vliv několika atributů na přesnost klasifikace zachycují jen obtížně. Kritérium sice lze počítat i pro celou *množinu* atributů, tedy pro všechny jejich kombinace, ale kombinatorická explozivnost to prakticky omezuje na hladové dopředné či zpětné prohledávání. [♦ nad rámec slidů] Praktickým důsledkem je, že filtr propustí dvojici silně korelovaných atributů, protože oba dostanou vysoké skóre, přestože druhý z nich nepřináší nic nového. A naopak zahodí atribut, který je užitečný jen *v kombinaci* s jiným, což je typicky případ XOR-vztahu.

2.8.1 Kritéria na kontingenčních tabulkách

Vztah vstupního atributu A s hodnotami $v_1, \dots, v_R$ a cílového atributu C se třídami $v_1, \dots, v_S$ se popíše kontingenční tabulkou o četnostech a_{kl} kombinace $A(v_k) \wedge C(v_l)$. Její marginály jsou

$$r_k = \sum_{l=1}^S a_{kl}, \quad s_l = \sum_{k=1}^R a_{kl}, \quad n = \sum_{k=1}^R \sum_{l=1}^S a_{kl}.$$

Z tabulky pak odhadneme pravděpodobnosti jako $P(A(v_k) \wedge C(v_l)) = a_{kl}/n$, $P(A(v_k)) = r_k/n$ a $P(C(v_l)) = s_l/n$. Očekávaná četnost při nezávislosti je $e_{kl} = r_k s_l / n$.

χ^2 jako kritérium relevance

$$\chi^2 = \sum_{k=1}^R \sum_{l=1}^S \frac{(a_{kl} - e_{kl})^2}{e_{kl}} = n \sum_{k=1}^R \sum_{l=1}^S \frac{(a_{kl} - \frac{r_k s_l}{n})^2}{r_k s_l} \quad (\text{čím větší, tím lepší}).$$

Pro ruční výpočet se hodí alternativní tvar $\chi^2 = \sum_k \sum_l \frac{a_{kl}^2}{e_{kl}} - n$.

Jako statistický test: při platnosti nulové hypotézy o nezávislosti

$$H_0 : P(A = v_k \wedge C = v_l) = P(A = v_k) P(C = v_l) \quad \forall k, l$$

má statistika χ^2 rozdělení s $(R-1)(S-1)$ stupni volnosti. Platí-li $\chi^2 \geq \chi^2_{(R-1)(S-1)}(\alpha)$, nulovou hypotézu na hladině významnosti α zamítáme a přijímáme alternativní hypotézu o *závislosti*.

Příklad — χ^2 test na čtyřpolní tabulce

		Poskytnutí úvěru		
		ANO	NE	r_k
Výše příjmu	vysoký	50	0	50
	nízký	30	40	70
s_l		80	40	120

Očekávané četnosti vycházejí $e_{11} = 50 \cdot 80/120 = 33,3$; $e_{12} = 50 \cdot 40/120 = 16,7$; $e_{21} = 70 \cdot 80/120 = 46,7$ a $e_{22} = 70 \cdot 40/120 = 23,3$. Rozdíly proti skutečnosti jsou tedy $+16,7$, $-16,7$, $-16,7$ a $+16,7$, což dává hodnotu statistiky $\chi^2 = 42,857$.

Na hladině významnosti $\alpha = 0,05$ je kritická hodnota $\chi^2_{(1)}(0,05) = 3,84$. Protože $42,857 > 3,84$, platí, že **mezi výší příjmu a poskytnutím úvěru je závislost**.

Fisherův exaktní test

χ^2 test lze použít jen pro dostatečně velké četnosti: musí platit $(r_k s_l)/n \geq 5$ pro všechna k, l . Pro čtyřpolní tabulky s malými četnostmi se proto používá *Fisherův test*. Pravděpodobnost, že čtyřpolní tabulka má při daných marginálech r_1, r_2, s_1, s_2 právě četnosti a_{kl} , je

$$p = \frac{r_1! r_2! s_1! s_2!}{n! a_{11}! a_{12}! a_{21}! a_{22}!} = \frac{\binom{s_1}{a_{11}} \binom{s_2}{a_{12}}}{\binom{n}{r_1}}.$$

Pravděpodobnosti se pak sečtou přes všechny uvažované hodnoty skutečných četností při daných marginálech (nechť $m = \min(r_1, s_1)$):

$$P = \sum_{i=0}^m \frac{r_1! r_2! s_1! s_2!}{n! (a_{11} - i)! (a_{12} + i)! (a_{21} + i)! (a_{22} - i)!}.$$

Platí-li $P \leq \alpha$, nulová hypotéza se na hladině α zamítá.

Příklad. Pro tabulku s marginály $r = (12, 18)$, $s = (6, 24)$ a $n = 30$ vychází $P_0 = 0,031$ a $P_1 = 0,173$, takže $P = P_0 + P_1 = 0,204 > 0,05$ a **nulovou hypotézu o nezávislosti přijímáme**. (Kontrola: součet $P_0, \dots, P_6 = 0,031 + 0,173 + 0,340 + 0,302 + 0,128 + 0,024 + 0,002 = 1,000$.)

2.8.2 Kritéria založená na entropii

Druhou rodinu filtračních kritérií dodává teorie informace.

Entropie, vzájemná informace, informační míra závislosti

Entropie atributu A jako vážený průměr entropií jeho hodnot (čím *menší*, tím lepší):

$$H(A) = \sum_{k=1}^R \frac{r_k}{n} H(A(v_k)), \quad H(A(v_k)) = - \sum_{l=1}^S \frac{a_{kl}}{r_k} \log_2 \frac{a_{kl}}{r_k}.$$

Vzájemná informace (*mutual information*):

$$MI(A, C) = \sum_{k=1}^R \sum_{l=1}^S \frac{a_{kl}}{n} \log_2 \frac{n a_{kl}}{r_k s_l}.$$

Informační míra závislosti (*information measure of dependence*) je normalizovaná vzájemná informace (čím *větší*, tím lepší):

$$ID(A, C) = \frac{MI(A, C)}{H(C)} = \frac{MI(A, C)}{-\sum_{l=1}^S \frac{s_l}{n} \log_2 \frac{s_l}{n}}.$$

Normalizace v ID není kosmetická. Samotná MI systematicky zvýhodňuje atributy s mnoha hodnotami, takže v limitě vychází jako „nejinformativnější“ atribut prostý identifikátor objektu. Totéž zvýhodnění řeší poměrný informační zisk v C4.5 (kap. 3).

2.9 Vztahy mezi atributy: regrese a korelace

Kontingenční tabulky popisují vztah *kategoriálních* atributů. U numerických atributů se místo nich používá regresní a korelační analýza, přičemž každá z nich odpovídá na jinou otázku. *Korelační analýza* říká, zda mezi dvěma numerickými veličinami lineární vztah vůbec *existuje*; *lineární regrese* určuje jeho *parametry*.

K **regresní analýze** vedou čtyři důvody. (1) Měření výstupu je nákladné, takže jej chceme predikovat ze snadno dostupných vstupů. (2) Vstupy jsou známy dříve než výstup, takže z nich lze výstup odhadnout dopředu. (3) Řízené vstupní hodnoty pomohou najít správné chování výstupu. (4) Mezi vstupem a výstupem může být kauzální vztah, který chceme najít.

Lineární regrese jedné proměnné metodou nejmenších kvadrátů

Vzory $[x_1, y_1], \dots, [x_p, y_p]$, regresní rovnice $y = \beta x + \alpha$, kvadratická chyba

$$E = \sum_{i=1}^p \varepsilon_i^2 = \sum_{i=1}^p (y_i - \hat{y}_i)^2 = \sum_{i=1}^p (y_i - \beta x_i - \alpha)^2.$$

Položíme parciální derivace nule:

$$\frac{\partial E}{\partial \alpha} = -2 \sum_i (y_i - \beta x_i - \alpha) = 0, \quad \frac{\partial E}{\partial \beta} = -2 \sum_i (y_i - \beta x_i - \alpha) x_i = 0,$$

odkud po úpravě $\alpha = \bar{y} - \beta \bar{x}$ a

$$\beta = \frac{\sum_{i=1}^p (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^p (x_i - \bar{x})^2} \quad (\text{tj. výběrová kovariance dělená výběrovým rozptylem } x).$$

Predikce: $\hat{y} = \beta x + \alpha$.

Korelační koeficient

Posouzení „množství“ lineární závislosti:

$$\rho(x, y) = \frac{\sigma_{xy}}{\sqrt{\sigma_x^2 \sigma_y^2}} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}, \quad \sigma_{xy} = \frac{1}{n-1} \sum_i (x_i - \bar{x})(y_i - \bar{y}),$$

$$\sigma_x^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2, \quad \sigma_y^2 = \frac{1}{n-1} \sum_i (y_i - \bar{y})^2.$$

Platí $\rho \in [-1, 1]$, přičemž $|\rho| = 1$ znamená přesný lineární vztah a $\rho = 0$ žádný *lineární* vztah. Nulová korelace ovšem neznamená žádnou závislost: ρ nezachytí vztahy nelineární.

Vícerozměrná lineární regrese předpokládá lineární závislost vysvětlované veličiny Y na několika vysvětlujících veličinách $X_1, \dots, X_m$, tedy $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m$, maticově $\hat{Y} = X\beta$. Zde je X rozšířená matice vstupních vzorů, jejíž první sloupec tvoří jedničky, a $\beta = (\beta_0, \dots, \beta_m)$. Kvadratická odchylka je $E = (Y - X\beta)^\top (Y - X\beta)$ a z podmínky $\partial E / \partial \beta = 0$ dostáváme normální rovnice $X^\top X \beta = X^\top Y$, tedy

$$\beta = (X^\top X)^{-1} X^\top Y.$$

U složitých praktických úloh je tento výpočet nákladný, a proto se sahá po aproximativních (iterativních) řešeních.

Nelineární regrese předpokládá složitější funkční závislost, například kvadratickou nebo exponenciální. Jejím nejdůležitějším případem je *logistická regrese*, která modeluje kategoriální (dvouhodnotový) výstup. Protože jde současně o klasifikátor, je zpracována v kap. 3, odd. 3.2.5.

2.10 Shrnutí ke zkoušce

- **Cílem přípravy dat** je vybrat relevantní data a reprezentovat je ve formě vhodné pro zvolený algoritmus; výsledkem je *jedna* tabulka objekty $\times$ atributy.
- **Chybějící hodnoty** se řeší čtyřmi způsoby: ignorovat celý objekt, zavést novou hodnotu „nevím“, dosadit existující hodnotu (modus, poměrně podle rozdělení, nebo libovolně), anebo hodnotu doplnit modelem. U zkoušky se chce zdůvodnit, kdy je která volba vhodná: nese-li informaci už sama absence, volí se „nevím“.
- **Odlehle hodnoty** se určují z mezikvartilového rozpětí $IQR = Q_3 - Q_1$; odlehle je každá hodnota mimo interval $[Q_1 - 1,5 IQR, Q_3 + 1,5 IQR]$. Ke zkoušce patří výpočet kvartilů ze vzestupně seřazených dat podle pravidla $n/4$ a $3n/4$ se zaokrouhlením nahoru a nakreslení box plotu.
- **Standardizaci** lze provést desítkovým škálováním, min-max normalizací do $[0, 1]$ i do $[-1, 1]$, z-skórem podle střední absolutní odchylky (ten je robustnější) nebo std-skórem podle směrodatné odchylky. Motivací je, že bez standardizace ovládne vzdálenost atribut s největším rozsahem.
- **Diskretizace** může být ekvidistantní, ekvifrekvenční nebo využívající informaci o třídě (entropie, χ^2). **Fuzzy diskretizace** hranice rozostří: jedné hodnotě odpovídají dvě kategorie se součtem charakteristických funkcí 1 a váha fuzzy-objektu je součin $\prod_j \mu_j$; rizikem je ztráta informace a kontradikce. **KEX** seskupuje hodnoty kategoriálního atributu podle χ^2 testu proti apriornímu rozdělení tříd, a to procedurami *Assign* a *Group*.
- **Redukce dat** má dvě větve. Je-li objektů příliš mnoho, sáhne se po vzorkování (a hlídá se reprezentativnost!), po efektivním uložení dat nebo po více modelech nad podmnožinami; nevyvážené třídy se řeší váhami chyb nebo různými pravděpodobnostmi výběru. Je-li příliš mnoho atributů, volí se buď *transformace* (PCA dá méně nových atributů jako lineární kombinace, ale je nutné znát všechny původní a chybí interpretace), nebo *selekce* (podmnožina původních atributů, u níž interpretace zůstává).
- **Filtry a wrappery** se liší v tom, co hodnotí. Filtr počítá charakteristiku pro každý atribut samostatně a je nezávislý na modelu, takže je rychlý, ale nezachytí interakce; wrapper staví model nad podmnožinou atributů a vyhodnocuje jej, takže je drahý, zato interakce zohledňuje. Prohledávat lze *bottom-up* (přidáváním) nebo *top-down* (odebíráním).
- **Filtračními kritérii** jsou χ^2 (větší = lepší), entropie $H(A)$ (menší = lepší), $MI(A, C)$ a $ID(A, C) = MI/H(C)$ (větší = lepší). Normalizaci v ID je nutné umět zdůvodnit: bez ní jsou zvýhodněny atributy s mnoha hodnotami.
- χ^2 test má $(R - 1)(S - 1)$ stupňů volnosti a podmínku $r_{kSI}/n \geq 5$; pro malé četnosti se místo něj použije **Fisherův exaktní test**. Výpočet na čtyřpolní tabulce se u zkoušky prochází celý (příklad s úvěrem dá $\chi^2 = 42,857 > 3,84$, a tedy závislost).
- **Regrese a korelace**: koeficienty se hledají metodou nejmenších kvadrátů, kde β vyjde jako kovariance dělená rozptylem a $\alpha = \bar{y} - \beta \bar{x}$; vícerozměrně je $\beta = (X^\top X)^{-1} X^\top Y$. Korelační koeficient

$\rho = \sigma_{xy}/(\sigma_x\sigma_y)$ měří jen závislost *lineární*.

3 Metody pro dobývání znalostí

Tohle je nejobsáhlejší věta celého okruhu a jako jediná jmenuje rovnou tři skupiny metod. Nejde o konkurenční přístupy k téže úloze, nýbrž o odpovědi na tři různé situace, do kterých se analytik dostává podle toho, jaká data má před sebou. *Asociační pravidla* se hodí na transakční data, v nichž žádný atribut nehraje roli cíle. *Učení s učitelem* naopak předpokládá, že cílový atribut existuje a má se předpovídat. *Klastrová analýza* pracuje s metrickými daty, kde cíl opět chybí a struktura se hledá v podobnosti samotných objektů. Podle těchto tří situací je kapitola rozdělena na tři oddíly.

3.1 Asociační pravidla a frekventované vzory

3.1.1 Analýza nákupního košíku

Analýza nákupního košíku (*market basket analysis*, MBA) je archetypální úloha celé oblasti a shrne ji jediná otázka: „které položky se v košíku vyskytují společně?“ Výsledky mají podobu pravidel, takže se dají použít okamžitě a bez dalšího překladu. Obchodník podle nich plánuje prodejnu a její rozvržení, nabízí kupony a časově omezené slevy nebo produkty „balíčkuje“.

Základní pojmy jsou dva a jsou jednoduché. **Položka** (*item*) je nabízený produkt nebo služba, **transakce** obsahuje jednu nebo více položek. Nad nimi se staví **frekvenční tabulka**, která udává počet společných výskytů každých dvou položek v transakcích; na diagonále pak nese počet transakcí obsahujících danou položku.

Příklad — frekvenční tabulka

Uvažujme následující transakce v obchodě s potravinami:

Zák.	Položky		chléb	máslo	mléko	káva
1	chléb, máslo	chléb	4	3	1	2
2	mléko, chléb, máslo	máslo	3	4	1	2
3	chléb, káva	mléko	1	1	1	0
4	chléb, máslo, káva	káva	2	2	0	3
5	káva, máslo					

I z takto malé tabulky je povaha prodeje vidět okamžitě: *chléb a máslo se kupují spolu*, kdežto *mléko se nikdy nekupuje s kávou*.

Nalezené vztahy se zapisují jako pravidla ve tvaru *IF podmínka THEN závěr*. Od dobrého asociačního pravidla se přitom žádají dvě vlastnosti:

- **snadno pochopitelné** pravidlo je takové, u kterého lze nalezený vztah snadno ověřit;
- **použitelné** pravidlo obsahuje užitečnou informaci, která vede k dalším zásahům;

a naopak se dvěma vlastnostem chceme vyhnout. Pravidlo nemá být **triviální**, tedy takové, jehož výsledek už každý zná, ani **nevysvětlitelné**: pro takové pravidlo neexistuje vysvětlení a nevede k žádné akci.

3.1.2 Support, confidence a lift

Pravidel se v datech najdou tisíce, a tak je potřeba měřit, která z nich stojí za pozornost. Slouží k tomu tři míry, a každá se ptá na něco jiného.

Support, confidence, lift

Support (podpora) říká, jak často lze pravidlo použít:

$$\text{sup}(R) = \frac{\text{počet transakcí obsahujících } i \text{ a } j}{\text{počet všech transakcí}} \cdot 100\%.$$

Confidence (spolehlivost) říká, jak spolehlivé jsou výsledky pravidla:

$$\text{conf}(R) = \frac{\text{počet transakcí obsahujících } i \text{ a } j}{\text{počet transakcí obsahujících } i} \cdot 100 \%.$$

Lift (zdvih) měří, kolikrát je lepší použít pravidlo pro predikci, než jen předpokládat jeho výsledek:

$$\text{lift}(R) = \frac{p(i \wedge j)}{p(i)p(j)}.$$

Je-li $\text{lift} < 1$, je pravidlo pro predikci *horší než náhodná volba*. V takovém případě může dát lepší pravidlo **negace závěru**, tedy tvar **IF podmínka THEN NOT závěr**.

Příklad — support, confidence, lift

Počítejme nad daty z předchozího příkladu, tedy nad 5 transakcemi:

Pravidlo 1: „kdo kupuje chléb, kupuje i máslo.“ $\text{sup} = 3/5 = 60 \%$, $\text{conf} = 3/4 = 75 \%$.

Pravidlo 2: „kdo kupuje kávu, kupuje i máslo.“ $\text{sup} = 2/5 = 40 \%$, $\text{conf} = 2/3 = 66 \%$.

Pravidlo 3: „kdo kupuje mléko, kupuje i máslo.“ $\text{sup} = 1/5 = 20 \%$, $\text{conf} = 1/1 = 100 \%$,
 $\text{lift} = \frac{1/5}{(1/5) \cdot (4/5)} = \frac{5}{4} = 1,25$.

Pravidlo 3 vypadá na první pohled nejlépe, protože má *stoprocentní* spolehlivost. Jenže jeho support je jen 20 %, takže se opírá o jedinou transakci. A lift 1,25 dodává, že zdvih proti náhodě je malý: máslo kupuje 80 % zákazníků tak jako tak. To je základní past celé metody, totiž že *vysoká confidence sama o sobě nestačí*.

3.1.3 Volba položek, taxonomie a virtuální položky

Než se dá cokoli dolovat, musí se rozhodnout, co vlastně bude položkou. Transakční data bývají méně kvalitní než jiné zdroje a vyžadují rozsáhlejší předzpracování, a navíc se relevance položek mění v čase. Volba úrovně zpracování je proto kompromisem: čím konkrétnější položky, tím přímo použitelnější výsledky, ale tím rychleji roste i počet kombinací a tím obtížněji se udrží dostatečně vysoký support.

Uspořádanou odpověď na tento kompromis je **taxonomie**, tedy hierarchie kategorií. Zpočátku se používají obecnější položky a teprve později se generují pravidla pro položky konkrétní, a to jen z těch transakcí, které je obsahují. Aby byly výsledky použitelné, mají se uvažované položky vyskytovat v přibližně stejném počtu transakcí. Proto se *vzácné položky posunou výše* v taxonomii, kde se vyskytují relativně častěji, kdežto *častější položky se nechají na nižších úrovních*, aby nejfrekventovanější položky pravidlům nedominovaly.

Virtuální položky jdou ještě dál a přesahují rámec tradiční taxonomie: rozostřují hranici mezi typy produktů (například značky, řekněme Calvin Klein) a mohou nést informaci o transakci samotné. Ta bývá buď anonymní, jako den v týdnu nebo čas, nebo podepsaná, tedy informace o zákazníkovi a jeho chování. Za tuto volnost se ale platí *redundancí* v pravidlech, a to dvojím způsobem. Buď virtuální položka odpovídá právě jedné položce z taxonomie („IF produkt_Škoda THEN Škoda“), nebo se v pravidle vyskytne virtuální i obecná položka současně: místo „IF Fabia THEN stan“ vyjde „IF produkt_Škoda AND malé auto THEN stan“.

Virtuální položky umožňují i **srovnání prodejen**. Zavede se virtuální položka, která označí typ prodejny, kde transakce proběhla, a neodpovídá tedy žádnému produktu ani službě. Postup má čtyři kroky: (1) poskytnout transakční data z nových prodejen za určitou dobu; (2) poskytnout přibližně stejné množství dat z již existujících prodejen; (3) aplikovat MBA na oba soubory transakcí; (4) zaměřit se na pravidla obsahující virtuální položky.

Zbývá se zbavit položek, které se objevují příliš zřídka. **Prořezávání podle minimálního supportu** málo frekventované položky eliminuje, protože podniknuté akce se musí týkat dostatečného počtu transakcí. Nabízejí se dvě možnosti: buď se vzácné položky eliminují a s nimi i odpovídající

pravidla, nebo se použije taxonomie a vytvoří se obecné položky, které musí splnit stanovený prah. Pozor na to, že proměnný minimální support vede ke *kaskádovému efektu*.

3.1.4 Algoritmus Apriori

Prohledávat všechny kombinace položek nelze, protože jich je exponenciálně mnoho. Apriori z toho vybrusí jedinou vlastnost, kterou frekventované kombinace mají a která umožňuje většinu prostoru vůbec nenavštívit.

Frekventovaná množina položek a Apriori princip

Frekventovaná množina položek (frequent itemset) je kombinace (konjunkce) kategorií, která na datech dosáhne předem stanovené frekvence, tedy supportu *minsup*.

Klíčovou ideou Apriori je **vlastnost uzavřenosti dolů** (*downward closure property*): *každá podmnožina frekventované množiny je také frekventovaná*. Ekvivalentně řečeno, je-li nějaká množina nefrekventovaná, jsou nefrekventované i všechny její nadmnožiny.

Při hledání kombinací délky k se proto využijí už známé kombinace délky $k - 1$, což je *prohledávání do šířky*. Aby byla kombinace délky k vůbec vygenerována, požaduje se, aby *všechny* její podkombinace délky $k - 1$ splňovaly požadavek na frekvenci.

Algoritmus APRIORI (R. Agrawal)

```

1:  $L_1 \leftarrow$  všechny kategorie, které dosahují alespoň požadované frekvence
2:  $k \leftarrow 2$ 
3: while  $L_{k-1} \neq \emptyset$  do
4:    $C_k \leftarrow$  APRIORI-GEN( $L_{k-1}$ ) ▷ generování kandidátů
5:    $L_k \leftarrow$  ty kombinace z  $C_k$ , které dosáhly alespoň požadované frekvence
6:    $k \leftarrow k + 1$ 
7: end while

```

Vlastní generování kandidátů obstará **funkce APRIORI-GEN**(L_{k-1}), která nejprve kandidáty vytvoří spojováním a pak ty beznadějně zase vyhodí:

```

1: for každou dvojici kombinací  $Comb_p, Comb_q \in L_{k-1}$  do
2:   if  $Comb_p$  a  $Comb_q$  se shodují v  $k - 2$  kategoriích then
3:     přidej  $Comb_p \cup Comb_q$  do  $C_k$  ▷ krok spojení
4:   end if
5: end for
6: for každou kombinaci  $Comb \in C_k$  do ▷ krok prořezání
7:   if některá její podkombinace délky  $k - 1$  není v  $L_{k-1}$  then
8:     odstraň  $Comb$  z  $C_k$ 
9:   end if
10: end for

```

Generování pravidel. Nalezením frekventovaných kombinací práce nekončí, protože kombinace ještě není pravidlo. Z kombinací s vyhovující frekvencí se proto teprve vytvoří asociační pravidla, přičemž platí, že *frekvence nadkombinace* $\leq$ *frekvence kombinace*. Každá kombinace $Comb$ se rozdělí na všechny možné dvojice podkombinací Ant a Suc tak, že

$$Suc = Comb \setminus Ant, \quad Ant \cap Suc = \emptyset, \quad Ant \cup Suc = Comb.$$

Pro pravidlo $Ant \Rightarrow Suc$ pak *support* odpovídá frekvenci kombinace $Comb$ a *confidence* poměru frekvencí kombinací $Comb$ a Ant . Obojí se tedy dá odečíst z toho, co už algoritmus spočítal.

Přednosti a slabiny celé metody se dají shrnout do dvou sloupců:

Výhody MBA	Nevýhody MBA
jasné a pochopitelné výsledky — IF-THEN pravidla okamžitě použitelná	výpočetní náklady: prostor asociačních pravidel je exponenciální, $O(2^m)$ pro m položek
dobývání bez cílových výstupních hodnot (bez apriorní znalosti)	nutnost taxonomií a virtuálních položek
umí zpracovat data proměnné délky	obtížné určení vhodného počtu položek; položky by měly mít přibližně stejnou frekvenci
jednoduchý a snadno pochopitelný výpočet	<i>produkuje nadměrné množství pravidel</i> — tisíce, desetitisíce, miliony

3.1.5 Vícenásobný minimální support: MS-Apriori

Problém vzácných položek. Jediný *minsup* mlčky předpokládá, že všechny položky mají podobnou povahu a frekvenci. V praxi tomu tak není: některé položky se vyskytují velmi často a jiné jen vzácně, jak je vidět na dvojici kuchyňský robot proti chlebu a mléku. Ať se tedy jediná hodnota nastaví jakkoli, něco se pokazí:

- je-li *minsup* příliš **vysoký**, pravidla se vzácnými položkami se vůbec nenajdou;
- je-li příliš **nízký**, aby se našla pravidla se vzácnými i s častými položkami, hrozí *kombinatorická explozivnost*, protože se časté položky pospojují všemi možnými způsoby.

Model vícenásobných minimálních supportů (MIS)

Minimální support pravidla se vyjádří pomocí *minimálních supportů položek* (*minimum item support*, MIS) těch položek, které v pravidle vystupují. Každá položka smí mít vlastní $MIS(i)$, a zadáním různých hodnot tak uživatel vyjádří různé požadavky na support pro různá pravidla.

Minsup pravidla $R : a_1, \dots, a_k \Rightarrow a_{k+1}, \dots, a_r$ je *nejnižší* hodnota MIS mezi jeho položkami. Pravidlo tedy splňuje minimální support tehdy, je-li jeho skutečný support $\geq \min(MIS(a_1), \dots, MIS(a_r))$.

Omezení rozdílu supportů má zabránit tomu, aby se v téže množině sešly velmi časté a velmi vzácné položky. Požaduje se proto

$$\max_{i \in s} \{\text{sup}(i)\} - \min_{i \in s} \{\text{sup}(i)\} \leq \varphi,$$

kde $0 \leq \varphi \leq 1$ je uživatelem zadaný maximální rozdíl supportů, stejný pro všechny množiny.

Příklad — minsup pravidla podle MIS

Mějme položky *chléb*, *obuv*, *oděvy* s $MIS(\text{chléb}) = 2\%$, $MIS(\text{obuv}) = 0,1\%$, $MIS(\text{oděvy}) = 0,2\%$.

R_1 : *oděvy* $\Rightarrow$ *chléb* [sup = 0,15 %] minsup **nesplňuje**, protože $0,15\% < MIS(\text{oděvy}) = 0,2\%$.

R_2 : *oděvy* $\Rightarrow$ *obuv* [sup = 0,15 %] minsup naopak **splňuje**, protože $0,15\% \geq MIS(\text{obuv}) = 0,1\%$.

Za pružnost modelu se ale platí draze: **uzavřenost dolů v novém modelu neplatí!** MIS totiž dovoluje vyšší minimální supporty pro pravidla složená jen z frekventovaných položek a nižší pro pravidla s položkami méně frekventovanými. Ukažme si to na hodnotách $MIS(1) = 10\%$, $MIS(2) = 20\%$, $MIS(3) = 5\%$, $MIS(4) = 6\%$. Množina $\{1, 2\}$ se supportem 9 % je nefrekventovaná, přesto $\{1, 2, 3\}$ a $\{1, 2, 4\}$ frekventované být mohou. Množinu $\{1, 2\}$ tedy *nesmíme* zahodit, i když by ji klasické Apriori vyřadilo bez rozmýšlení.

Řešení je překvapivě jednoduché: položky se seřadí *vzestupně podle hodnot MIS* a toto pořadí se pak používá ve všech dalších operacích algoritmu. Každá množina w má díky tomu formu $\{w[1], \dots, w[k]\}$, kde $MIS(w[1]) \leq \dots \leq MIS(w[k])$.

Algoritmus MS-Apriori

```

1:  $M \leftarrow \text{SORT}(I, MS)$  ▷ setřídění položek podle MIS( $i$ )
2:  $L \leftarrow \text{INIT-PASS}(M, T)$  ▷ první průchod daty;  $L$  je množina semen
3:  $F_1 \leftarrow \{\{i\} \mid i \in L, i.\text{count}/n \geq \text{MIS}(i)\}$ 
4: for  $k = 2; F_{k-1} \neq \emptyset; k++$  do
5:   if  $k = 2$  then
6:      $C_k \leftarrow \text{LEVEL2-CANDIDATE-GEN}(L, \varphi)$ 
7:   else
8:      $C_k \leftarrow \text{MSCANDIDATE-GEN}(F_{k-1}, \varphi)$ 
9:   end if
10:  for každou transakci  $t \in T$  do
11:    for každého kandidáta  $c \in C_k$  do
12:      if  $c \subseteq t$  then
13:         $c.\text{count}++$ 
14:      end if
15:      if  $c \setminus \{c[1]\} \subseteq t$  then
16:         $c.\text{tailCount}++$  ▷ pro generování pravidel
17:      end if
18:    end for
19:  end for
20:   $F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq \text{MIS}(c[1])\}$ 
21: end for
22: return  $F = \bigcup_k F_k$ 

```

Jednotlivé kroky si zaslouží komentář. Během **prvního průchodu daty** (INIT-PASS) algoritmus zjistí support každé položky. Pak v setříděném pořadí najde *první* položku i , která splní MIS(i), a vloží ji do množiny semen L . Pro každou další položku j za i pak platí jednoduché kritérium: je-li $j.\text{count}/n \geq \text{MIS}(i)$, patří j do L také.

Příklad. Necht $\text{MIS}(1) = 10\%$, $\text{MIS}(2) = 20\%$, $\text{MIS}(3) = 5\%$, $\text{MIS}(4) = 6\%$ a mějme 100 transakcí s četnostmi $\{3\} = 6$, $\{4\} = 3$, $\{1\} = 9$, $\{2\} = 25$. Pak $L = \{3, 1, 2\}$ a $F_1 = \{\{3\}, \{2\}\}$. Položka 4 není v L , protože $4.\text{count}/n < \text{MIS}(3) = 5\%$; množina $\{1\}$ naopak není v F_1 , protože $1.\text{count}/n < \text{MIS}(1) = 10\%$.

Generování kandidátů úrovně 2 (LEVEL2-CANDIDATE-GEN) prochází položky $l \in L$ v daném pořadí. Splňuje-li l podmínku $l.\text{count}/n \geq \text{MIS}(l)$, přidá se pro každou položku h za l kandidát $\{l, h\}$, a to tehdy, když $h.\text{count}/n \geq \text{MIS}(l)$ a zároveň $|\text{sup}(h) - \text{sup}(l)| \leq \varphi$. Podstatné je, že se pracuje s L , a nikoli s F_1 : F_1 totiž neobsahuje položky, které splňují MIS nějaké dřívější položky, ale ne své vlastní, což je v uvedeném příkladu položka 1.

Generování kandidátů vyšších úrovní (MSCANDIDATE-GEN) má dvě části. *Krok spojení* je stejný jako v Apriori: spojí se $f_1 = \{i_1, \dots, i_{k-2}, i_{k-1}\}$ a $f_2 = \{i_1, \dots, i_{k-2}, i'_{k-1}\}$, které se liší jen v poslední položce, pokud $i_{k-1} < i'_{k-1}$ a $|\text{sup}(i_{k-1}) - \text{sup}(i'_{k-1})| \leq \varphi$. V *kroku prořezání* se prochází každá $(k-1)$ -podmnožina s kandidáta c : není-li $s \in F_{k-1}$, lze c smazat. Existuje ovšem **jediná výjimka**. Neobsahuje-li s položku $c[1]$, tedy položku s nejnižší hodnotou MIS (a taková s je právě jedna), nesmíme c smazat. Nevíme totiž, zda s nesplňuje MIS($c[1]$); víme jen, že nesplňuje MIS($c[2]$), a to by k smazání stačilo, jen pokud MIS($c[2]$) = MIS($c[1]$).

Proč tailCount. U Apriori platí, že je-li f frekventovaná a $f_{\text{sub}} \subseteq f$, je frekventovaná i f_{sub} , takže k určení confidence pravidel stačí supporty frekventovaných množin. U MS-Apriori to nestačí. Problém vzniká tehdy, když by v závěru pravidla stála položka s *nejnižší* hodnotou MIS. Proto se zvlášť počítá $c.\text{tailCount}$, tedy support množiny c bez její první položky.

Jak zadat hodnoty MIS. Nabízejí se dva postupy. Buď (a) se hodnota odvodí ze skutečného supportu položky v datech, například jako $\lambda \cdot \text{sup}(i)$ pro parametr $0 \leq \lambda \leq 1$ stejný pro všechny po-

ložky, nebo (b) se položky seskupí do bloků s podobnými frekvencemi a všem položkám z bloku se dá stejná hodnota MIS. Nastavením MIS = 100 % (nebo více) se generování pravidel obsahujících pouze tyto položky fakticky zakáže, což je praktický trik. Stojí za zapamatování, že model vícenásobných minsupů *zahrnuje* model jediného supportu jako speciální případ.

3.1.6 Třídní asociační pravidla (CAR)

Běžné dolování asociačních pravidel na žádný atribut necílí. Najde všechna pravidla, která v datech existují, a libovolná položka může vystupovat jako podmínka i jako závěr. V některých aplikacích ale uživatele zajímají právě *cílové* atributy či třídy, například které slovo je asociováno s kterým tématem dokumentu. Pak se vyplatí hledání rovnou zaměřit.

Třídní asociační pravidlo

Nechť T je množina n transakcí, každá navíc označovaná třídou y ; dále nechť I je množina všech položek, Y množina všech tříd a $I \cap Y = \emptyset$.

Třídní asociační pravidlo (*class association rule*, CAR) je implikace $X \rightarrow y$, kde $X \subseteq I$ a $y \in Y$. Support a confidence se definují stejně jako u běžných asociačních pravidel.

Příklad — CAR na textové kolekci

Dokument	Třída
1: Student, Učitel, Škola	Vzdělávání
2: Student, Škola	Vzdělávání
3: Učitel, Škola, Město, Hra	Vzdělávání
4: Baseball, Basketbal	Sport
5: Basketbal, Hráč, Divák	Sport
6: Baseball, Trenér, Hra, Tým	Sport
7: Basketbal, Tým, Město, Hra	Sport

Zvolíme-li $\text{minsup} = 20\%$ a $\text{minconf} = 60\%$, patří mezi nalezená CAR například tato dvě:

Student, Škola $\rightarrow$ Vzdelávání [$\text{sup} = 2/7$, $\text{conf} = 2/2$], Hra $\rightarrow$ Sport [$\text{sup} = 2/7$, $\text{conf} = 2/3$].

Na rozdíl od běžných asociačních pravidel se CAR dají dolovat **přímo v jednom kroku**. Klíčovou operací je nalezení všech *pravidlových položek* (*rule-items*) tvaru $(\text{condset}, y)$, kde $\text{condset} \subseteq I$ a $y \in Y$, které mají support nad minsup . Každá taková pravidlová položka totiž reprezentuje pravidlo $\text{condset} \rightarrow y$. Apriori se na generování CAR dá snadno upravit.

Také zde se dá použít idea vícenásobných minimálních supportů, jen se uplatní na úrovni tříd. Uživatel zadá *různé minimální supporty různým třídám*, a tím fakticky i různý minsup pravidlům každé třídy: pravidla třídy *Ano* mohou mít minsup 5 % a pravidla třídy *Ne* 10 %. Nastavením minimálního třídního supportu na 101 % se generování pravidel pro danou třídu zakáže.

3.1.7 Dolování sekvenčních vzorů

Asociační pravidla o *pořadí* transakcí nevědí nic, protože je vůbec neuvažují. V mnoha aplikacích je ale pořadí to podstatné. Už v MBA je zajímavé vědět, zda lidé kupují položky v posloupnosti, tedy nejprve postel a o nějakou dobu později povlečení. Při dolování z webových logů (*web usage mining*) se pak hledají navigační vzory podle posloupností návštěv stránek.

Sekvence, velikost, délka, podsekvence

Nechť $I = \{i_1, \dots, i_m\}$ je množina položek. *Sekvence* je uspořádaný seznam množin položek. *Množina položek* (*prvek* sekvence) je neprázdná $X \subseteq I$; sekvenci s značíme $\langle a_1 a_2 \dots a_r \rangle$, kde a_i je prvek. Bez újmy na obecnosti předpokládáme položky v prvku v lexikografickém pořadí.

Velikost sekvence je počet jejích prvků, kdežto délka je počet položek; sekvence délky k se nazývá k -sekvence.

Sekvence $s_1 = \langle a_1 \dots a_r \rangle$ je *podsekvencí* $s_2 = \langle b_1 \dots b_v \rangle$ (a s_2 je *nadsekvencí* s_1 , též s_2 *obsahuje* s_1), existují-li celá čísla $1 \leq j_1 < j_2 < \dots < j_r \leq v$ taková, že $a_1 \subseteq b_{j_1}, \dots, a_r \subseteq b_{j_r}$.

Příklad. Pro $I = \{1, \dots, 9\}$ je $\langle \{3\}\{4, 5\}\{8\} \rangle$ podsekvencí $\langle \{6\}\{3, 7\}\{9\}\{4, 5, 8\}\{3, 8\} \rangle$, protože $\{3\} \subseteq \{3, 7\}$, $\{4, 5\} \subseteq \{4, 5, 8\}$ a $\{8\} \subseteq \{3, 8\}$. Naopak $\langle \{3\}\{8\} \rangle$ *není* obsažena v $\langle \{3, 8\} \rangle$ ani naopak; na tom je vidět, že na rozdělení položek do prvků záleží. Velikost sekvence $\langle \{3\}\{4, 5\}\{8\} \rangle$ je 3, její délka 4.

Úloha se pak formuluje takto: pro danou množinu S vstupních datových sekvencí, tedy pro sekvenční databázi, najít všechny sekvence, které mají uživatelem zadaný minimální support. Takové sekvenci se říká *frekvencovaná sekvence* nebo *sekvenční vzor* a jejím supportem je podíl datových sekvencí v S , které ji obsahují.

Vzory hledá algoritmus **GSP** (*Generalized Sequential Pattern*), a to způsobem obdobným Apriori: prohledává do šířky s generováním a testováním kandidátů, kde kandidáti délky k vznikají spojováním frekvencovaných sekvencí délky $k - 1$.

Časový rozměr se dá do MBA vpašovat i jinak. **Analýza časových řad pomocí MBA** je analýza „příčina–následek“: zavedou se nové položky pro transakce, které nastaly *před* uvažovanou událostí (příčiny) a *po* ní (následky), a duplicitní položky se z transakcí odstraní. *Okno* je snímek všech dat za určité období, například všechny transakce z posledního měsíce, a slouží k zachycení trendů u vzácných položek. Může být nutná identifikace zákazníka.

Posledním rozšířením jsou **disociační pravidla** tvaru „IF A AND NOT B THEN C “. Získají se tak, že se zavedou nové položky komplementární k původním: neobsahuje-li transakce původní položku, obsahuje její negaci. Nevýhody jsou trojí. Počet položek se zdvojnásobí, velikost transakcí naroste a negované položky se vyskytují častěji než původní, takže pravidla se všemi položkami negovanými se obtížně využívají.

3.1.8 FP-Growth a FP-strom

Apriori pracuje ve stylu *generuj a testuj* a za to platí hned dvakrát. Generování kandidátů je nákladné prostorově i časově a nákladné je i počítání supportu, protože kontrola podmnožin je výpočetně drahá a databáze se prochází *opakovaně*, tedy s vysokou zátěží na I/O. **FP-Growth** proto hledá frekvencované množiny *bez generování kandidátů*, a to ve dvou krocích: (1) postaví dvěma průchody daty kompaktní datovou strukturu *FP-strom* a (2) extrahuje frekvencované množiny přímo z *FP-stromu* tím, že jej projde.

FP-strom

Uzly *FP-stromu* odpovídají položkám a každý má počítadlo. *FP-Growth* čte transakce po jedné a mapuje každou na *cestu* ve stromu. Používá se přitom *fixní pořadí* položek, takže cesty se mohou překrývat, mají-li transakce stejný *prefix*; v takovém případě se počítadla inkrementují. Mezi uzly, které obsahují tutéž položku, se udržují ukazatele tvořící jednosměrné spojové seznamy. Čím více cest se překrývá, tím vyšší je komprese a tím spíše se *FP-strom* vejde do paměti.

Konstrukce (2 průchody). *Průchod 1* zjistí support každé položky, nefrekventované zahodí a frekvencované seřadí *klesajícím* supportem, řekněme a, b, c, d, e ; toto pořadí se pak použije při stavbě stromu, aby se sdílely společné prefixy. *Průchod 2* strom postaví. Transakce $\{a, b\}$ vytvoří cestu $\text{null} \rightarrow a \rightarrow b$ s počítadly 1. Transakce $\{b, c, d\}$ s ní nemá společný prefix, a vytvoří proto *disjunktní* cestu $\text{null} \rightarrow b \rightarrow c \rightarrow d$; navíc se přidá ukazatel mezi oba uzly b . Transakce $\{a, c, d, e\}$ už s první transakcí společný prefix a má, takže se cesty překryjí a počítadlo uzlu a se inkrementuje.

Velikost FP-stromu. Obvykle bývá menší než nekomprimovaná data, protože transakce typicky sdílejí položky, a tedy i prefixy. V *nejlepší* případě obsahují všechny transakce stejnou množinu

položek, což dá jedinou cestu. V *nejhorším případě* má každá transakce unikátní množinu položek, strom je pak alespoň tak velký jako původní data a nároky na uložení jsou dokonce *vyšší*, protože je nutné uložit ukazatele a počítadla. Velikost navíc závisí na pořadí položek a klesající support je jen *heuristika*, která k nejmenšímu stromu vést nemusí.

Extrakce frekventovaných množin z FP-stromu

Algoritmus postupuje *zdola nahoru*, od listů ke kořeni, metodou *rozděl a panuj*: nejprve hledá frekventované množiny končící na e , pak na de , $\dots$, pak na d , cd atd.

1. **Prefixový podstrom** (*prefix path sub-tree*) končící danou položkou se získá pomocí spojových seznamů.
2. Ověř, zda je položka frekventovaná, a to sečtením počítadel po spojovém seznamu. (V příkladu pro $\text{minSup} = 2$ vyjde $\text{count}(e) = 3$, takže $\{e\}$ je frekventovaná množina.)
3. Je-li položka frekventovaná, hledej rekurzivně frekventované množiny končící na de , ce , be , ae . K tomu je nutné získat *podmíněný FP-strom*.

Podmíněný FP-strom (*conditional FP-tree*) pro položku e je takový strom, který by vznikl, kdybychom uvažovali *jen transakce obsahující e* a pak e ze všech transakcí odstranili. Získá se z prefixového podstromu ve třech krocích:

1. aktualizuj počítadla po prefixových cestách tak, aby odrážela počet transakcí obsahujících e ;
2. odstraň uzly obsahující e , protože informace o nich už není potřeba;
3. odstraň z prefixových cest nefrekventované položky. Má-li například b support 1, znamená to, že be má support 1, takže be je nefrekventovaná a b lze odstranit.

Podmíněný strom se pak použije k nalezení množin končících na de , ce , ae ; množina be se neuvažuje, protože b v podmíněném stromu není. Pro každou z nich se opět získají prefixové cesty, extrahují se frekventované množiny a vytvoří se podmíněný strom, a tak rekurzivně dál, například $e \rightarrow de \rightarrow ade$.

Také u FP-Growth se přednosti a slabiny dají postavit vedle sebe:

Výhody FP-Growth	Nevýhody FP-Growth
jen 2 průchody daty „komprimuje“ data	FP-strom se nemusí vejít do paměti stavba FP-stromu je nákladná (ale poté se množiny čtou snadno)
žádné generování kandidátů, výrazně rychlejší než Apriori	prořezávat lze jen na úrovni jednotlivých položek; support lze spočítat teprve po vložení <i>všech</i> dat do stromu

♦ **nad rámeček slidů** Doplnkově se ještě rozlišují dva úsporné druhy frekventovaných množin. *Uzavřená* frekventovaná množina (*closed*) je taková, jejíž žádná vlastní nadmnožina nemá stejný support, kdežto *maximální* (*maximal*) je taková, jejíž žádná vlastní nadmnožina není frekventovaná. Maximální množiny tvoří nejkompaktnější reprezentaci, ale ztrácejí informaci o supportech podmnožin. Uzavřené množiny jsou kompromis, protože reprezentují všechny supporty bezztrátově.

3.2 Přístupy založené na principu učení s učitelem

3.2.1 Formulace úlohy

O učení s učitelem mluvíme tam, kde ke každému trénovacímu vzoru známe i správnou odpověď. Úloha je tím pádem dobře vymezená: máme se naučit vztah mezi popisem vzoru a jeho správnou odpovědí tak, aby fungoval i na vzorech, které jsme nikdy neviděli.

Klasifikace a predikce

Je dána trénovací množina $\{(x_1, y_1), \dots, (x_p, y_p)\}$, kde x_i je vektor hodnot vstupních atributů a y_i hodnota *cílového* atributu. Úkolem je najít model $\hat{y} = f(x)$, který dobře předpovídá cílový atribut i pro dosud neviděné vstupy.

Podle povahy cílového atributu se úloha rozpadá na dvě. Je-li cílový atribut *diskrétní*, jde o **klasifikaci**; je-li *spojitý*, jde o **regresi** (predikci). Řešení má vždy dvě fáze: nejprve (1) **indukce** (učení, trénování) modelu z trénovacích dat a teprve potom (2) **aplikace** modelu na nová data.

Zásadní je, že se model hodnotí na *nových* datech. Schopnost naučit se trénovací data nazpaměť je bezcenná, protože o chování na datech dosud neviděných nic nevypovídá; takovému selhání se říká *přeučení* (*overfitting*) a vrací se jako téma u každé metody v tomto oddílu. Jak se chyba modelu odhaduje a jakými mírami se kvalita vyjadřuje, popisuje kap. 5, odd. 5.2.

3.2.2 Rozhodovací stromy

Nejnázornějším klasifikátorem je rozhodovací strom: rozhodnutí o třídě se v něm rozpadne na poloupnost jednoduchých dotazů na hodnoty jednotlivých atributů.

Rozhodovací strom

Nechť D je databáze $\{d_1, \dots, d_p\}$, $\{A_1, \dots, A_n\}$ množina atributů a $C = \{C_1, \dots, C_m\}$ množina tříd. *Rozhodovací strom* pro D je strom, kde:

- každý vnitřní uzel je označen některým atributem A_a ;
- každá hrana je označena predikátem, který lze aplikovat na atribut rodičovského uzlu;
- každý list nese označení některé třídy C_j .

Geometricky strom rozděluje prostor příznaků na *obdélníkové oblasti* a vzor se klasifikuje podle toho, do které oblasti padne.

Výhody	Nevýhody
snadná implementace a interpretace	obtížné zpracování spojitých dat (nutná kategorizace → obdélníkové oblasti)
efektivnost	nelze vždy použít — hranice tříd může být diagonální a strom s osově zarovnanými řezy ji umí jen schodovitě aproximovat (v příkladu bankovního úvěru vede hranice napříč rovinou <i>zůstatek na kontě</i> × <i>příjem</i>)
extrakce jednoduchých pravidel	obtížné zpracování chybějících dat
použitelnost i na velké datové soubory	přeučení ⇒ nutné <i>prořezávání</i>
	neuvažují vzájemné korelace mezi atributy

Ustálená terminologie pojmenovává tři věci. *Rozhodovací atributy*, podle nichž se datová množina dělí, označují rozhodovací uzly stromu, kdežto *rozhodovací predikáty* označují hrany. *Zastavovací kritérium* pak říká, kdy se má větvení ukončit; typicky zní „všechny vzory z redukované množiny patří do téže třídy“.

Algoritmus TDIDT (*Top Down Induction of Decision Trees*)

Strom se indukuje shora dolů metodou *rozděl a panuj*:

1. Vyber jeden atribut a umísti jej jako *kořen* částečného (pod)stromu.
2. Rozděl data přicházející do tohoto uzlu na podmnožiny podle hodnot vybraného atributu a pro každou podmnožinu přidej nový uzel.
3. Existuje-li uzel takový, že vzory do něj přicházející nepatří všechny do téže třídy, zopakuj pro něj celý proces od kroku 1; jinak skonči.

Entropie a volba dělicího atributu

Cílem je vybrat takový atribut, který nejlépe oddělí vzory z různých tříd. Mírou neuspořádanosti, tedy „překvapení“ v systému, je *entropie*

$$H = - \sum_{j=1}^m p_j \log_2 p_j,$$

kde p_j je pravděpodobnost výskytu třídy j , odhadnutá relativní frekvencí na uvažované množině vzorů, a m je počet tříd. Platí konvence $0 \log_2 0 = 0$ a pro převod z dekadického logaritmu vztah $\log_2 x = \log_{10} x / 0,301$.

Atribut A se hodnotí ve dvou krocích. Nejprve se pro každou jeho možnou hodnotu v spočítá entropie $H(A(v))$ na množině příkladů pokrytých kategorií $A(v)$,

$$H(A(v)) = - \sum_{j=1}^m p_j^{A(v)} \log_2 p_j^{A(v)},$$

a z těchto dílčích hodnot se pak jako vážený součet sestaví *vážená průměrná entropie* atributu

$$H(A) = \sum_v \frac{|A(v)|}{|D|} H(A(v)).$$

Vybírá se atribut s *minimální* hodnotou $H(A)$.

Příklad — žádost o úvěr

Klient	Příjem	Konto	Pohlaví	Nezaměstnaný	Úvěr
C1	vysoký	vysoké	žena	ne	ano
C2	vysoký	vysoké	muž	ne	ano
C3	nízký	nízké	muž	ne	ne
C4	nízký	vysoké	žena	ano	ano
C5	nízký	vysoké	muž	ano	ano
C6	nízký	nízké	žena	ano	ne
C7	vysoký	nízké	muž	ne	ano
C8	vysoký	nízké	žena	ano	ano
C9	nízký	střední	muž	ano	ne
C10	vysoký	střední	žena	ne	ano
C11	nízký	střední	žena	ano	ne
C12	nízký	střední	muž	ne	ano

Krok 1 — výběr atributu podle minimální entropie. Začneme atributem *příjem* a sestavíme pro něj a pro *úvěr* čtyřpolní tabulku: při vysokém příjmu je 5 klientů s úvěrem a 0 bez něj, při nízkém příjmu 3 s úvěrem a 4 bez.

$$\begin{aligned} H(\text{příjem}(\text{vysoký})) &= -\frac{5}{5} \log_2 \frac{5}{5} - \frac{0}{5} \log_2 \frac{0}{5} = 0, \\ H(\text{příjem}(\text{nízký})) &= -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0,9852, \\ H(\text{příjem}) &= \frac{5}{12} \cdot 0 + \frac{7}{12} \cdot 0,9852 = 0,5747. \end{aligned}$$

Stejným postupem se spočítají zbylé tři atributy: $H(\text{konto}) = \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot 1 + \frac{1}{3} \cdot 1 = 0,6667$, $H(\text{pohlaví}) = \frac{1}{2} \cdot 0,9183 + \frac{1}{2} \cdot 0,9183 = 0,9183$ a $H(\text{nezaměstnaný}) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 0,65 = 0,825$. Nejmenší entropii má **příjem**, podle něj se tedy strom větví poprvé.

Krok 2. Ve větvi *příjem(vysoký)* mají všechny vzory *úvěr(ano)*, a vzniká proto rovnou list. Ve větvi *příjem(nízký)* zůstává 7 klientů, takže je nutné větvit dál. Vychází $H(\text{konto}) = \frac{2}{7} H(\dots) + \frac{3}{7} H(\dots) = 0,3935$, $H(\text{pohlaví}) = 0,965$ a $H(\text{nezaměstnaný}) = 0,9792$, takže se větví podle **konta**.

Krok 3. Větev *konto(vysoké)* obsahuje samá *ano* a větev *konto(nízké)* samá *ne*, obě se tedy uzavřou listem. Ve větvi *konto(střední)* zbývají 3 klienti a z hodnot $H(\text{pohlaví}) = \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot 0 = 0,6667$ a $H(\text{nezaměstnaný}) = 0$ plyne třetí větvení podle **nezaměstnanosti**.

Výsledný strom a převod na pravidla. Přečtením cest od kořene k listům dostaneme pět

pravidel:

```
IF příjem(vysoký) THEN úvěr(ano)
IF příjem(nízký) ∧ konto(vysoké) THEN úvěr(ano)
IF příjem(nízký) ∧ konto(nízké) THEN úvěr(ne)
IF příjem(nízký) ∧ konto(střední) ∧ nezam.(ano) THEN úvěr(ne)
IF příjem(nízký) ∧ konto(střední) ∧ nezam.(ne) THEN úvěr(ano)
```

Obecně platí, že každá cesta od kořene k listu odpovídá právě jednomu pravidlu. Vnitřní uzly na cestě spolu s hodnotou uvedenou na odpovídající hraně tvoří levou stranu pravidla, tedy jeho tělo, a list s cílovou třídou tvoří stranu pravou, tedy hlavu.

Kvalitu výsledného stromu ovlivňuje celá řada rozhodnutí, která se během indukce dělají:

- *Volba rozhodovacích atributů* rozhoduje o efektivitě stromu; užitečný přitom bývá „informovaný“ vstup experta na danou oblast.
- *Pořadí rozhodovacích atributů* umožňuje vyloučit redundantní porovnání.
- *Větvení* určuje, kolik dělení je vůbec nutné provést; náročnější je u spojitých hodnot nebo u atributů s mnoha hodnotami.
- *Forma stromu* rozhoduje o jeho hloubce. Vhodnější jsou vyvážené stromy s co nejmenším počtem úrovní, byť za cenu složitějších porovnání; některé algoritmy tvoří pouze binární stromy, které bývají hluboké.
- *Zastavovací kritéria* určují, kdy indukce skončí. Základním kritériem je správná klasifikace trénovacích dat, *předčasné zastavení* navíc zabrání budování velkých stromů a přeučení, takže jde o kompromis mezi přesností a efektivitou. Za volbou jednodušších stromů stojí *Occamova břitva* (Vilém Occam, 1320), tedy preference nejjednodušší hypotézy pro D .
- *Trénovací data a jejich množství* působí na obě strany: příliš málo dat znamená, že strom nemusí být dost spolehlivý pro obecnější data, příliš mnoho dat zase přináší nebezpečí přeučení.
- *Prořezávání (pruning)* zvyšuje přesnost i efektivitu na testovacích datech. Odstraňují se jím redundantní porovnání, celé podstromy i irrelevantní atributy. Prořezávat lze buď už během indukce, což zabrání tvorbě nadměrně velkých stromů, nebo až na hotovém stromu.
- *Časová a prostorová složitost* závisí na počtu trénovacích dat p , počtu atributů n a formě stromu. Indukce stromu stojí $O(np \log p)$, klasifikace q vzorů stromem hloubky $O(\log p)$ pak $O(q \log p)$.

Konkrétní podobu dává tomuto schématu **algoritmus ID3** (*Iterative Dichotomiser*, Ross Quinlan, 1986). Učí funkce s booleovskými hodnotami a staví strom shora dolů *hladově*: v každém uzlu zvolí ten atribut, který lokální trénovací data klasifikuje nejlépe, a za nejlepší se považuje atribut s *největším informačním ziskem*. Stavba pokračuje, dokud nejsou všechna trénovací data správně klasifikována nebo dokud nejsou využity všechny atributy.

Algoritmus ID3 a informační zisk

ID3(EXAMPLES, TARGET_ATTRIBUTE, ATTRIBUTES):

- 1: vytvoř kořen *ROOT*
- 2: **if** všechny *EXAMPLES* jsou pozitivní **then return** *ROOT* s jediným uzlem označeným „+“
- 3: **end if**
- 4: **if** všechny *EXAMPLES* jsou negativní **then return** *ROOT* s jediným uzlem označeným „-“
- 5: **end if**
- 6: **if** *ATTRIBUTES* = $\emptyset$ **then return** *ROOT* s uzlem označeným nejčastější hodnotou *TARGET_ATTRIBUTE* v *EXAMPLES*
- 7: **end if**


```

8:  $A \leftarrow$  atribut z  $ATTRIBUTES$ , který nejlépe klasifikuje  $EXAMPLES$ 
9: rozhodovací atribut pro  $ROOT \leftarrow A$ 
10: for každou možnou hodnotu  $v_i$  atributu  $A$  do
11:   připoj k  $ROOT$  novou hranu odpovídající predikátu  $A = v_i$ 
12:    $EXAMPLES_{v_i} \leftarrow$  podmnožina  $EXAMPLES$  s hodnotou  $v_i$  pro  $A$ 
13:   if  $EXAMPLES_{v_i} = \emptyset$  then
14:     připoj k hraně list označený nejčastější hodnotou  $TARGET\_ATTRIBUTE$ 
15:   else
16:     připoj podstrom  $ID3(EXAMPLES_{v_i}, TARGET\_ATTRIBUTE, ATTRIBUTES \setminus \{A\})$ 
17:   end if
18: end for
19: return  $ROOT$ 

```

Kritériem volby atributu je **informační zisk** (*information gain*), tedy očekávané snížení entropie poté, co se data rozdělí podle uvažovaného atributu:

$$GAIN(D, A) = H(D) - \sum_{v \in VALUES(A)} \frac{|D_v|}{|D|} H(D_v).$$

Hodnota $H(D)$ je přitom pro všechny atributy stejná, takže maximalizovat informační zisk znamená totéž jako minimalizovat váženou entropii $H(A)$.

Algoritmus C4.5 (Quinlan, 1993) je modifikací ID3 a doplňuje jej hned v několika směrech:

- **Chybějící data** se při konstrukci stromu ignorují, tedy při vyhodnocení kritéria dělení se uvažují jen známé hodnoty daného atributu. Při klasifikaci se chybějící hodnoty odhadují podle hodnot známých u jiných vzorů.
- **Spojitá data** se kategorizují podle hodnot, kterých atribut nabývá na trénovacích vzorech.
- **Prořezávání validací** vyžaduje rozdělit trénovací data na vlastní trénovací ($\frac{2}{3}$) a validační ($\frac{1}{3}$) množinu. Při metodě *reduced-error pruning* se podstromy nahrazují listy označenými nejčastější třídou příslušných trénovacích vzorů, přičemž nový strom *nesmí* být na validační množině horší než původní; jinak se podstrom nenahradí. Při metodě *subtree raising* se podstrom nahradí svým nejčastěji používaným podstromem, který se zvedne na vyšší úroveň, a testuje se klasifikační přesnost.
- **Prořezávání pravidel** (*rule post-pruning*) jde jinou cestou. Z trénovací množiny se nejprve odvodí strom, u něhož je přeučení dovoleno, a převede se na ekvivalentní množinu pravidel, jedno pravidlo na cestu od kořene k listu. Pravidla se pak prořezou, tedy zobecní, vypouštěním všech možných předpokladů, pokud to nezhorší odhadovanou klasifikační přesnost, a nakonec se seřadí podle očekávané přesnosti; v tomto pořadí se pak používají ke klasifikaci. Prořezávání pravidel je snazší a radikálnější než prořezávání celých stromů a výsledná pravidla jsou průhledná a snadno pochopitelná.
- **Odhad přesnosti pravidel** se standardně dělá validační množinou, C4.5 však hodnotí přesnost *na trénovacích datech* jako podíl vzorů správně klasifikovaných pravidlem ke všem vzorům, které pravidlo pokrývá, $acc = k/n$. Směrodatná odchylka přesnosti se odhaduje z binomického rozdělení a jako charakteristika pravidla se bere *dolní mez* přesnosti pro zvolený interval spolehlivosti, na 95% hladině

$$acc - 1,96 \sqrt{\frac{acc(1 - acc)}{n}}.$$

- **Poměrný informační zisk** (*information gain ratio*) nahrazuje dosavadní kritérium dělení:

$$GAIN_RATIO(D, A) = \frac{GAIN(D, A)}{-\sum_v \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}}.$$

Dělí se podle atributu s *největším* poměrným informačním ziskem, přičemž jmenovatel penalizuje *nadměrné větvení*, tedy atributy s mnoha hodnotami.

Algoritmus C5.0 je komerční verzi C4.5 určenou pro velké databáze. Indukce stromu probíhá podobně, generování pravidel je ale zlepšené a efektivnější a vyšší přesnosti se dosahuje *boostingem*. Z původních trénovacích dat se staví různé trénovací podmnožiny a každému trénovacímu vzoru se přiřadí váha odpovídající jeho vlivu při klasifikaci. Pro každou kombinaci vah pak vznikne samostatný klasifikátor a při klasifikaci má každý klasifikátor svůj hlas, přičemž vítězí většinová třída.

CART — Classification And Regression Trees

CART generuje výhradně *binární* stromy: při každém dělení dat se tvoří právě dvě hrany. Nejlepší rozhodovací atribut se volí podle entropie, nebo podle **Gini indexu**. Pro atribut o r hodnotách $v_1, \dots, v_r$ při n vzorech a m třídách je

$$G = \sum_{i=1}^r \frac{n_i}{n} G(v_i), \quad G(v_i) = 1 - \sum_{j=1}^m p_j^2, \quad \sum_{i=1}^r n_i = n,$$

kde n_i je počet vzorů s hodnotou v_i a p_j pravděpodobnost třídy j v těchto n_i vzorech. Čím *nižší* je hodnota G , tím lepší je diskriminace.

V uzlu t se dělí podle „nejlepšího“ nalezeného bodu, který se hledá průchodem všech možných hodnot v rozhodovacího atributu A ; při rovnosti se použije pravý podstrom. Označme P_L, P_R pravděpodobnosti zpracování levým, resp. pravým podstromem a $P(C_j | t_L), P(C_j | t_R)$ pravděpodobnosti, že vzor patří do třídy C_j a je v levém, resp. pravém podstromu uzlu t . Kritérium dělení pak zní

$$\Phi = 2P_L P_R \sum_{j=1}^m |P(C_j | t_L) - P(C_j | t_R)|,$$

a nabývá maxima tehdy, jsou-li všechny vzory dané třídy seskupeny v témže podstromu.

Pro CART je charakteristických několik vlastností. Pořadí atributů ve stromu odpovídá jejich vlivu při klasifikaci. Chybějící data se ignorují a nezapočítávají se do vyhodnocení rozhodovacích kritérií. Zastavovacím kritériem je zjištění, že žádné další dělení už klasifikační přesnost nezlepší. Vysoká přesnost na trénovací množině přitom nemusí odrážet přesnost na datech testovacích.

CHAID — Chi-square Automatic Interaction Detection

Kritériem větvení je zde χ^2 . Algoritmus navíc automaticky *seskupuje hodnoty* kategoriálních atributů: hodnoty se postupně slučují od původního počtu, dokud nezůstanou jen dvě grupy, a teprve pak se vybere atribut spolu s kategorizací, které jsou pro dělení dat v daném kroku nejlepší. Při větvení tak nevzniká tolik větví, kolik má atribut hodnot.

Seskupování hodnot atributu probíhá takto:

1. Opakuj, dokud nezůstanou jen dvě grupy hodnot: (a) vyber pár kategorií atributu, které jsou si z hlediska χ^2 nejvíce podobné a lze je sloučit; (b) novou kategorii uvažuj jako možné seskupení v daném kroku.
2. Pro každou možnou grupu hodnot testuj χ^2 testem její relevanci k predikované proměnné.
3. Vyber seskupení, které dává „nejlepší“ rozdělení s nejvýznamnějším rozštěpením.
4. Urči, zda toto seskupení statisticky významně přispívá k rozlišení vzorů z různých tříd.

3.2.3 Bayesovská klasifikace

Zatímco strom rozhoduje posloupností testů, bayesovský přístup počítá pravděpodobnost jednotlivých tříd a vybírá tu nejpravděpodobnější.

Bayesova formule a hypotéza MAP

Pravděpodobnost, že hypotéza H platí, pozorujeme-li E , je

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)},$$

kde $P(H)$ je *apriorní* pravděpodobnost hypotézy, $P(H | E)$ *aposteriorní* (podmíněná) pravděpodobnost a $P(E)$ pravděpodobnost jevu E .

Za *nejpravděpodobnější hypotézu* H_{MAP} (*maximum a posteriori*) se považuje ta s největší aposteriorní pravděpodobností. Jmenovatel $P(E)$ je pro všechny hypotézy stejný, takže jej lze zanedbat:

$$H_{MAP} = \arg \max_{H \in \mathcal{H}} P(H | E) = \arg \max_{H \in \mathcal{H}} P(E | H) P(H).$$

Naivní bayesovský klasifikátor

Předpoklad: jevy $E_1, \dots, E_k$ jsou *podmíněně nezávislé* při platnosti hypotézy H . V reálných úlohách bývá tento předpoklad splněn jen zřídka, a právě odtud pochází označení „naivní“. Za tohoto předpokladu je

$$P(H | E_1, \dots, E_k) = \frac{P(E_1, \dots, E_k | H) P(H)}{P(E_1, \dots, E_k)} = \frac{P(H)}{P(E_1, \dots, E_k)} \prod_{i=1}^k P(E_i | H),$$

a klasifikuje se hypotézou s největší aposteriorní pravděpodobností

$$H_{MAP} = \arg \max_{H_t} P(H_t) \prod_{i=1}^k P(E_i | H_t).$$

Potřebné pravděpodobnosti se odhadnou přímo z dat jako $P(H_t) = P(\text{třída } t) = \frac{\text{počet vzorů z } t}{\text{počet všech vzorů}}$ a $P(E_k | H_t) = \frac{\text{počet vzorů z } t \text{ splňujících } E_k}{\text{počet vzorů z } t}$.

Výhodou klasifikátoru je, že si poradí i s *nekompletně popsanými* vzory. **Nevýhodou** je nulová aposteriorní pravděpodobnost $P(E_k | H_t)$ v případě, že v trénovací množině není žádný odpovídající vzor; podobně jsou podhodnocené i ty členy, u nichž se E_k a H_t vyskytnou společně jen zřídka.

Příklad — bayesovská klasifikace úvěru

Jednoduchý případ vystačí s jediným pozorováním. Nechť $P(\text{půjčit}) = 0,667$, $P(\text{zamítnout}) = 0,333$, $P(\text{vysoký příjem} | \text{půjčit}) = 0,91$ a $P(\text{vysoký příjem} | \text{zamítnout}) = 0,12$. Porovnáním $0,91 \cdot 0,667 = 0,607$ proti $0,12 \cdot 0,333 = 0,040$ vychází $H_{MAP} = \text{půjčit}$.

Naivní klasifikátor nad daty z příkladu o úvěru (12 klientů) potřebuje odhadnout tyto pravděpodobnosti: $P(\text{úvěr(ano)}) = 8/12 = 0,667$, $P(\text{úvěr(ne)}) = 4/12 = 0,333$; $P(\text{konto(střední)} | \text{ano}) = 2/8 = 0,25$, $P(\text{konto(střední)} | \text{ne}) = 2/4 = 0,5$; $P(\text{nezam.(ne)} | \text{ano}) = 5/8 = 0,625$, $P(\text{nezam.(ne)} | \text{ne}) = 1/4 = 0,25$.

Pro klienta se středním kontem, který není nezaměstnaný, pak vyjde

$$0,667 \cdot 0,25 \cdot 0,625 = 0,1042 \quad \text{vs.} \quad 0,333 \cdot 0,5 \cdot 0,25 = 0,0416,$$

klient se tedy zařadí do třídy **úvěr(ano)**.

3.2.4 Support Vector Machines

Metodu SVM vynalezl V. Vapnik se spolupracovníky už v 70. letech v Rusku, na Západě se ale stala známou teprve v roce 1992. Základem jsou *lineární* klasifikátory, které hledají hyperrovinu oddělující dvě třídy, pozitivní a negativní. Tam, kde lineární separace nestačí, nastupují *kernelové* funkce.

SVM stojí na důkladném teoretickém základu a zároveň v aplikacích dosahuje vyšší přesnosti než většina jiných metod, a to zejména na vysokorozměrných datech. Pro klasifikaci textu patří k vůbec nejlepším klasifikátorům.

Lineární SVM, separabilní případ

Trénovací množina $D = \{(x_1, y_1), \dots, (x_r, y_r)\}$, $x_i \in X \subseteq \mathbb{R}^n$, $y_i \in \{1, -1\}$. SVM hledá lineární funkci $f(x) = \langle w \cdot x \rangle + b$ tak, že

$$y_i = \begin{cases} 1 & \text{pro } \langle w \cdot x_i \rangle + b \geq 0, \\ -1 & \text{pro } \langle w \cdot x_i \rangle + b < 0. \end{cases}$$

Hyperrovina $\langle w \cdot x \rangle + b = 0$ se nazývá *rozhodovací hranice* (*decision boundary*). Takových hyperrovin existuje mnoho a SVM z nich vybírá tu s *největším okrajem* (*margin*), protože podle teorie strojového učení právě ona minimalizuje mez chyby.

Vzdálenost bodu x_i od hyperroviny je $\frac{|\langle w \cdot x_i \rangle + b|}{\|w\|}$, kde $\|w\| = \sqrt{\langle w \cdot w \rangle}$. Pro okrajové hyperroviny H_+ a H_- platí

$$d_+ = d_- = \frac{1}{\|w\|}, \quad \text{margin} = d_+ + d_- = \frac{2}{\|w\|}.$$

Maximalizace okraje se tak převede na **optimalizační úlohu**

$$\text{minimalizuj } \frac{\langle w \cdot w \rangle}{2} \quad \text{za podmíněk } y_i(\langle w \cdot x_i \rangle + b) \geq 1, \quad i = 1, \dots, r,$$

kde je omezení zapsáno souhrnně za obě třídy najednou: $\langle w \cdot x_i \rangle + b \geq 1$ pro $y_i = 1$ a $\langle w \cdot x_i \rangle + b \leq -1$ pro $y_i = -1$.

Duální formulace a podpůrné vektory

Úloha se řeší standardní *Lagrangeovou* metodou:

$$L_P = \frac{\langle w \cdot w \rangle}{2} - \sum_{i=1}^r \alpha_i [y_i(\langle w \cdot x_i \rangle + b) - 1], \quad \alpha_i \geq 0.$$

Optimální řešení musí splňovat *Kuhn–Tuckerovy podmínky*, které jsou pro *konvexní* účelovou funkci s *lineárními* omezeními nejen nutné, ale i postačující. Z podmínky komplementarity plyne, že $\alpha_i > 0$ mohou mít jen body ležící na okrajových hyperrovinách, neboť právě pro ně je $y_i(\langle w \cdot x_i \rangle + b) - 1 = 0$. Těmto bodům se říká **podpůrné vektory** (*support vectors*); pro všechny ostatní body je $\alpha_i = 0$.

Vynulováním parciálních derivací L_P podle w a b a zpětnou substitucí dostaneme *Wolfeho duální* úlohu

$$L_D = \sum_{i=1}^r \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle x_i \cdot x_j \rangle \rightarrow \max,$$

jejíž maximum se nabývá pro tytéž hodnoty w, b, α_i jako minimum L_P . Numericky se duální úloha řeší snáz než úloha primární.

Tvar **rozhodovací hranice** a způsob **testování** vzoru z jsou pak

$$\langle w \cdot x \rangle + b = \sum_{i \in sv} y_i \alpha_i \langle x_i \cdot x \rangle + b = 0, \quad \text{sign} \left(\sum_{i \in sv} \alpha_i y_i \langle x_i \cdot z \rangle + b \right).$$

Vrátí-li výraz 1, je z klasifikován jako pozitivní, jinak jako negativní.

Kernelový trik

Reálná data mohou vyžadovat nelineární separaci. Řešením je *transformovat* je do jiného, obvykle mnohem vícerozměrného prostoru, v němž už třídy oddělí *lineární* hranice: $\phi : X \rightarrow F, x \mapsto \phi(x)$. Transformovaný prostor F se nazývá *prostor příznaků* (*feature space*), původní prostor *vstupní*. **Problém explicitní transformace** spočívá v tom, že počet dimenzí prostoru příznaků může být pro užitečné transformace enormní i při rozumném počtu vstupních atributů; explicitní výpočet se tak stává kvůli *prokletí dimenzionality* nezvládnutelným.

Klíčové pozorování je, že v duální formulaci vyžadují jak konstrukce optimální hyperroviny, tak vyhodnocení rozhodovací funkce *pouze skalární součiny* $\langle \phi(x) \cdot \phi(z) \rangle$, nikdy transformovaný vektor $\phi(x)$ samotný. Umíme-li tedy počítat skalární součin přímo ze vstupních vektorů, nemusíme ϕ znát vůbec. Takové funkci se říká *kernelová funkce*

$$K(x, z) = \langle \phi(x) \cdot \phi(z) \rangle.$$

Nahradit ve všech výrazech skalární součin kernelem znamená použít **kernelový trik**.

Příklad (polynomiální kernel $d = 2$, 2-D vstup):

$$\begin{aligned} \langle x \cdot z \rangle^2 &= (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2x_1 z_1 x_2 z_2 + x_2^2 z_2^2 \\ &= \langle (x_1^2, x_2^2, \sqrt{2}x_1 x_2) \cdot (z_1^2, z_2^2, \sqrt{2}z_1 z_2) \rangle = \langle \phi(x) \cdot \phi(z) \rangle. \end{aligned}$$

Kernel $\langle x \cdot z \rangle^d$ je tedy skalární součin v transformovaném prostoru, aniž bychom ϕ kdy vyčíslili. Na otázku, zda je daná funkce kernelem, tj. zda je skalárním součinem v nějakém prostoru příznaků, odpovídá *Mercerova věta*. Mezi běžně používané kernely patří lineární (identita), polynomiální, gaussovský (RBF) a sigmoidální.

Metoda má i několik omezení, která je dobré znát. SVM pracuje jen v reálném prostoru, takže kategoriální atributy je nutné převést na numerické, a provádí jen *dvoutřídní* klasifikaci. Pro více tříd se proto sahá po strategii *jeden proti ostatním* (*one-against-rest*) nebo po kódech opravujících chyby (*error-correcting output coding*). Hyperrovina, kterou SVM vyprodukuje, je navíc pro člověka obtížně pochopitelná a kernely to ještě zhoršují; SVM se proto používá tam, kde se srozumitelnost pro člověka nevyžaduje. [♦ nad rámec slidů] Pro nelineárně separabilní data se zavádějí *proměnné vůle* (*slack variables*) $\xi_i \geq 0$ a minimalizuje se $\frac{\langle w \cdot w \rangle}{2} + C \sum_i \xi_i$, kde parametr C řídí kompromis mezi šířkou okraje a počtem chyb.

3.2.5 Logistická regrese

Logistická regrese má dvojí tvář. Je to nejdůležitější případ nelineární regrese (kap. 2) a zároveň plnohodnotný klasifikátor: závislá proměnná y je kategoriální, typicky dvouhodnotová, a model nepředpovídá přímo její hodnotu, nýbrž *pravděpodobnost*, že y nabude určité hodnoty, a to v závislosti na kombinaci hodnot nezávislých veličin.

Logistická regrese

Pro binární $y \in \{0, 1\}$ se modeluje logaritmus „podmíněné šance“ (*conditional odds*):

$$\ln \frac{P(y = 1 \mid x_1, \dots, x_m)}{1 - P(y = 1 \mid x_1, \dots, x_m)} = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m = z,$$

odkud

$$P(y = 1 \mid x) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z) \quad (\text{sigmoida}).$$

Učení probíhá maximalizací věrohodnosti (*maximum likelihood*):

$$L = \prod_{i=1}^p P(y_i \mid x_{i,1}, \dots, x_{i,m}) = \prod_{i=1}^p \left(\frac{1}{1 + e^{-z_i}} \right)^{y_i} \left(\frac{1}{1 + e^{z_i}} \right)^{1-y_i}.$$

Místo maximalizace L se v praxi minimalizuje *křížová entropie* $E = -\log L$:

$$E = - \sum_i \left[y_i \log \sigma(z_i) + (1 - y_i) \log (1 - \sigma(z_i)) \right].$$

Derivace sigmoidy má tvar $\sigma' = \sigma(1 - \sigma)$, takže po zjednodušení vyjdou velmi jednoduchá adaptační pravidla

$$\Delta\beta_j = -\eta \frac{\partial E}{\partial \beta_j} = \eta \sum_i (y_i - \sigma(z_i)) x_{i,j}, \quad \Delta\beta_0 = \eta \sum_i (y_i - \sigma(z_i)),$$

kde η je rychlost učení. Chyba tedy vstupuje do úpravy vah přímo jako rozdíl *pozorované* a *predikované* hodnoty.

Logistická regrese bývá běžnou volbou základního klasifikátoru všude tam, kde se požaduje pravděpodobnostní výstup a interpretovatelné koeficienty. Používá se například při predikci vazeb v sociálních sítích (kap. 6).

3.2.6 Diskriminační analýza

Diskriminační analýza

Úlohou je *klasifikace vzorů do známých tříd*, a hledá se proto závislost nominální veličiny, která určuje příslušnost ke třídě, na ostatních numerických veličinách. Předpokládáme, že ke každé třídě C_j , $j = 1, \dots, m$ existuje *diskriminační funkce* g_j ; vzor $x = (x_1, \dots, x_m)$ patří do třídy C_j , právě když

$$g_j(x) = \max_k g_k(x).$$

Lineární diskriminační analýza volí $g_j(x) = \beta_{j0} + \beta_{j1}x_1 + \dots + \beta_{jm}x_m$. Jsou-li třídy jen dvě, stačí místo g_1, g_2 hledat jedinou funkci $g = g_1 - g_2$ a klasifikovat podle jejího *znaménka*: „+“ znamená třídu 1, „-“ třídu 2.

Optimální klasifikace ve smyslu minimální chyby nastane tehdy, jsou-li diskriminační funkce $\propto$ aposteriorní (podmíněné) pravděpodobnosti zařazení pozorování x do třídy C_j , tedy $g_j(x) = P(C_j | x) = \frac{p(x | C_j)P(C_j)}{p(x)}$; pro dvě třídy $g(x) = p(x | C_1)P(C_1) - p(x | C_2)P(C_2)$.

Konkrétní tvar diskriminační funkce závisí na rozdělení dat. Pro *normální rozdělení* s různými středními hodnotami μ_1, μ_2 a obecnými kovariančními maticemi S_1, S_2 vychází funkce *kvadratická*. Jsou-li kovarianční matice *shodné* ($S_1 = S_2 = S$), redukuje se na *lineární*

$$g(x) = (\mu_1 - \mu_2)^\top S^{-1}x - \frac{1}{2}(\mu_1 - \mu_2)^\top S^{-1}(\mu_1 + \mu_2) - \ln \frac{P(C_2)}{P(C_1)},$$

a jsou-li navíc obě třídy stejně pravděpodobné a $S = I$, zbude jednoduché srovnávání vzdáleností od středních hodnot. Pro normální rozdělení se tedy celé hledání diskriminačních funkcí redukuje na **odhad středních hodnot** výběrovými průměry a **kovariančních matic** korigovanými výběrovými rozptyly.

3.2.7 Neuronové sítě a extrémní učící stroje (ELM)

Síť ELM (*Extreme Learning Machine*, G.-B. Huang)

Jde o dopřednou síť s *jednou* skrytou vrstvou: n vstupních neuronů, L skrytých a 1 výstupní. Výstup skrytých neuronů je $h(x) = [G(a_1, b_1, x), \dots, G(a_L, b_L, x)]$, kde a_i je váhový vektor a b_i bias i -tého neuronu; například $G(a_i, b_i, x) = g(\langle a_i \cdot x \rangle + b_i)$ (sigmoida) nebo $G(a_i, b_i, x) = g(b_i \| x - a_i \|)$ (RBF). Výstup celé sítě je $f_L(x) = \sum_{i=1}^L \beta_i G(a_i, b_i, x)$.

Klíčová myšlenka: lze-li spojitou funkci f aproximovat dopřednou sítí se skrytou vrstvou neuronů počítajících G , pak *není nutné parametry skrytých neuronů adaptovat*. Mohou být vygenerovány *náhodně*, dokonce bez znalosti trénovacích dat. Pro každou spojitou f a náhodně generovanou posloupnost $\{(a_i, b_i)\}$ totiž platí s pravděpodobností 1

$$\lim_{L \rightarrow \infty} \left\| f - \sum_{i=1}^L \beta_i G(a_i, b_i, \cdot) \right\| = 0$$

při nastavení β_i tak, aby minimalizovaly $\|f - f_L\|$.

Maticový model: soustava $\sum_{i=1}^L \beta_i G(a_i, b_i, x_j) = t_j, j = 1, \dots, N$ je ekvivalentní zápisu $H\beta = T$, kde H je *výstupní matice skryté vrstvy*; její j -tý řádek tvoří výstupy skrytých neuronů pro vzor x_j a i -tý sloupec výstupy i -tého neuronu pro všechny vzory.

Algoritmus učení se tím smyslně na tři neiterativní kroky:

- 1: zvol *náhodně* parametry skrytých neuronů $(a_i, b_i), i = 1, \dots, L$
- 2: spočítej výstupní matici skryté vrstvy H
- 3: spočítej výstupní váhy $\beta = H^+ T$

kde H^+ je Mooreova–Penroseova pseudoinverze H . Stabilnější řešení s lepší generalizací dá regularizace, tedy přičtení I/C na diagonálu $H^T H$: $f(x) = h(x)(I/C + H^T H)^{-1} H^T T$.

Výhody ELM plynou přímo z té tříkrokové procedury. Učicí procedura je jednoduchá a neiterativní, zatímco tradiční algoritmy jsou podstatně složitější. Trénování je díky tomu velmi rychlé a přímočaré: odpadá řešení lokálních minim, hledání vhodné inicializace parametrů i starost o přeučení. Parametry skrytých neuronů a_i, b_i jsou nezávislé na trénovacích datech i na sobě navzájem, takže je lze vygenerovat *už před* předložením trénovacích dat; konvenční techniky naproti tomu data „vidět“ musí. Poslední výhodou je volnost ve výběru přenosové funkce, protože ELM je použitelný s *libovolnou* omezenou po částech spojitou nekonstantní přenosovou funkcí, kdežto gradientní algoritmy vyžadují funkce hladké.

3.2.8 Ensemble metody

Ensemble metody nespolehají na jediný dokonalý model, ale kombinují predikce mnoha klasifikátorů. Liší se přitom v tom, kterou složku chyby snižují.

Bagging (*bootstrapped aggregating*)

Bagging se snaží snížit *rozptyl* klasifikátoru. Je-li rozptyl jednoho klasifikátoru σ^2 , pak rozptyl průměru k nezávislých a stejně rozdělených (i.i.d.) klasifikátorů klesne na σ^2/k .

Takové i.i.d. klasifikátory se aproximují **bootstrappingem**: body se vybírají *s opakováním* uniformně z původních dat. Vzorek má přibližně stejnou velikost jako původní data, může obsahovat duplikáty a obsahuje podíl

$$1 - \left(1 - \frac{1}{n}\right)^n \approx 1 - \frac{1}{e} = 63,2\%$$

různých bodů z původních dat, neboť pravděpodobnost, že bod ve vzorku *není*, je $(1 - 1/n)^n$. Vylosuje se k nezávislých bootstrapových vzorků velikosti n , na každém se natrénuje klasifikátor a predikce pro testovací vzor se určí *většinovým hlasem*.

Vhodnost: ideálním základem pro bagging jsou rozhodovací stromy, protože mají *nízké zkreslení* a *vysoký rozptyl*, jsou-li dostatečně hluboké. **Omezení:** bagging *nesnižuje zkreslení* modelu. Je-li zkreslení hlavním zdrojem chyby, může přesnost dokonce mírně zhoršit, a to kvůli korelacím mezi komponentami, které porušují předpoklad i.i.d.

Náhodné lesy (*random forests*)

Mají-li k klasifikátorů s rozptylem σ^2 vzájemnou *pozitivní korelaci* ρ , je rozptyl zprůměrované predikce

$$\rho\sigma^2 + \frac{(1-\rho)\sigma^2}{k}.$$

Člen $\rho\sigma^2$ je *nezávislý* na počtu komponent k , a právě on omezuje přínos baggingu. Bootstrapované stromy přitom pozitivně korelované bývají.

Náhodný les je ensemble rozhodovacích stromů, do jejichž *konstrukce* je explicitně vložena náhodnost, aby byly komponenty méně korelované. Vedou k tomu dvě cesty. (1) **Bagging** znamená, že se stromy trénují na mírně odlišných datech, totiž na bootstrapových vzorcích. (2) **Omezený počet příznaků** znamená, že se stromu v každém uzlu předloží jen *náhodná podmnožina* m příznaků a informační zisk se počítá jen na ní. Druhá cesta navíc zrychluje trénování, protože je k prohledání méně příznaků, a stromy není nutné prořezávat; snížený rozptyl přitom nezhoršuje zkreslení. Výstupem lesa je většinový hlas stromů.

Algoritmus je pro každý z N stromů stejný: vytvoř nový bootstrapový vzorek trénovací množiny a nad ním trénuj rozhodovací strom tak, že v každém uzlu vybereš náhodně m příznaků, spočítáš informační zisk jen na nich a vybereš optimální; opakuj, dokud není strom hotový.

Parametry jsou dva. Počet příznaků m se běžně volí jako $\sqrt{\text{počet příznaků}}$, na tento parametr ostatně nejsou náhodné lesy příliš citlivé. S počtem stromů lze pokračovat tak dlouho, dokud chyba klesá.

Boosting a AdaBoost (Freund & Schapire, 1996)

Princip: každému trénovacímu vzoru je přiřazena *váha*, různé klasifikátory se trénují s těmito vahami a váhy se iterativně upravují podle výkonu klasifikátoru. Budoucí modely tedy *závisí na předchozích*: každý klasifikátor se staví tímtež algoritmem, ale na jinak váženém trénovacím souboru. **Idea** je soustředit se v dalších iteracích na *nesprávně klasifikované* vzory tím, že se zvýší jejich relativní váha. Iterativním opakováním a vytvořením vážené kombinace klasifikátorů tak lze získat klasifikátor s nižším celkovým *zkreslením*, na rozdíl od baggingu, který snižuje rozptyl.

ADABOOST(data D , bazový klasifikátor A , max. počet kol T):

- 1: $t \leftarrow 0$; pro každý vzor i inicializuj $W_1(i) = 1/n$
- 2: **repeat**
- 3: $t \leftarrow t + 1$
- 4: urči váženou chybu ϵ_t na D , aplikuje-li se A na data s vahami $W_t(\cdot)$
- 5: $\alpha_t \leftarrow \frac{1}{2} \ln((1 - \epsilon_t)/\epsilon_t)$
- 6: **for** každý *nesprávně* klasifikovaný $X_i \in D$ **do**
- 7: $W_{t+1}(i) \leftarrow W_t(i)e^{\alpha_t}$
- 8: **end for**
- 9: **for** každý *správně* klasifikovaný $X_i \in D$ **do**
- 10: $W_{t+1}(i) \leftarrow W_t(i)e^{-\alpha_t}$
- 11: **end for**
- 12: normalizuj $W_{t+1}(i) \leftarrow W_{t+1}(i) / \sum_j W_{t+1}(j)$
- 13: **until** $t \geq T$ **nebo** $\epsilon_t = 0$ **nebo** $\epsilon_t \geq 0,5$
- 14: klasifikuj testovací vzory ensemblem s vahami komponent α_t

Zde ϵ_t je podíl trénovacích vzorů, které model v t -té iteraci predikoval nesprávně, a to po vážení vahami W_t .

Vlastnosti: boosting produkuje *posloupnost* klasifikátorů se stejným bazovým učicím algoritmem, kde každý závisí na předchozím a soustředí se na jeho chyby. Při testování se výsledky celé posloupnosti kombinují. **Citlivost na šum** je jeho slabinou: je-li odlehlých hodnot velmi mnoho, důraz na „obtížné“ příklady výkon spíš zhorší.

Náhodné lesy a boosting se tedy hodí do různých situací:

Náhodné lesy	Boosting
rychlé — mohou běžet <i>paralelně</i> a v každém uzlu prohledávají malou množinu příznaků	vyčerpávající — v každé fázi prohledává <i>celou</i> množinu příznaků a každá fáze závisí na předchozí
stromy jsou levné na trénování, lze jich ve stejném čase vypěstovat více i na velkých a složitých datech	musí běžet <i>sekvencně</i> , což může být nákladné
překvapivě dobré výsledky vzhledem k malé části prostoru problému, kterou každý strom vidí	při <i>stejném počtu</i> stromů boosting náhodné lesy překonává (lesy v každé fázi prohledávají jen podmnožinu)

3.2.9 Srovnání klasifikačních metod

Následující tabulka staví probrané metody vedle sebe podle srozumitelnosti výsledného modelu, dosahované přesnosti a rychlosti:

Metoda	Srozumitelnost	Přesnost	Rychlost	Poznámka
Rozhodovací strom	vysoká	střední	vysoká	obdélníkové oblasti, nutné prořezávání, ignoruje korelace atributů
Pravidla z C4.5	nejvyšší	střední	vysoká	prořezávání pravidel je radikálnější než stromů
Naivní Bayes	střední	překvapivě dobrá	velmi vysoká	zvládne nekompletní vzory; problém s nulovými pravděpodobnostmi
SVM	velmi nízká	vysoká	nízká (trénink)	nejlepší pro vysokorozměrná data (text); jen 2 třídy, jen reálný prostor
Logistická regrese	vysoká	střední	vysoká	pravděpodobnostní výstup, interpretovatelné koeficienty
Diskriminační analýza	střední	dobrá při normalitě	vysoká	stačí odhadnout μ_j a S_j
Neuronová síť / ELM	velmi nízká	vysoká	ELM velmi vysoká	ELM negeneruje lokální minima, netrénuje skrytou vrstvu
Náhodný les	nízká	velmi vysoká	vysoká (paralelně)	snižuje rozptyl, nezhoršuje zkreslení
Boosting / AdaBoost	nízká	velmi vysoká	nízká (sekvencně)	snižuje zkreslení; citlivý na šum a odlehlé hodnoty

3.3 Klastrová analýza

3.3.1 Formulace úlohy a míry vzdálenosti

Předchozí oddíly se zabývaly klasifikací, kde byla u každého trénovacího vzoru známa jeho třída. Shlukování žádné takové označení nemá a strukturu si musí v datech najít samo.

Klastrová analýza (shlukování, *cluster analysis*) rozděljuje pozorované vzory do skupin, tedy klastrů či shluků, které sdružují vzájemně podobné vzory. Aby to bylo možné, musí platit jeden **předpoklad**: umíme mezi dvěma vzory změřit *vzdálenost*. Každý vzor je přitom charakterizován m numerickými veličinami, takže vzdálenost se počítá z jejich hodnot.

Míry vzdálenosti

Pro vzory $x_1 = (x_{11}, \dots, x_{1m})$ a $x_2 = (x_{21}, \dots, x_{2m})$ se používají zejména tyto čtyři míry:

1. **Hammingova** vzdálenost (*Manhattan*, L_1) sečte absolutní rozdíly v jednotlivých souřadnicích

cích: $d_H(x_1, x_2) = \sum_{i=1}^m |x_{1i} - x_{2i}|$

2. **Euklidovská vzdálenost** (L_2) je běžná přímá vzdálenost v prostoru: $d_E(x_1, x_2) = \sqrt{\sum_{i=1}^m (x_{1i} - x_{2i})^2}$

3. **Čebyševova vzdálenost** (L_∞) se dívá jen na největší rozdíl mezi souřadnicemi: $d_C(x_1, x_2) = \max_i |x_{1i} - x_{2i}|$

4. **Minkowského metrika** (L_q) všechny tři předchozí zastřešuje, protože jsou jejími speciálními případy: $d_q(x_1, x_2) = \left(\sum_{i=1}^m |x_{1i} - x_{2i}|^q \right)^{1/q}$

Konkrétně platí $d_H = d_1$, $d_E = d_2$ a $d_C = \lim_{q \rightarrow \infty} d_q$. Vezmeme-li vzor $(0, \dots, 0)$ a vzor jednotkové vzdálenosti, dají všechny tři metriky shodně hodnotu 1.

Kterou metriku zvolit, závisí na zúčastněných veličinách a na jejich rozsahu. Veličina měřená ve velkých číslech by jinak přehlušila ostatní, a proto je nutné veličiny **normalizovat**: dělením průměrem, směrodatnou odchylkou, rozsahem max – min a podobně (viz kap. 2). Uvedené metriky navíc mlčky předpokládají, že všechny veličiny mají *stejný rozptyl*. Mají-li rozptyl *různý*, sáhne se po jiné míře:

Mahalanobisova vzdálenost

$$d_M^2(x_1, x_2) = (x_1 - x_2)^T S^{-1} (x_1 - x_2),$$

kde S je kovarianční matice. Tato míra zohledňuje jak různé rozptyly veličin, tak korelace mezi nimi; pro $S = I$ se redukuje na kvadrát euklidovské vzdálenosti.

Vzdálenost mezi jednotlivými vzory ale nestačí. Shlukovací algoritmy potřebují umět změřit i vzdálenost dvou celých skupin, a na to existuje několik různých konvencí.

Vzdálenost mezi dvěma klastry C_k a C_l

Metoda nejbližšího souseda (*single linkage*) bere minimum ze vzdáleností jejich prvků: $D(C_k, C_l) = \min_{x_i, x_j} d(x_i, x_j)$, $x_i \in C_k$, $x_j \in C_l$.

Metoda nejvzdálenějšího souseda (*complete linkage*) pracuje zrcadlově a vezme maximum: $D(C_k, C_l) = \max_{x_i, x_j} d(x_i, x_j)$.

Metoda průměrné vzdálenosti (*average linkage*) zprůměruje vzdálenosti mezi všemi dvojicemi vzorů, přičemž n_k a n_l jsou počty vzorů v obou klastrech:

$$D(C_k, C_l) = \frac{1}{n_k n_l} \sum_{i=1}^{n_k} \sum_{j=1}^{n_l} d(x_i, x_j).$$

Centroidní metoda měří vzdálenost středů klastrů, tedy $D(C_k, C_l) = d(c_k, c_l)$, kde c_k a c_l jsou středy (centroidy) obou klastrů.

Centroid je střed klastru a slouží jako jeho „prototyp“, tedy jediný bod, který celý klastř zastupuje. Tentýž klastř přitom může být reprezentován jedním centroidem, ale i více centroidy současně; záleží na tvaru klastru a na zvolené metrice.

3.3.2 Metoda k -means

Nejrozšířenější shlukovací algoritmus vychází z prosté úvahy: klastř je popsán svým centroidem a vzor patří tam, kde má centroid nejbližší. Jenže jedno závisí na druhém, protože centroidy se počítají z rozdělení vzorů a rozdělení vzorů se řídí centroidy. Řešením je obě části střídavě přepočítávat tak dlouho, dokud se rozdělení neustálí.

Algoritmus k -means (Lloyd, 1982)

- 1: rozděl vzory *náhodně* do k klastrů $C_1, \dots, C_k$ (disjunktních, ve sjednocení všechny vzory)
- 2: urči centroidy všech klastrů v aktuálním rozdělení, např. $c_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$

```

3: for každý vzor  $x$  do
4:   urči vzdálenosti  $d(x, c_k)$  pro  $1 \leq k \leq K$ 
5:   nechť  $d(x, c_l) = \min_k d(x, c_k)$ 
6:   if  $x$  nepatří do klastru  $l$  then
7:     přesuň  $x$  do klastru  $l$ 
8:   end if
9: end for
10: if došlo k přesunu then
11:   goto krok 2
12: else
13:   konec
14: end if

```

Jakmile algoritmus skončí, jsou klastry reprezentovány svými centroidy.

Varianty. Místo náhodného počátečního rozdělení lze za centroidy rovnou prohlásit *prvních k vzorů*, čímž v první iteraci odpadá krok 2. Druhá obměna upravuje centroidy *po každém* přesunu, tedy uvnitř cyklu, a ne až po jeho skončení.

Rozšíření Lloydova algoritmu. Střídání minimalizačních kroků neposkytuje žádnou záruku aproximace. Lloydův algoritmus ale cenu počátečního shlukování *nikdy nezvýší*, takže výsledek stojí a padá s tím, odkud se vyjde. Vyplatí se proto dodat mu *prokazatelně dobrou inicializaci*:

- **k -means++** (Arthur, Vassilvitskii, 2007) vybírá k centroidů z P vzorů postupně, a to s pravděpodobností úměrnou kvadrátu vzdálenosti k nejbližšímu už existujícímu centroidu: $\frac{d^2(x_i, c)}{\sum_{j=1}^P d^2(x_j, c)}$. Vzdálené body tedy mají větší šanci stát se centroidem.
- **LocalSearch++** (Lattanzi, Sohler, 2019) nejprve spustí k -means++ pro volbu k počátečních centroidů. Poté vzorkuje nové vzory s analogickou pravděpodobností a nahrazuje jimi existující centroidy, ovšem jen tehdy, pokud se tím sníží celková kvadratická vzdálenost vzorů k jejich nejbližším centroidům.

Průměr není jediná možná volba reprezentanta. **Metoda k -medians** používá jako účelovou funkci Manhattanovu vzdálenost, tedy $d(x_i, c) = \|x_i - c\|_1$. Lze ukázat, že optimálním reprezentantem c je pak *medián* datových bodů v klastru, počítaný *podél každé dimenze zvlášť*. Právě proto, že se medián volí v každé dimenzi nezávisle, výsledný m -rozměrný reprezentant obvykle *nepatří* do původní datové množiny. Zisk je ve větší odolnosti: k -medians volí reprezentanty *robustněji* než k -means, protože medián není na odlehlé hodnoty tak citlivý jako průměr.

3.3.3 Metoda k -medoids, CLARA a CLARANS

k -means i k -medians reprezentanta *dopočítávají*, takže výsledný bod v datech ležet nemusí. Následující rodina metod jde opačnou cestou a za reprezentanty prohlašuje přímo některé z pozorovaných bodů.

Algoritmus k -medoids

Pro datovou množinu D o n d -rozměrných bodech $x_1, \dots, x_n$ se hledá k reprezentantů $y_1, \dots, y_k$ z *původní databáze* D , jejichž počet k zadává uživatel. Reprezentanti mají minimalizovat účelovou funkci danou součtem vzdáleností bodů k jejich nejbližším reprezentantům:

$$O = \sum_{i=1}^n \min_j d(x_i, y_j).$$

Jedna iterace stojí $O(knd)$ času a obvykle stačí malý konstantní počet iterací.

Proč volit reprezentanty z D ? Důvody jsou dva. (1) Klastry, které najde k -means, mohou být zkresleny odlehlými hodnotami: reprezentanti se pak ocitnou v prázdných oblastech, jež

nejsou pro většinu bodů klastru reprezentativní, a různé klustry mohou částečně splynout. Částečně se to dá řešit pečlivým zacházením s odlehklými hodnotami. (2) U složitých dat, například u množiny časových řad různé délky, může být obtížné optimálního centrálního reprezentanta vůbec spočítat. k -medoids lze proto definovat prakticky pro *libovolný typ dat*; stačí poskytnout vhodnou funkci podobnosti či vzdálenosti.

Postup je *hill-climbing*. Množina reprezentantů S se inicializuje body z D a pak se iterativně zlepšuje tím, že se bod z S vymění za datový bod z D . Které výměny zkusíme, řeší dvě strategie:

1. Vyzkoušet všech $|S| \cdot |D|$ možností a vybrat tu nejlepší. Je to *extrémně drahé*.
2. Pracovat s *náhodně vybranou* množinou r párů (x, y) , kde $x \in D$ a $y \in S$, a provést nejlepší z nich. Tato varianta vyžaduje čas proporcionální $r \cdot |D|$. Algoritmus konverguje, když se účelová funkce už nezlepší, nebo když průměrné zlepšení klesne pod uživatelem zadaný práh. Ve srovnání s k -means je k -medoids *pomalejší*, zato ho lze použít na různé typy dat.

Škálovatelné varianty: CLARA a CLARANS

V mnoha aplikacích jsou data tak velká, že se nevejdou do hlavní paměti a musí zůstat na disku, což svazuje ruce při návrhu algoritmu. CLARA a CLARANS jsou škálovatelné implementace k -medoids a dědí po bazovém algoritmu výhody i nevýhody: CLARANS je sice snadno generalizovatelný na různé typy dat, ale nese si i jeho poměrně vysokou výpočetní složitost.

CLARA (*Clustering LARge Applications*) vychází z varianty k -medoids zvané PAM (*Partitioning Around Medoids*). PAM testuje na výměnu *všech* $k(n - k)$ párů medoid–nemedoid, vymění pár s nejlepším zlepšením účelové funkce a takto pokračuje až do konvergence k lokálnímu optimu. Každá iterace si vyžádá $O(kn^2)$ výpočtů vzdálenosti, tedy $O(kn^2d)$ času, což je *značně drahé*. CLARA proto celý algoritmus aplikuje jen na menší *vzorek* velikosti $f \cdot n$ (kde $f \ll 1$) a zbylé nevzorkované body pouze přiřadí k nalezeným medoidům. Postup se zopakuje nad několika nezávisle volenými vzorky stejné velikosti a vybere se nejlepší nalezené shlukování. Složitost iterace tím klesne na $O(kf^2n^2d + k(n - k))$, což je pro malá f o řády rychlejší. **Hlavní problém** zůstává: předvybrané vzorky nemusí dobrou volbu medoidů vůbec obsahovat.

CLARANS (*Clustering Large Applications based on RANdomized Search*) se tomuto problému vyhýbá tím, že pracuje s *plnými* daty. Iterativně zkouší výměny *náhodných* medoidů za *náhodné* nemedoidy a po každém pokusu kontroluje, zda se kvalita zlepšila. Když ano, výměna zůstane definitivně; když ne, přičte se neúspěšný pokus. Lokální optimum je nalezeno, jakmile počet neúspěšných pokusů dosáhne uživatelem zadané meze *MaxAttempt*. Celý proces se opakuje *MaxLocal*-krát a vybere se nejlepší z nalezených lokálních optim. Díky tomu CLARANS prozkoumá *větší rozmanitost* prostoru řešení než CLARA.

3.3.4 Hierarchické shlukování

Všechny dosavadní metody vyžadovaly, aby uživatel počet klastrů k zadal dopředu. To je nepříjemné, protože právě to obvykle nevíme. Hierarchické metody volbu odsouvají až na konec: vybudují celou posloupnost postupně vnořených rozdělení a teprve z ní se vybere ta úroveň, která dává smysl.

Aglomerativní hierarchické shlukování

Tento přístup jde „zdola nahoru“ (*bottom-up*).

Inicializace má dva kroky: (1) určí vzájemné vzdálenosti mezi všemi vzory a (2) přiřadí každý vzor do samostatného klastru.

Hlavní cyklus běží, dokud je klastrů více než jeden. V každém průchodu (1) najdi dva vzájemně nejbližší klustry a slouč je a (2) pro nově vzniklý klustr spočítej jeho vzdálenost od ostatních klastrů.

Výsledek se zobrazuje jako **dendrogram**, který zleva doprava ukazuje postupné slučování klastrů; výška spojení odpovídá vzdálenosti, při níž ke sloučení došlo. *Optimální počet klastrů není znám předem* a volí se až dodatečně, řezem dendrogramu ve vhodné výšce.

Opačným směrem postupuje **divizivní** hierarchické shlukování („shora dolů“). Začíná jediným klastrem, který obsahuje všechny vzory, a ten postupně dělí. Nejznámějším příkladem v kontextu sítí je Girvanové–Newmanův algoritmus (kap. 6).

CURE (*Clustering Using REpresentatives*)

Aglomerativní hierarchický algoritmus, který je *velmi rychlý* a umí odhalit klustry *libovolného tvaru*, což třeba CLARANS nedokáže. Postupuje v pěti krocích:

1. Vyber vzorek s bodů z databáze D o n bodech.
2. Rozděť těchto s bodů na p částí velikosti s/p .
3. Každou část shlukuj *nezávisle* hierarchickým slučováním do k' klastrů. Celkový počet $k' \cdot p$ klastrů přes všechny části je stále větší než uživatelem požadované k .
4. Nad těmito $k' \cdot p$ klustry proveď hierarchické shlukování až na požadovaných k .
5. Každý z $(n - s)$ nevzorkovaných bodů přiřaď ke klastru s nejbližším reprezentantem.

3.3.5 Vektorová kvantizace: algoritmus LVQ

Předchozí metody si vystačily s neoznačenými daty. LVQ je proti nim výjimkou: pracuje se vzory, u nichž třídu známe, a tuto informaci využívá k posouvání reprezentantů.

LVQ — *Learning Vector Quantization*

Slouží pro **shlukování s učitelem** (třídy jsou označeny C):

- 1: inicializuj všechny váhové vektory $w_j(0)$, rychlosti učení $\mu(0)$, $k \leftarrow 0$
- 2: **while** není splněna zastavovací podmínka **do**
- 3: **for** každý trénovací vzor x **do**
- 4: urči index $j = q$ váhového vektoru s *minimální* euklidovskou vzdáleností, tj. $\min_j \|x - w_j\|_2^2$
- 5: **if** třída x souhlasí s třídou w_q **then**
- 6: $w_q(k+1) \leftarrow w_q(k) + \mu(k)(x - w_q(k))$ ▷ přitáhni
- 7: **else**
- 8: $w_q(k+1) \leftarrow w_q(k) - \mu(k)(x - w_q(k))$ ▷ odtlač
- 9: **end if**
- 10: **end for**
- 11: $k \leftarrow k + 1$; sniž rychlost učení, např. $\mu(k) = \mu(k-1)/(k+1)$ pro $k > 0$
- 12: **end while**

Váhové vektory se obvykle inicializují *prvními m vektory* trénovací množiny, které musí zahrnovat vzory ze všech tříd. Neurony se tím automaticky „kalibrují“ s třídami.

3.3.6 Mřížkové a hustotní metody

Metody postavené na reprezentantech mají společné omezení: klastr je u nich vždy okolí jednoho bodu, takže vycházejí kulovité. Protáhlé či jinak nepravidelné tvary vyžadují změnu úhlu pohledu. Neptáme se, jak daleko je bod od středu, ale kde v prostoru je bodů hodně a kde málo.

Mřížkové metody (např. MAFIA)

- Data se nejdřív diskretizují do p (ekvidistantních) intervalů, což pro d -rozměrná data dá p^d hyperkrychlí.
- Tyto hyperkrychle pak slouží jako stavební bloky pro shlukování.
- Pomocí prahu hustoty τ se identifikují *husté* hyperkrychle, tedy ty s alespoň τ body.
- Klastr libovolného tvaru se skládá z více hustých oblastí, které jsou *propojeny*. Dvě mřížkové oblasti přitom považujeme za propojené, sdílejí-li stěnu.

Cílem je určit klustry jako husté a vzájemně propojené oblasti. Formálně: hustým mřížkovým

buňkám odpovídají uzly grafu a hrany reprezentují sousedskou propojenost. Data pokrytá buňkami z jedné komponenty souvislosti tohoto grafu pak tvoří jeden výsledný klastr.

DBSCAN — *Density-Based Spatial Clustering of Applications with Noise*

Myšlenka je podobná, jen stavebními bloky nejsou mřížkové buňky, ale *jednotlivé datové body*, přesněji kruhové oblasti okolo nich. Hustota je dána počtem bodů ležících v jejich poloměru ε (včetně bodu samotného) a podle ní se body dělí na tři druhy:

1. **Jádrový bod** (*core point*) je takový, jehož kulová oblast o poloměru ε obsahuje alespoň τ datových bodů.
2. **Hraniční bod** (*border point*) má ve své oblasti *méně* než τ bodů, ale zároveň v ní leží alespoň jeden jádrový bod ve vzdálenosti ε .
3. **Šumový bod** (*noise point*) není ani jádrový, ani hraniční.

Algoritmus má tři kroky:

1. Určí jádrové, hraniční a šumové body.
2. Zkonstruuje graf propojenosti, v němž každý uzel odpovídá jádrovému bodu. Hrana mezi dvěma jádrovými body existuje *právě když* leží od sebe ve vzdálenosti nejvýše ε .
3. Najde všechny komponenty souvislosti tohoto grafu; ty odpovídají klastrům postaveným na jádrových bodech. Hraniční body se přiřadí ke klastru, s nímž mají nejvyšší úroveň propojenosti, a šumové body se ohlásí jako *odlehle hodnoty*.

Probrané metody se dají srovnat podle několika mála otázek: jaký tvar klastrů zvládnou najít, jestli je nutné jejich počet zadat předem a jak se chovají vůči odlehlým hodnotám.

Metoda	Tvar klastrů	Počet klastrů	Odehlé hodnoty	Poznámka
k -means	kulovité	zadán	citlivá	rychlá; závislá na inicializaci $\rightarrow k$ -means++
k -medians	kulovité	zadán	robustnější	medián po dimenzích; reprezentant není z dat
k -medoids	kulovité	zadán	robustní	reprezentanti z dat; libovolný typ dat; pomalejší
CLARA / CLARANS	kulovité	zadán	robustní	škálovatelné k -medoids (vzorkování / náhodné prohledávání)
Hierarchické	podle metriky spojování	určí se z dendrogramu	citlivé	bez nutnosti zadat k předem; $O(n^2)$ – $O(n^3)$
CURE	libovolný	zadán	robustní	vzorkování + hierarchické slučování po částech
LVQ	kulovité	dán počtem neuronů	—	shlukování s učitelem
Mřížkové (MAFIA)	libovolný	vyplyne	šum vypadne	citlivé na granularitu mřížky
DBSCAN	libovolný	vyplyne	<i>detekuje je</i>	parametry ε, τ ; problém s různou hustotou klastrů

3.4 Shrnutí ke zkoušce

Asociační pravidla.

- **Support** je podíl transakcí obsahujících i i j , kdežto **confidence** je podíl transakcí s i i j mezi transakcemi obsahujícími i ; **lift** se spočítá jako $p(i \wedge j) / (p(i)p(j))$. Je-li $\text{lift} < 1$, je pravidlo horší než náhoda a lepší může být negace jeho závěru. Typický úkol u tabule je spočítat všechny tři veličiny z malé tabulky transakcí.
- **Apriori** stojí na *uzavřenosti dolů*, tedy na tom, že každá podmnožina frekventované množiny je sama frekventovaná, a prohledává *do šířky*. Procedura APRIORI-GEN se skládá z kroku spojení, který kombinuje množiny shodné v $k - 2$ kategoriích, a z kroku prořezání, jenž vyřadí kandidáta,

chybí-li některá jeho podkombinace délky $k - 1$. Prostor pravidel má velikost $O(2^m)$ a MBA produkuje *nadměrné množství pravidel*.

- **Taxonomie a virtuální položky** řeší otázku, na jaké úrovni obecnosti položky brát: vzácné položky se posunou výše v taxonomii, časté níže. Virtuální položky nesou informaci o transakci samotné (den, čas, zákazník); u obojího hrozí redundance pravidel.
- **MS-Apriori** útočí na problém vzácných položek tím, že každá položka dostane vlastní MIS; minsup celého pravidla je pak *minimum* z MIS jeho položek, k čemuž se přidává omezení rozdílu supportů φ . Cenou za to je, že *uzavřenost dolů přestává platit*, a algoritmus se musí přizpůsobit: položky se setřídí vzestupně podle MIS, místo F_1 se pracuje s množinou semen L a pro generování pravidel slouží **tailCount**. U zkoušky se hodí umět uvést protipříklad k uzavřenosti dolů.
- **CAR** pracuje s transakcemi označovanými třídou a hledá pravidla tvaru $X \rightarrow y$. Dolují se *přímo v jednom kroku* přes pravidlové položky (*condset*, y) a různým třídám lze zadat různé minsupy, přičemž hodnota 101 % danou třídu fakticky zakáže.
- **Sekvenční vzory**: sekvence je uspořádaný seznam množin položek, její *velikost* je počet prvků a *délka* počet položek; podsekvence se definuje přes vnořené inkluze. Algoritmus **GSP** pracuje obdobně jako Apriori. Pozor na to, že $\langle\{3\}\{8\}\rangle$ a $\langle\{3, 8\}\rangle$ se navzájem *neobsahují*.
- **FP-Growth** si vystačí se dvěma průchody daty a negeneruje žádné kandidáty. FP-strom sdílí *prefixy*, k čemuž slouží heuristické uspořádání položek podle klesajícího supportu, a doplňují ho spojové seznamy položek. Extrakce jde zdola nahoru metodou *rozděl a panuj* přes *prefixové podstromy* a *podmíněné FP-stromy*: aktualizovat počítadla, odstranit uzly položky a odstranit nefrekventované položky. V nejhorším případě, kdy jsou všechny transakce unikátní, vyjde strom větší než samotná data.

Učení s učitelem.

- **Rozhodovací stromy** se budují metodou TDIDT, tedy rozděl a panuj, a atribut se volí podle *minimální vážené entropie* $H(A) = \sum_v \frac{|A(v)|}{|D|} H(A(v))$, což je ekvivalentní maximálnímu *informačnímu zisku*. Očekává se schopnost projít příklad s úvěrem (příjem 0,5747; konto 0,6667; pohlaví 0,9183; nezaměstnaný 0,825) a převést hotový strom na pravidla. Indukce stojí $O(np \log p)$, klasifikace jednoho vzoru $O(q \log p)$.
- **ID3** (Quinlan 1986) je hladový a řídí se informačním ziskem. **C4.5** (1993) přidává práci s chybějícími a spojitými daty, prořezávání validací (*reduced-error pruning*, *subtree raising*), *rule post-pruning*, odhad přesnosti dolní mezí $acc - 1,96\sqrt{acc(1 - acc)/n}$ a *poměrný* informační zisk, který penalizuje nadměrné větvení. **C5.0** je komerční verze s boostingem. **CART** staví *binární* stromy, používá Gini index $G(v_i) = 1 - \sum_j p_j^2$ (nižší je lepší) a kritérium $\Phi = 2P_L P_R \sum_j |P(C_j|t_L) - P(C_j|t_R)|$. **CHAID** se opírá o χ^2 a automaticky seskupuje hodnoty kategoriálních atributů.
- **Bayesovská klasifikace** hledá $H_{MAP} = \arg \max_H P(E | H)P(H)$, přičemž jmenovatel lze zanedbat. *Naivní* předpoklad podmíněné nezávislosti vede na $\arg \max_{H_t} P(H_t) \prod_i P(E_i | H_t)$. Metoda si poradí s nekompletními vzory, naráží ale na problém nulových pravděpodobností. Příklad je potřeba umět spočítat (0,1042 vs. 0,0416).
- **SVM** maximalizuje okraj $2/\|w\|$, což vede na primární úlohu $\min \frac{\langle w \cdot w \rangle}{2}$ za podmínek $y_i(\langle w \cdot x_i \rangle + b) \geq 1$. Kuhn–Tuckerovy podmínky jsou pro konvexní úlohu s lineárními omezeními nejen nutné, ale *i postačující*; podmínka $\alpha_i > 0$ platí jen pro *podpůrné vektory* a úloha se řeší přes Wolfoho duál. **Kernelový trik** využívá toho, že duál potřebuje jen skalární součiny, takže stačí zvolit $K(x, z) = \langle \phi(x) \cdot \phi(z) \rangle$ a samotné ϕ nikdy nevyčíslíme (Mercerova věta). Omezení: jen 2 třídy, jen reálný prostor a neinterpretovatelný výsledek.
- **Logistická regrese** modeluje $\ln \frac{p}{1-p} = \beta^\top x$, což dává $p = \sigma(z) = 1/(1 + e^{-z})$. Učí se maximální věrohodností, ekvivalentně minimalizací křížové entropie, a protože $\sigma' = \sigma(1 - \sigma)$, vychází adaptační pravidlo $\Delta \beta_j = \eta \sum_i (y_i - \sigma(z_i)) x_{ij}$.
- **Diskriminační analýza** zavádí pro každou třídu funkci $g_j(x)$ a klasifikuje se podle jejího maxima; pro dvě třídy stačí znaménko rozdílu $g = g_1 - g_2$. Optimální volbou je $g_j \propto P(C_j | x)$. Pro normální rozdělení vyjde funkce kvadratická, a jsou-li kovarianční matice shodné, dokonce *lineární* — pak stačí odhadnout μ_j a S .

- **ELM** volí parametry skrytých neuronů náhodně, dokonce i bez znalosti dat, a výstupní váhy dopočítá *jedinou* pseudoinverzí $\beta = H^+T$; regularizace se přidá jako I/C na diagonálu. Je proto rychlé, nemá lokální minima a funguje s libovolnou omezenou po částech spojitou přenosovou funkcí.
- **Ensembley**: *bagging* snižuje *rozptyl*, pro i.i.d. komponenty z σ^2 na σ^2/k , přičemž bootstrapový vzorek obsahuje přibližně 63,2 % různých bodů, tedy $1 - 1/e$. Ideální komponentou jsou stromy s nízkým zkreslením a vysokým rozptylem; zkreslení bagging nesnižuje. *Náhodné lesy* navíc řeší korelaci komponent, protože rozptyl je $\rho\sigma^2 + (1 - \rho)\sigma^2/k$ a první člen na k vůbec nezávisí; náhodnost se vnáší jednak bootstrapem, jednak náhodnou podmnožinou $m \approx \sqrt{\#\text{příznaků příznaků}}$ v každém uzlu. *Boosting* a konkrétně **AdaBoost** naopak snižuje *zkreslení*: váhy chybně klasifikovaných vzorů rostou, hlas komponenty je $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$, běh je *sekvenční* a metoda je citlivá na šum. U zkoušky se často ptá na srovnání lesů a boostingu: lesy jsou paralelní a levné, boosting vyčerpávající a při stejném počtu stromů přesnější.

Klastrová analýza.

- **Metriky**, které je třeba znát: Hammingova neboli Manhattan (L_1), euklidovská (L_2), Čebyševova (L_∞) a Minkowského (L_q), jejímiž speciálními případy ostatní jsou. Pro veličiny s různým rozptylem slouží **Mahalanobisova** vzdálenost $d_M^2 = (x_1 - x_2)^T S^{-1}(x_1 - x_2)$. Před shlukováním je vždy potřeba data *normalizovat*.
- **Vzdálenost dvou klastrů** se měří čtyřmi způsoby: metodou nejbližšího souseda (minimum), nejvzdálenějšího souseda (maximum), průměrné vzdálenosti a centroidní metodou.
- **k-means** (Lloyd 1982) vyjde z náhodného rozdělení, spočítá centroidy, přesune vzory k nejbližšímu centroidu a opakuje, dokud k přesunům dochází. Cenu shlukování nikdy nezvýší, ale nedává záruku aproximace, a proto se doplňuje o lepší inicializaci: **k-means++** volí centroid s pravděpodobností úměrnou kvadrátu vzdálenosti k nejbližšímu už zvolenému centroidu, **LocalSearch++** na to navazuje výměnami.
- **k-medians** používá Manhattanovu vzdálenost a medián po dimenzích, takže je robustnější, zatímco **k-medoids** volí reprezentanty z *dat*, zlepšuje je metodou hill-climbing s výměnami, stojí $O(knd)$ na iteraci a zvládne libovolný typ dat. **PAM** testuje všech $k(n - k)$ výměn za cenu $O(kn^2d)$; **CLARA** pracuje se vzorkem, čímž se dostane na $O(kf^2n^2d + k(n - k))$, ovšem s rizikem špatně zvoleného vzorku, a **CLARANS** prohledává náhodně nad plnými daty s parametry *MaxAttempt* a *MaxLocal*.
- **Hierarchické shlukování** zdola nahoru začíná s každým vzorem ve vlastním klastru a postupně slučuje dva nejbližší; výstupem je **dendrogram** a počet klastrů se volí až jeho řezem. **CURE** vezme vzorek, rozdělí ho na p částí, každou nezávisle shlukuje hierarchicky na k' klastrů, výsledné klastry hierarchicky sloučí na k a nakonec přiřadí zbytek bodů; je rychlé a zvládne klastry libovolného tvaru.
- **LVQ** je shlukování s *učitelem*: vítězný váhový vektor se ke vzoru přitáhne, souhlasí-li třídy, a odtlačí, když nesouhlasí. Rychlost učení μ přitom klesá jako $\mu(k - 1)/(k + 1)$.
- **Mřížkové metody** (MAFIA) rozdělí prostor na p^d hyperkrychlí, prahem hustoty τ vyberou husté buňky a klastry sestaví jako propojené husté oblasti, formálně jako komponenty souvislosti grafu buněk. **DBSCAN** dělí body na *jádrové* (alespoň τ bodů v okolí o poloměru ϵ), *hraniční* (méně než τ , ale jádrový bod do vzdálenosti ϵ) a *šumové*; klastry jsou komponenty souvislosti grafu jádrových bodů a šumové body se hlásí jako odlehlé. Zvládne tedy klastry *libovolného tvaru* a odlehlé hodnoty detekuje sám.

4 Charakteristická a diskriminační pravidla a jejich zajímavost

[♦ nad rámec slidů] Celá kapitola stojí nad rámec slidů obou kurzů. Okruh ji ale jmenuje výslovně a jde o standardní látku, která je zde zpracovaná podle Han–Kamber. Jak generalizace souvisí s OLAP, ukazuje kap. 1.

4.1 Charakterizace vs. diskriminace

Deskriptivní dobývání znalostí odpovídá na dvě otázky, které spolu těsně souvisejí, a přesto se nesmějí zaměňovat. První se ptá, jak vybraná skupina objektů vypadá; druhá, čím se tato skupina liší od zbytku dat.

Charakterizace vs. diskriminace

Charakterizace dat (*data characterization*) shrnuje obecné vlastnosti *cílové třídy* (*target class*). Odpovídá tedy na otázku „Jak vypadá typický klient s hypotékou?“ a jejím výsledkem jsou *charakteristická pravidla* (*characteristic rules*).

Diskriminace dat (*data discrimination*) porovnává obecné vlastnosti cílové třídy s vlastnostmi jedné či více *kontrastních tříd* (*contrasting classes*). Odpovídá tedy na otázku „Čím se klient s hypotékou liší od klienta bez hypotéky?“ a jejím výsledkem jsou *diskriminační pravidla* (*discriminant rules*).

Rozdíl mezi oběma úlohami je zásadní. Vlastnost může být pro cílovou třídu *typická*, tedy může ji mít vysoký podíl objektů uvnitř třídy, a přesto nemá žádnou *rozlišovací sílu*, protože je stejně typická i pro kontrastní třídu.

Obě úlohy řeší jedna a tatáž metoda, *indukce orientovaná na atributy* (*attribute-oriented induction*, AOI), kterou navrhli Han, Cai a Cercone (1991).

Alternativou k ní je datově-kostkový (*data cube*) přístup známý z OLAP, ve kterém generalizaci odpovídá operace *roll-up* a specializaci operace *drill-down*. Každý z obou přístupů má svou přednost: AOI je flexibilnější, protože si poradí i s atributy, pro které není kostka předem materializovaná, kdežto OLAP je rychlejší, protože sahá po předpočítaných agregacích.

4.2 Koncepční hierarchie

Aby se dala hodnota atributu nahradit obecnější, musí být napřed řečeno, co je vůči čemu obecnější. Tuto informaci nese koncepční hierarchie.

Koncepční hierarchie

Koncepční hierarchie (*concept hierarchy*) atributu *A* je částečné uspořádání jeho hodnot od nejkonkrétnějších (listy) po nejobecnější (kořen ANY). Podle toho, odkud se uspořádání bere, se rozlišují čtyři typy:

- *schémátovou* (*schema hierarchy*) dává přímo struktura dat: ulice $\prec$ město $\prec$ kraj $\prec$ stát;
- *seskupovací* (*set-grouping*) vzniká tím, že se hodnoty explicitně seskupí, třeba {20–29} $\prec$ mladý;
- *operační* (*operation-derived*) se odvodí výpočtem z hodnoty samotné: e-mailová adresa $\rightarrow$ doména;
- *pravidlová* (*rule-based*) je definovaná pravidly, která smějí sahat přes více atributů najednou.

Hierarchii buď zadá expert, nebo se generuje automaticky: u numerických atributů diskretizací, u kategoriálních podle počtu různých hodnot, protože atribut s více různými hodnotami leží obvykle na nižší úrovni hierarchie.

4.3 Indukce orientovaná na atributy (AOI)

Myšlenka AOI je jednoduchá. Dokud je tabulka příliš podrobná, nahrazují se hodnoty obecnějšími a řádky, které se tím staly shodnými, se slučují do jednoho.

Algoritmus AOI

Vstup: databáze, dotaz vymezující cílovou třídu, seznam atributů, prahy generalizace T_i pro jednotlivé atributy a celkový práh T pro velikost výsledné relace.

- 1: **sběr dat:** vyber dotazem množinu objektů cílové třídy (*initial working relation*)
- 2: ke každému objektu přidej atribut **count** = 1
- 3: **for** každý atribut A_i **do**
- 4: **if** A_i má mnoho různých hodnot a *neexistuje* pro něj koncepční hierarchie **then**
- 5: **odstranění atributu** (*attribute removal*)
- 6: **else if** A_i má mnoho různých hodnot, ale hierarchie existuje **then**
- 7: **generalizace atributu** (*attribute generalization*): nahraď hodnoty
- 8: jejich předky o úroveň výš, dokud počet různých hodnot $\leq T_i$
- 9: **end if**
- 10: **end for**
- 11: **agregace:** slouč identické zobecněné objekty a sečti jejich **count**
- 12: **while** počet řádků výsledné relace $> T$ **do**
- 13: zobecni dále atribut, jehož generalizace nejvíce sníží počet řádků
- 14: **end while**
- 15: **prezentace:** zobraz *primární relaci* (*prime relation*) jako tabulku, křížovou
- 16: tabulku, graf, nebo ji převed' na kvantitativní pravidla

Veškerou práci přitom odvedou **dvě základní generalizační operace:**

- **Odstranění atributu** přichází ke slovu tehdy, má-li atribut příliš mnoho různých hodnot a zároveň buď (a) pro něj neexistuje generalizační operátor, nebo (b) jeho vyšší úrovně jsou už vyjádřeny jinými atributy. Takový atribut se z relace jednoduše odstraní; typicky jde o *jméno*, *telefon* nebo *rodné číslo*.
- **Generalizace atributu** se použije tam, kde generalizační operátor existuje, tedy kde je po ruce koncepční hierarchie. Hodnoty se nahradí hodnotami na vyšší úrovni, jak ukazuje řetězec *datum narození* $\rightarrow$ *věk* $\rightarrow$ *věková kategorie* nebo *město* $\rightarrow$ *kraj* $\rightarrow$ *region*.

Kam až generalizace zajde, **řídí dva prahy**. *Prah generalizace atributu* T_i udává maximální povolený počet různých hodnot jednoho atributu a volí se typicky mezi 2 a 8; *prah velikosti relace* T omezuje počet řádků výsledné primární relace, obvykle na 10 až 30. Obě meze je nutné trefit. Příliš vysoká generalizace vede k triviálním výsledkům typu „všichni klienti jsou lidé“, příliš nízká skončí u tabulky, ve které se nikdo nevyzná. Vhodná úroveň se proto hledá interaktivně, střídáním operací drill-down a roll-up.

Cena za to je nízká. Realizuje-li se generalizace přes hašovací tabulku zobecněných záznamů, stačí jediný průchod daty a agregace, takže složitost činí $O(na)$ pro n objektů a a atributů. AOI je tedy *lineární* v počtu objektů, což je pro velké databáze zásadní.

4.4 Míry t -weight a d -weight

Primární relace sama o sobě říká jen to, jaké profily se v datech vyskytují. Aby z ní šlo udělat pravidlo, potřebuje každý zobecněný záznam ještě číslo, které vyjádří jeho váhu. Taková čísla jsou dvě a každé míří na jiný typ pravidla.

t-weight a d-weight

Nechť primární relace obsahuje zobecněné záznamy (*generalized tuples*) $q_1, \dots, q_N$.

Typičnost (*typicality*, t-weight) záznamu q_a v cílové třídě se definuje jako

$$t\text{-weight}(q_a) = \frac{\text{count}(q_a)}{\sum_{i=1}^N \text{count}(q_i)} \in [0, 1],$$

tj. jako podíl objektů cílové třídy, které záznam q_a pokrývá. Přes všechny záznamy primární relace dávají t-weight dohromady 1, tedy 100 %. Tato míra slouží pro *charakteristická* pravidla.

Diskriminační váha (d-weight) záznamu q_a se měří vůči cílové třídě C_{target} a kontrastním třídám $C_1, \dots, C_m$:

$$d\text{-weight}(q_a) = \frac{\text{count}(q_a \in C_{\text{target}})}{\sum_{j=1}^m \text{count}(q_a \in C_j) + \text{count}(q_a \in C_{\text{target}})} \in [0, 1],$$

tj. jako podíl objektů pokrytých záznamem q_a , které patří do cílové třídy. Tato míra slouží pro *diskriminační* pravidla. Vysoká d-weight ($\rightarrow 1$) znamená, že záznam cílovou třídu silně odlišuje, kdežto hodnota blízká apriornímu podílu cílové třídy znamená nulovou rozlišovací schopnost.

Kvantitativní charakteristické, diskriminační a popisné pravidlo

Kvantitativní charakteristické pravidlo vyjadřuje nutnou podmínku, tedy implikaci z třídy:

$$\forall X \text{ target_class}(X) \Rightarrow \text{cond}_1(X) [t: w_1] \vee \dots \vee \text{cond}_N(X) [t: w_N]$$

Čte se: patří-li X do cílové třídy, pak splňuje cond_1 s typičností w_1 , nebo cond_2 s typičností w_2 , ...

Kvantitativní diskriminační pravidlo vyjadřuje naopak postačující podmínku, tedy implikaci *do* třídy:

$$\forall X \text{ target_class}(X) \Leftarrow \text{cond}(X) [d: w]$$

Čte se: splňuje-li X podmínku cond , patří s pravděpodobností w do cílové třídy.

Kvantitativní popisné pravidlo (*quantitative description rule*) obě předchozí podoby kombinuje, takže podmínka je zároveň nutná i postačující:

$$\forall X \text{ target_class}(X) \Leftrightarrow \text{cond}_1(X) [t: w_1, d: w'_1] \vee \dots \vee \text{cond}_N(X) [t: w_N, d: w'_N]$$

Příklad — charakterizace a diskriminace klientů banky

Cílová třída: klienti s hypotékou (200 osob). **Kontrastní třída:** klienti bez hypotéky (800 osob).

Nejprve proběhne AOI: odstraní se *jméno*, *rodné číslo* a *telefon*, generalizuje se *datum narození* $\rightarrow$ *věková kategorie*, *plat* $\rightarrow$ *příjmová kategorie* a *obec* $\rightarrow$ *region*. Vznikne tato primární relace:

Věk	Příjem	Region	počet		t-weight	d-weight
			cíl	kontrast	(cíl)	
30–40	vysoký	Praha	90	10	45 %	90 %
30–40	střední	Praha	50	150	25 %	25 %
40–50	vysoký	mimo Prahu	40	160	20 %	20 %
20–30	nízký	mimo Prahu	20	480	10 %	4 %
Celkem			200	800	100 %	

Výpočet. U prvního záznamu vyjde $t\text{-weight} = 90/200 = 45\%$ a $d\text{-weight} = 90/(90 + 10) = 90\%$. Pro druhý záznam je $t = 50/200 = 25\%$ a $d = 50/(50 + 150) = 25\%$, pro čtvrtý $t = 20/200 = 10\%$ a $d = 20/(20 + 480) = 4\%$.

Kvantitativní charakteristické pravidlo:

$\forall X \text{ hypotéka}(X) \Rightarrow (\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{vysoký} \wedge \text{region}(X)=\text{Praha}) [t: 45 \%]$
 $\vee (\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{střední} \wedge \text{region}(X)=\text{Praha}) [t: 25 \%] \vee \dots$

Kvantitativní diskriminační pravidlo:

$\forall X \text{ hypotéka}(X) \Leftarrow (\text{věk}(X)=30-40 \wedge \text{příjem}(X)=\text{vysoký} \wedge \text{region}(X)=\text{Praha}) [d: 90 \%]$

Interpretace rozdílu. Právě poslední řádek tabulky ukazuje, proč jsou obě míry potřeba. Profil „20–30, nízký příjem, mimo Prahu“ pokrývá 10 % klientů s hypotékou, což není zanedbatelná typičnost, jeho d -weight je ale pouhá 4 %, tedy výrazně *méně* než apriorní podíl cílové třídy $200/1000 = 20 \%$. Tento profil hypotéku spíše *vyklučuje*. U druhého řádku leží d -weight jen nepatrně *nad* apriorním podílem (25 % proti 20 %), takže rozlišovací síla je zanedbatelná. A u třetího řádku je d -weight *přesně rovna* apriornímu podílu (20 %), takže rozlišovací síla je **nulová**: profil o příslušnosti k cílové třídě neříká vůbec nic, přestože pokrývá pětinu klientů s hypotékou.

4.5 Měření zajímavosti pravidel

Když už jsou pravidla na světě, zbývá rozhodnout, která z nich stojí za pozornost. Zajímavost odvozených pravidel se posuzuje obdobně jako u asociačních pravidel (odd. 3.1), s důrazem na následující míry:

Míra	Význam
t -weight	typičnost profilu uvnitř cílové třídy; pravidlo s nízkou t -weight popisuje okrajový jev
d -weight	rozlišovací síla profilu; <i>zajímavá</i> jsou pravidla s d -weight výrazně vyšší (nebo nižší) než apriorní podíl třídy
Lift / zdvih	$\frac{d\text{-weight}}{P(C_{\text{target}})}$ — kolikrát profil zvýší šanci na cílovou třídu
Pokrytí (<i>coverage</i>)	podíl všech objektů pokrytých pravidlem; nízké pokrytí $\Rightarrow$ statisticky nespolehlivé
χ^2	statistická významnost vztahu profilu a třídy
Informační zisk	snížení entropie třídy po znalosti profilu
Jednoduchost	délka pravidla (počet podmínek), počet pravidel; Occamova břitva
Neočekávanost	rozpor s modelem světa uživatele (subjektivní míra)
Akceschopnost	lze na jeho základě jednat (subjektivní míra)

Prakticky je pravidlo zajímavé tehdy, splňuje-li současně čtyři požadavky: musí být (a) *srozumitelné*, (b) *platné* na nových datech s vysokou spolehlivostí, (c) *potenciálně užitečné* a (d) *nové*. První tři se dají měřit objektivně, poslední vyžaduje znalost uživatele.

Z toho plyne **klasická past**. Pravidlo s vysokou t -weight, ale s d -weight rovnou apriornímu podílu třídy není žádným objevem, i když na první pohled vypadá impozantně. Věta „80 % klientů s hypotékou má bankovní účet“ zní dobře jen do chvíle, než si člověk uvědomí, že účet má i 80 % klientů bez hypotéky.

4.6 Shrnutí ke zkoušce

- **Charakterizace** popisuje cílovou třídu, kdežto **diskriminace** ji porovnává s kontrastními třídami. Celý rozdíl se dá shrnout jedním vztahem: typičnost $\neq$ rozlišovací síla.
- **AOI** postupuje tak, že dotazem sebere data a ke každému objektu přidá **count**. Pak každý atribut buď *odstraní* (má-li mnoho hodnot a neexistuje pro něj hierarchie), nebo *generalizuje* podle koncepční hierarchie. Následně agreguje stejné záznamy a generalizuje dál, dokud relace není menší než práh T ; nakonec výsledek prezentuje. Složitost $O(na)$ je lineární v počtu objektů.
- **Koncepční hierarchie** se dělí na schémátovou, seskupovací, operační a pravidlovou.

- **t-weight** = $\frac{\text{count}(q_a)}{\sum_i \text{count}(q_i)}$ měří typičnost uvnitř cílové třídy a její součet přes všechny záznamy dává 100 %. Používá se pro *charakteristická* pravidla, tedy pro nutnou podmínku ($\Rightarrow$).
- **d-weight** = $\frac{\text{count}(q_a \in C_{\text{target}})}{\sum_j \text{count}(q_a \in C_j)}$ měří rozlišovací sílu a používá se pro *diskriminační* pravidla, tedy pro postačující podmínku ($\Leftarrow$). Popisné pravidlo obě podoby kombinuje ($\Leftrightarrow$).
- Obě váhy je potřeba umět spočítat z tabulky a vysvětlit, proč vysoká t-weight neznamena vysokou d-weight; klíčem je srovnání s *apriorním* podílem třídy.
- **Zajímavost** má stránku objektivní (t/d-weight, lift, pokrytí, χ^2 , informační zisk, jednoduchost) a subjektivní (neočekávanost, akceschopnost).

5 Reprezentace, vyhodnocování a vizualizace znalostí

Věta okruhu spojuje tři samostatné otázky a podle nich je členěná i tato kapitola. První se ptá, v jaké formě nalezenou znalost vůbec vyjádříme. Druhá chce vědět, jak ověříme, že je ta znalost dobrá. Třetí se týká toho, jak ji zobrazíme. Prostřední část, tedy vyhodnocování klasifikátorů, je jediná, kterou slidy pokrývají v celém rozsahu, a je zároveň ta, na kterou se u zkoušky ptá nejčastěji.

5.1 Formy reprezentace znalostí

Znalost vydolovaná z dat může mít velmi různou podobu a volba formy rozhoduje o tom, kdo jí nakonec porozumí. Následující přehled shrnuje obvyklé formy reprezentace, metody, z nichž vznikají, a cenu, kterou se za jejich srozumitelnost platí.

♦ nad rámec slidů: tabulka jako přehled; jednotlivé metody v ní jsou ve slidech doloženy]

Forma	Typický zdroj	Vlastnosti
IF-THEN pravidla	asociační pravidla, C4.5	nejrozumitelnější; lokální (každé pravidlo se čte samostatně); riziko konfliktů a nadprodukce
Rozhodovací strom	ID3/C4.5/CART	globální model, snadno čitelný do hloubky 4–5; nad ni nepřehledný
Rozhodovací tabulka	diskretizovaná data	úplný výčet kombinací; přehledná jen pro málo atributů
Matematická funkce	regrese, SVM, NN	kompaktní a přesná; obtížně interpretovatelná (kromě lineárního modelu s normalizovanými vahami)
Prototypy / centroidy	k -means, k -medoids, LVQ	„typický zástupce“ skupiny; velmi intuitivní pro byznys
Pravděpodobnostní model	naivní Bayes, bayesovská síť	explicitní nejistota; graf sítě je čitelný
Grafy a sítě	analýza sociálních sítí	vztahy mezi entitami; vizuálně silné
Instance (paměť případů)	k -NN, CBR	„model“ je samotná data; vysvětlení odkazem na podobné případy

Interpretovatelnost a vysvětlitelnost

♦ nad rámec slidů: vysvětlitelnost a post-hoc metody; interpretovatelnost jako kritérium hodnocení modelu je ve slidech — viz odd. 5.2] Interpretovatelnost (*interpretability*) je schopnost člověka porozumět tomu, jak model dospěl k výstupu. Vysvětlitelnost (*explainability*) je schopnost podat srozumitelné odůvodnění konkrétního rozhodnutí.

Modely se z tohoto hlediska dělí na dvě skupiny. *Inherentně interpretovatelné modely* umožňují nahlédnout přímo dovnitř: patří sem lineární modely, rozhodovací stromy, sady pravidel a prototypy. U ostatních zbývá *post-hoc vysvětlení černých skříněk*, tedy dodatečná analýza už hotového modelu. Do této skupiny patří důležitost atributů (permutační, SHAP), lokální surogátní modely (LIME), grafy částečné závislosti (PDP), extrakce pravidel z neuronové sítě, analýza citlivosti a kontrafaktuální vysvětlení typu „kdyby byl váš příjem o 5000 vyšší, úvěr by byl schválen“.

Interpretovatelnost přitom není jen pohodlí analytika. V regulovaných doménách, jako jsou úvěry, pojištění nebo medicína, ji vyžaduje právo: GDPR ve svém čl. 22 zakládá právo na vysvětlení automatizovaného rozhodnutí a totéž platí podle AI Actu pro vysoce rizikové systémy.

5.2 Vyhodnocování modelů

Model, který nikdo neposoudil, není znalost, ale hypotéza. Následující oddíly proto postupně odpovídají na tři otázky: podle čeho se klasifikační metody vůbec porovnávají, jak se odhaduje jejich chyba a jakými mírami se popisuje kvalita klasifikace, když samotná přesnost nestačí.

5.2.1 Kritéria hodnocení klasifikačních metod

Kvalita modelu není jediné číslo. Klasifikační metody se posuzují podle několika kritérií zároveň, a ta si zpravidla navzájem konkurují:

- **Prediktivní přesnost** (*accuracy*) je podíl správně klasifikovaných vzorů:

$$Accuracy = \frac{\text{počet správných klasifikací}}{\text{počet všech testovaných případů}}.$$

- **Efektivita** se měří časem, a to jak časem potřebným pro konstrukci modelu, tak časem pro jeho použití.
- **Robustnost** vypovídá o tom, jak si metoda poradí se šumem a s chybějícími hodnotami.
- **Škálovatelnost** znamená, že metoda zůstane efektivní i pro databáze uložené na disku.
- **Interpreovatelnost** se ptá, jakou jasnost a jaký vhled model svému uživateli poskytuje.
- **Kompaktnost modelu** se posuzuje podle velikosti stromu, počtu pravidel a podobně.

5.2.2 Metody odhadu chyby

Přesnost naměřená na datech, ze kterých se model naučil, je nadhodnocená. Následující postupy se liší tím, jak velkou část dat obětují testování, a volí se hlavně podle velikosti dostupné datové sady.

Holdout, křížová validace, leave-one-out, validační množina

Při metodě **holdout** (*holdout set*) se dostupná množina D rozdělí na dvě disjunktní podmnožiny: *trénovací* D_{train} slouží k naučení modelu a *testovací* D_{test} k jeho otestování. Platí přitom **zásadní pravidlo**, že trénovací množina se nesmí použít při testování a testovací při učení, protože jen neviděná testovací množina dává *nezkreslený* odhad přesnosti. Holdout je vhodný především pro *velká* D .

k -násobná křížová validace (*k -fold cross-validation*) rozdělí data uniformně na k stejně velkých disjunktních podmnožin. Každá podmnožina se jednou použije jako testovací a zbývajících $k - 1$ tvoří trénovací data. Procedura se tedy provede k -krát a získá se k přesností $Accuracy_i$, jejichž průměr je výsledným odhadem:

$$Accuracy_{CV} = \frac{1}{k} \sum_{i=1}^k Accuracy_i.$$

Běžně se volí 10-násobná a 5-násobná křížová validace a celá metoda se používá spíše pro *menší* data.

Leave-one-out je speciální případ křížové validace, ve kterém každá testovací podmnožina obsahuje jediný vzor a na všem ostatním se trénuje. Pro m vzorů tak dostáváme m -násobnou křížovou validaci. Uplatní se u *velmi malých* dat.

Validační množina (*validation set*) vzniká tak, že se data rozdělí nikoli na dvě, ale na *tři* části: trénovací, validační a testovací. Validační množina slouží k odhadu *parametrů* učících algoritmů, protože se použijí ty hodnoty, které na ní dávají nejlepší přesnost. K odhadu parametrů lze místo ní použít i křížovou validaci.

Interval spolehlivosti pro křížovou validaci

Odhad z křížové validace je průměr k čísel, a proto se k němu dá připojit i jeho nejistota. Přibližný $N\%$ interval spolehlivosti má tvar

$$Accuracy_{CV} \pm t_{N,k-1} \cdot s_{Accuracy}, \quad s_{Accuracy} = \sqrt{\frac{1}{k(k-1)} \sum_{i=1}^k (Accuracy_i - Accuracy_{CV})^2}.$$

Hodnoty $t_{N,k-1}$ pro dvoustranné intervaly jsou:

k	hladina spolehlivosti N		
	90 %	95 %	99 %
2	2,92	4,30	9,92
10	1,81	2,23	3,17
30	1,70	2,04	2,75
∞	1,64	1,96	2,58

V praxi se často pracuje rovnou s $t = 1,96$, což je hodnota pro $k \rightarrow \infty$ a $N = 95 \%$.

5.2.3 Matice záměn a odvozené míry

Přesnost je jen *jedna* míra, doplněná vztahem $error = 1 - accuracy$, a v některých aplikacích se pro hodnocení nehodí. Při dolování z textu nás mohou zajímat pouze dokumenty určitého tématu, tedy malá část velké kolekce. Podobně při klasifikaci silně *nevyvážených* dat, jako je detekce průniků do sítě nebo finančních podvodů, jde jen o minoritní třídu. Tvoří-li průniky 1 % případů, dosáhneme 99% přesnosti tím, že *neuděláme nic*, a přitom neodhalíme jediný průnik. Zajímavá třída se v této terminologii nazývá *pozitivní* a ostatní *negativní*.

Matice záměn, precision, recall, F_1

		klasifikováno jako	
		pozitivní	negativní
skutečnost	pozitivní	TP	FN
	negativní	FP	TN

Jednotlivá pole znamenají toto: TP jsou správné klasifikace pozitivních vzorů (*true positive*), FN nesprávné klasifikace pozitivních vzorů (*false negative*), FP nesprávné klasifikace negativních vzorů (*false positive*) a TN správné klasifikace negativních vzorů.

Z matice se odvozují dvě základní míry:

$$p = precision = \frac{TP}{TP + FP}, \quad r = recall = \frac{TP}{TP + FN}.$$

Precision p je počet správně klasifikovaných pozitivních vzorů dělený počtem všech vzorů klasifikovaných jako pozitivní. *Recall* r je počet správně klasifikovaných pozitivních vzorů dělený počtem všech skutečně pozitivních vzorů v testovací množině. Podstatné je, že **obě míry hodnotí klasifikaci pouze na pozitivní třídě**.

Obě míry spojuje do jednoho čísla F_1 -**míra**, která je jejich *harmonickým průměrem*:

$$F_1 = \frac{2pr}{p + r} = \frac{2}{\frac{1}{p} + \frac{1}{r}}.$$

Harmonický průměr dvou čísel leží blíž tomu *menšímu* z nich, a proto vyjde F_1 velká jen tehdy, jsou-li velké *obě* míry.

Jak snadno může jedna míra klamat, ukazuje *ilustrace pasti*. Klasifikátor, který správně označí jediný pozitivní vzor a žádný negativní neoznačí chybně, má $p = 100 \%$, ale $r = 1 \%$. Vysoká precision sama o sobě tedy o užitečnosti modelu neříká nic.

5.2.4 Skórování, řazení a lift analýza

Skórování (*scoring*) je klasifikaci příbuzné, ale zajímá nás u něj jediná, pozitivní třída, například třída kupujících v marketingové databázi. Místo definitivního zařazení dostane každý testovací vzor *odhad pravděpodobnosti* (*probability estimate*, PE), že do pozitivní třídy patří. Klasifikační systémy se ke skórování použít dají, musí ale pravděpodobnostní odhad poskytnout: u rozhodovacích stromů se k tomu hodí hodnota confidence v listu.

Oskórované vzory se pak *seřadí* podle PE a rozdělí do n přihrádek (*bins*), například do 10. Podle počtu pozitivních vzorů v každé přihrádce se nakreslí **lift křivka** a celému postupu se říká *lift analýza*.

Příklad — lift analýza pro cílenou kampaň

Chceme rozeslat propagační materiály potenciálním zákazníkům, konkrétně nabídku hodinek. Každá zásilka stojí \$0,50 a prodej jedné hodinek přinese \$5 zisku. Otázka zní, kolik zásilek poslat a komu.

Testovací množina má 10 000 vzorů, z nichž 500 je pozitivních. Po oskórování a seřazení do 10 přihrádek po 1000 vzorech vypadá rozdělení takto:

Přihrádka	1	2	3	4	5	6	7	8	9	10
Pozitivních	210	120	60	40	22	18	12	7	6	5
% z celku	42	24	12	8	4,4	3,6	2,4	1,4	1,2	1,0
Kumulativně	42	66	78	86	90,4	94	96,4	97,8	99	100

První přihrádka obsahuje 42 % všech pozitivních případů a první tři dohromady 78 %. Náhodné rozeslání by přitom v každé přihrádce zasáhlo 10 % pozitivních, takže právě rozdíl mezi lift křivkou a touto diagonálou vyjadřuje přínos modelu. Odpověď na otázku „kolik zásilek“ pak vychází z ekonomiky: zásilkou do první přihrádky získáme $210 \cdot \$5 - 1000 \cdot \$0,50 = \$550$, kdežto do desáté $5 \cdot \$5 - 1000 \cdot \$0,50 = -\$475$.

5.2.5 ROC křivka a AUC

ROC křivka (*Receiver Operating Characteristic*)

ROC křivka vynáší *míru správně pozitivních* (TPR) proti *míře falešně pozitivních* (FPR):

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}, \quad TNR = \frac{TN}{FP + TN}.$$

TPR je podíl skutečně pozitivních případů, které jsou správně klasifikovány, takže je to v podstatě *recall* pozitivní třídy; nazývá se též *senzitivita*. *FPR* je naproti tomu podíl skutečně negativních případů klasifikovaných jako pozitivní. Pro *TNR*, což je *recall* negativní třídy, se používá název *specifita* a platí $FPR = 1 - \text{specifita}$.

Každá ROC křivka začíná v bodě (0,0), kdy je vše klasifikováno jako negativní, a končí v (1,1), kdy je vše pozitivní. **Hlavní diagonála** odpovídá náhodnému hádání, tedy predikci pozitivní třídy s fixní pravděpodobností, při níž $TPR = FPR$.

Křivka se sestavuje z klasifikace ve formě žebříčku testovacích případů. **Konstrukce ROC křivky** má čtyři kroky:

1. Získej žebříček setříděním testovacích případů *klesajícím* výstupem klasifikátoru, například aposteriorní pravděpodobností.
2. Rozděl žebříček *na každém* testovacím případě na dvě části a považuj každý případ v horní části za pozitivní a každý v dolní za negativní.
3. Pro každé takové rozdělení spočítej dvojici (FPR , TPR). Je-li horní část prázdná, dostaneme bod (0,0); je-li prázdná dolní část, bod (1,1).
4. Spoj sousední body.

AUC — plocha pod ROC křivkou

Dva klasifikátory se mohou v ROC prostoru *křížit*. V příkladu ze slidů je pro $FPR < 0,43$ lepší C_1 , zatímco pro $FPR > 0,43$ lepší C_2 , takže na otázku „který je lepší“ neexistuje jednoznačná odpověď. Celkově proto lze použít *plochu pod ROC křivkou* (*area under the curve*, AUC): platí-li $AUC(C_i) > AUC(C_j)$, říkáme, že C_i je lepší než C_j . Perfektní klasifikátor má $AUC = 1$ a náhodné hádání $AUC = 0,5$.

5.3 Vizualizační techniky

Zbývá poslední ze tří otázek: jak nalezenou znalost i samotná data ukázat. Každá technika odpovídá na jiný typ dotazu a každá má hranici, za kterou přestává být čitelná.

♦ nad rámec slidů: tabulka jako přehled; krabicový graf, dendrogram a ROC/lift křivka samy ve slidech jsou]

Technika	K čemu	Omezení / poznámka
Histogram	rozdělení jedné numerické veličiny	citlivý na volbu šířky košů
Krabicový graf (<i>box plot</i>)	medián, kvartily, odlehlé hodnoty; srovnání skupin	neukáže bimodalitu (řeší <i>violin plot</i>)
Bodový graf (<i>scatter plot</i>)	vztah dvou numerických veličin	při mnoha bodech překryv → průhlednost, hexbin
Matice bodových grafů (SPLOM)	všechny dvojice atributů najednou	použitelné do zhruba 10 atributů
Paralelní souřadnice	vysokorozměrná data — objekt je lomená čára přes svislé osy atributů	silně závisí na pořadí os; nutné škálování; křížení čar = negativní korelace
Teplotní mapa (<i>heatmap</i>)	korelační matice, matice záměn	volba barevné škály (divergentní pro korelace)
Dendrogram	výsledek hierarchického shlukování	pořadí listů není jednoznačné
Projekce (PCA, MDS, t-SNE, UMAP)	2-D pohled na vysokorozměrná data	t-SNE/UMAP zkreslují globální vzdálenosti — <i>velikosti</i> shluků a mezery mezi nimi nelze interpretovat
ROC / lift křivka	kvalita klasifikátoru	viz odd. 5.2
Mozaikový graf	kontingenční tabulka kategoriálních atributů	vizuální alternativa k χ^2
Vizualizace sítě	sociální sítě, grafy vztahů	„chlupatá koule“ u velkých grafů → agregace komunit

Vizualizace hraje v KDD tři různé role. První z nich je *explorativní analýza* ve fázi porozumění datům, kdy graf odhalí rozdělení hodnot, odlehlé i chybějící hodnoty a nečekané vztahy. Klasickým dokladem je Anscombeho kvartet: čtyři datové sady mají identické průměry, rozptyly i korelace, ale zcela odlišnou strukturu, kterou odhalí jedině obrázek. Druhou rolí je *vizualizace modelu* samotného, tedy stromu, dendrogramu, sítě nebo hranice rozhodování. Třetí je *prezentace výsledků* manažerovi, kde se sází na agregaci a jednoduchost podle zásady jeden graf = jedno sdělení.

Zásady dobré vizualizace se dají shrnout do několika bodů. Poměr datové a inkoustové složky má být co nejvyšší, jak požaduje Tufte. Graf nesmí zkreslovat, takže sloupcové grafy patří na nulovou osu a osy mají být lineární, nebo zřetelně označené jako logaritmické. Barva se volí podle typu dat: kategorie žádají kvalitativní paletu, pořadí sekvenční a odchylka od středu divergentní, a to vždy s ohledem na barvoslepost. Popisky a jednotky patří do grafu bezpodmínečně. Naopak trojrozměrné efekty a koláčové grafy s mnoha výsečemi je lepší vynechat.

5.4 Vyhodnocení znalostí z pohledu uživatele

Pojem „vyhodnocování získaných znalostí“ se používá ve třech různých rovinách a ty je nutné od sebe odlišovat:

- *Statistické vyhodnocení modelu* pracuje s přesností, maticí záměn, ROC a AUC a s odhadem chyby křížovou validací (odd. 5.2).
- *Vyhodnocení zajímavosti jednotlivých vzorů* se opírá o support, confidence, lift, χ^2 , t-weight a d-weight (odd. 3.1 a kap. 4).
- *Vyhodnocení z pohledu uživatele* se ptá, zda je nalezená znalost srozumitelná, platná, užitečná a nová.

První dvě roviny jsou objektivní a měřitelné, kdežto třetí je nutně subjektivní: tentýž vzor může být pro jednoho uživatele objevem a pro druhého notorií. Část výsledků bývá nezajímavá nebo z hlediska uživatele zřejmá, část se dá použít přímo a část je nutné prezentovat srozumitelněji. Užitečné bývá shrnout výsledky do *analytické zprávy*, přičemž výstupem celého procesu může být i provedení rozumné akce.

5.5 Shrnutí ke zkoušce

- **Formy reprezentace znalostí** sahají od IF-THEN pravidel (nejsrozumitelnější, lokální) přes rozhodovací strom (globální, čitelný do hloubky 4–5), rozhodovací tabulku, matematickou funkci (kompaktní, ale neinterpretovatelná), prototypy a centroidy a pravděpodobnostní model až po grafy a sítě a instance (k -NN, CBR). U každé formy je třeba umět říct, z jaké metody vzniká a čím se platí za její srozumitelnost.
- **Kritéria hodnocení metod** jsou prediktivní přesnost, efektivita (čas konstrukce i čas použití), robustnost, škálovatelnost, interpretovatelnost a kompaktnost modelu.
- **Odhad chyby** má čtyři podoby. *Holdout* se hodí na velká data a trénovací s testovací množinou se u něj nesmí míchat. *k-násobná křížová validace* se používá na menší data, běžné je k rovné 5 nebo 10, přesnost vyjde jako průměr k přesností a doplní se interval spolehlivosti s $t_{N,k-1}$. *Leave-one-out* je m -násobná CV pro velmi malá data. *Validační množina* je třetí část dat, na které se ladí *parametry* algoritmu.
- **Matice záměn** obsahuje TP , FN , FP a TN , z nichž se počítá $p = \frac{TP}{TP+FP}$, $r = \frac{TP}{TP+FN}$ a $F_1 = \frac{2pr}{p+r}$, tedy harmonický průměr, který leží blíž menšímu z obou čísel. Je nutné umět vysvětlit, proč accuracy nestačí u nevyvážených dat: při 1 % průniků dá strategie „nedělat nic“ $\Rightarrow$ 99 % přesnosti.
- **ROC** vynáší $TPR = \frac{TP}{TP+FN}$, což je senzitivita čili recall pozitivní třídy, proti $FPR = \frac{FP}{FP+TN}$; TNR je specifita a rovná se $1 - FPR$. Křivka vede z (0,0) do (1,1) a diagonála odpovídá náhodnému hádání. Je třeba umět popsat konstrukci ze žebříčku, tedy dělení na každém případě. **AUC** má hodnotu 1 pro perfektní a 0,5 pro náhodný klasifikátor a potřebujeme ji proto, že se ROC křivky dvou klasifikátorů mohou *křížit*.
- **Skórování a lift**: místo třídy se každému vzoru přiřadí odhad pravděpodobnosti (u stromu confidence v listu), vzory se seřadí a rozdělí do příhrádek a lift křivka se poměruje s diagonálou náhodného výběru. U zkoušky se k tomu očekává ekonomická úvaha o tom, kolik zásilek se vyplatí poslat.
- **Tři roviny „vyhodnocování“** jsou statistická kvalita modelu, zajímavost vzorů a vyhodnocení uživatelem, přičemž první dvě jsou objektivní a třetí subjektivní. **Čtyři kritéria zajímavosti znalosti** zní: srozumitelnost, platnost, užitečnost a novost.
- **Vizualizace** se vybírá podle úlohy: histogram, box plot, scatter, SPLOM, paralelní souřadnice, heatmapa, dendrogram, projekce, ROC a lift křivka. V KDD má tři role: exploraci (dokládá ji Anscombeho kvartet), vizualizaci modelu a prezentaci. Zásady jsou Tufteho poměr datové a inkoustové složky, nezkreslovat osami a volit barvu podle typu dat. Znat je nutné i omezení: t-SNE a UMAP zkreslují globální vzdálenosti.

6 Modely pro analýzu sociálních sítí, centralita a detekce komunit

Věta okruhu jmenuje tři věci: *modely* pro analýzu sociálních sítí, *míry centrality* a *detekci komunit*. Tomu odpovídá i členění kapitoly. Nejprve přijde na řadu reprezentace sítí a míry, kterými se popisuje jednotlivý aktér (centralita) a síť jako celek (koheze). Na to navazují modely: jaké vlastnosti mají reálné sítě, jak se v nich šíří vliv, co znamená homofilie, segregace a strukturní balance a jak se analyzují vazby (HITS, PageRank). Poslední vrstvu tvoří detekce komunit a predikce vazeb.

6.1 Vymezení oboru a motivace

Analýza sociálních sítí

Analýza sociálních sítí (*social network analysis*, SNA) je interdisciplinární obor, který vyrostl ze tří oblastí:

- *sociální vědy* dodávají otázky a problémy, tedy snahu vysvětlit sociální chování analýzou společností;
- *analýza sítí* umožňuje formulovat a řešit problémy, které mají síťovou strukturu;
- *teorie grafů* poskytuje matematický aparát pro reprezentaci a analýzu grafů.

Zásadní je posun perspektivy: SNA se soustředí na **vztahy** mezi sociálními entitami (jednotlivci, skupinami, institucemi) *spíše než na entity samotné* a jejich atributy. Studium společnosti ze síťové perspektivy chápe jednotlivce jako *zasazené* do sítě vztahů a vysvětlení sociálního chování hledá ve struktuře těchto sítí, nikoli v jednotlivcích samotných.

Praktickou motivací je **analýza vazeb** (*link analysis*), jejímž cílem je najít vztahy mezi daty a tyto vazby vizualizovat. Uplatní se v telekomunikacích, v kriminalistice a právu, kde jsou skupiny zločinců vzájemně propojené a analýza vztahů může pomoci je odhalit, a v marketingu, kde jde o vztahy mezi zákazníky.

Šest stupňů odloučení (Milgram, 1967) je experiment o fenoménu malého světa. Náhodně vybraní lidé z Nebrasky měli poslat dopis burzovnímu makléři v Bostonu, a to jen přes prostředníky: poslat jej směli výhradně někomu, koho znali osobně, tedy na křestní jméno. Mezi dopisy, které cíl našly, byl průměrný počet vazeb *šest*. Do cíle jich ovšem mnoho nedorazilo. Experiment předznamenal pozdější objevy, zejména huby a tendenci postupovat podle geografie či profese. Podobně vypovídají *Erdősova čísla*: Paul Erdős napsal přes 1500 článků s 507 spoluautory, průměrné Erdősovo číslo mezi matematiky je 4,65 a maximum 13.

Druhou motivací jsou **bezškálové sítě** (*scale-free*, SF), v nichž mají některé uzly extrémně mnoho spojení (*huby*), zatímco většina uzlů jen několik. Důsledkem je dvojaká povaha takové sítě: *odolnost proti náhodné poruše* a současně *zranitelnost koordinovaným útokem*. Selhat může až 80 % náhodně vybraných uzlů, aniž se síť fragmentuje, ale eliminace 5–15 % *hubů* způsobí rozpad celého systému.

Aplikace této úvahy sahají daleko. V ochraně proti šíření počítačových virů internetem z ní plyne, že vymýcení internetových virů je v SF architektuře v podstatě nemožné. V medicíně vede k vakcinačním kampaním cíleným na huby a k novým lékům mířícím na klíčové molekuly. V byznysu vysvětluje kaskádové finanční krachy a podpírá marketing, tedy šíření „nákazy“ a influencer marketing.

Bezškálovou strukturu mají *sociální sítě* (vědecká spolupráce prostřednictvím spoluautorství článků, Hollywood a herci ve stejném filmu), *biologické sítě* (metabolismus buňky, proteinové regulační sítě) i *sociotechnické sítě* (internet tvořený routery a spoji, WWW tvořené stránkami a URL).

6.2 Reprezentace sítí

Než se dá cokoli z toho měřit, musí být síť zapsaná, a to způsobem, s nímž umí pracovat algoritmus.

Základní reprezentace

Síť je graf $G = (V, E)$, kde *uzly* (vrcholy) jsou aktéři a *hrany* (vazby) vztahy mezi nimi. Zapsat ji lze třemi ekvivalentními způsoby:

- **obrázkem grafu**;
- **seznamem hran** (*edge list*), tedy dvojicemi uzlů, u ohodnocených grafů doplněnými o sloupec vah;
- **maticí sousednosti** (*adjacency matrix*) $A = (a_{ij})$, kde $a_{ij} = 1$, je-li uzel i spojen s uzlem j , a 0 jinak; u vážených grafů obsahuje matice váhy.

U **orientovaných** grafů (kdo koho kontaktuje) matice sousednosti *nemusí* být symetrická, u **neorientovaných** (kdo koho zná) symetrická je. Tentýž seznam hran má proto v obou případech jinou interpretaci.

Váhy hran nesou informaci o tom, jak silný vztah hrana zachycuje, a jejich význam závisí na tom, co se v dané úloze měří. Vyjadřovat mohou frekvenci interakce v období pozorování nebo počet vyměněných předmětů. Jinde jde o individuální percepci silné vazby, o náklady komunikace nebo výměny (například o vzdálenost), případně o kombinaci předchozích. Samy hrany pak reprezentují interakce, toky informací či zboží, podobnosti a afiliace, nebo sociální vztahy.

Ego síť a „celé“ síť. *Ego síť* je osobní síť jednoho aktéra (*ego*) a jeho sousedů (*alterů*). Samotné ego přitom pro další analýzu *není* potřebné, protože všichni alteri jsou s egem spojeni už z definice. „Celá“ síť oproti tomu obsahuje i izolované uzly (*isolates*). Platí ovšem, že **žádná studovaná síť není v praxi „celá“**: obvykle jde jen o částečný obraz skutečné sítě, čemuž se říká *problém specifikace hranic* (*boundary specification problem*).

6.3 Síla vazeb

Sociální média propojují uživatele extrémně snadno, takže uživatel může mít online tisíce „přátel“. Tyto vazby ale nejsou stejně silné a hrají různou roli při formování komunit a difuzi informací. *Silné vazby* jsou blízcí přátelé, *slabé vazby* známí. Právě na ty druhé upozornil Granovetter (1973) tezí o **síle slabých vazeb**: občasná setkání se vzdálenými známými mohou poskytnout důležitou informaci o nových příležitostech, například při hledání práce.

Homofilie, tranzitivita, přemostění

Homofilie (*homophily*) je tendence vztahovat se k lidem s podobnými charakteristikami (status, přesvědčení, ...). Vede k tvorbě homogenních skupin (klastřů), v nichž je snazší vytvářet vztahy. *Extrémní* homogenizace ale může působit proti inovaci a generování idejí, takže *heterofilie* je v některých kontextech žádoucí. Homofilní vazby mohou být silné i slabé.

Tranzitivita (*transitivity*) říká, že „existuje-li vazba mezi A a B a mezi B a C , budou v tranzitivní síti spojeni i A a C .“ Silné vazby jsou tranzitivní častěji než slabé, takže tranzitivita existenci silných vazeb *indikuje*, není to však ani nutná, ani postačující podmínka. Tranzitivita a homofilie společně vedou ke vzniku *klik*, tedy plně propojených klastřů.

Přemostění (*bridging*) stojí na pojmu *mostu* (*bridge*): most je uzel či hrana spojující různé skupiny. Obvykle jde o slabé vazby, ale ne každá slabá vazba je most. Mosty usnadňují mezigrupovou komunikaci, zvyšují sociální kohezi a podporují inovaci.

Určování síly vazby z topologie sítě. Mosty spojující dvě různé komunity jsou slabé vazby a hrana je *most* tehdy, jestliže její odstranění rozpojí její koncové uzly. V reálných sítích ale mosty nejsou časté, a proto se definice *uvolní* na **zkratkový most** (*shortcut bridge*): kontroluje se, zda se po odstranění hrany *zvětší* vzdálenost jejích koncových uzlů, konkrétně na více než 2. Čím větší tato vzdálenost je, tím *slabší* vazba. Například $d(2, 5) = 4$ po odstranění hrany $(2, 5)$, zatímco $d(5, 6) = 2$ po odstranění $(5, 6) \Rightarrow (5, 6)$ je silnější vazba než $(2, 5)$.

Překryv sousedství (*neighborhood overlap*)

Pro dva uzly i, j spojené vazbou platí, že čím větší je jejich překryv, tím *silnější* je vazba mezi nimi:

$$O(i, j) = \frac{\text{počet společných přátel } i \text{ a } j}{\text{počet přátel, kteří jsou sousedy alespoň } i \text{ nebo } j} = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2},$$

kde -2 ve jmenovateli vylučuje i a j samotné.

Příklady: $O(2, 5) = 0$; $O(5, 6) = \frac{|\{4\}|}{|\{2, 4, 5, 6, 7, 10\}| - 2} = \frac{1}{4}$; pro graf $G = (\{1, 2\}, \{(1, 2)\})$ je $O(1, 2) = 0$.

Určování síly vazby z profilů a interakcí. Na Twitteru, kde lze sledovat jiné bez jejich potvrzení, není skutečná síť přátelství určena sítí follower–followee, nýbrž *frekvencí*, s jakou spolu dva uživatelé mluví. A právě tato skutečná síť má silnější vliv na používání Twitteru. Na Facebooku lze sílu vazeb predikovat z různých informací: z příspěvků iniciovaných přítelem, ze zpráv vyměněných na zdi nebo z počtu společných přátel. Numerickou sílu vazby se lze i učit, například metodou maximální věrohodnosti: podobnost profilů určuje sílu vazby a síla vazby určuje interakci uživatelů, takže se maximalizuje věrohodnost na základě pozorovaných profilů a interakcí.

6.4 Klíčoví hráči: míry centrality

Následující míry se všechny ptají, kdo je v síti klíčový. Každá z nich ale měří jiný druh důležitosti, a proto se jejich pořadí uzlů zpravidla neshodují.

Stupňová centralita (*degree centrality*)

Vstupní stupeň uzlu je počet vazeb vedoucích do uzlu a výstupní stupeň počet vazeb vedoucích z uzlu. V neorientovaných grafech jsou oba stupně shodné.

Míra zachycuje *propojenost*, *vliv* nebo *popularitu* uzlu a pomáhá zjistit, které uzly jsou centrální z hlediska šíření informace a ovlivňování ostatních ve svém „sousedství“.

Stupňová centralita vystačí s bezprostředním okolím uzlu. Následující dvě míry, mezilehlá a blízkostní, už ale potřebují vědět, jak daleko od sebe uzly leží, a tedy pojem cesty.

Cesty a nejkratší cesty

Cesta mezi dvěma uzly je libovolná posloupnost neopakujících se uzlů, která je spojuje. *Nejkratší cesta* spojuje dva uzly nejmenším počtem hran a tento počet se nazývá *vzdálenost* uzlů. Kratší cesty jsou preferované pro rychlou komunikaci či výměnu, i když ne vždy; příkladem opačné situace je šíření nemoci.

Algoritmy nejkratších cest. Volba algoritmu se řídí dvěma otázkami: hledáme cesty z jednoho zdroje, nebo mezi všemi páry uzlů, a připouštíme záporné váhy? Pro *jeden zdroj* a obecné grafy, tedy orientované a se zápornými vahami, slouží Bellmanův–Fordův algoritmus se složitostí $O(|V||E|)$. Nemá-li graf záporné váhy, stačí Dijkstrův algoritmus, $O(|V|^2)$. Hledá-li se cesta jen mezi dvěma uzly, lze algoritmus zastavit hned po jejím nalezení.

Pro *všechny páry* uzlů se při povolených záporných vahách používá Floydův–Warshallův algoritmus, $O(|V|^3)$. U řídkých grafů se zápornými vahami je výhodnější Johnsonův algoritmus, $O(|E||V| + |V|^2 \log_2 |V|)$, a bez záporných vah opět Dijkstra, $O(|V|^3)$.

Mezilehlá centralita (*betweenness centrality*)

Mezilehlá centralita je počet nejkratších cest procházejících uzlem, dělený všemi nejkratšími cestami v síti. Pro uzel v se pro každou dvojici s, t spočítá počet nejkratších cest mezi s a t procházejících v , ten se vydělí všemi nejkratšími cestami mezi s a t a výsledné hodnoty se sečtou přes všechny dvojice. Někdy se výsledek ještě normalizuje tak, aby nejvyšší hodnota byla 1.

Míra indikuje uzly, které leží na *komunikačních cestách* mezi jinými uzly, a pomáhá určit místa, kde by se síť mohla *rozpadnout*.

Blízkostní centralita (*closeness centrality*)

Blízkostní centralita je míra *dosahu*, tedy rychlosti, s jakou se informace z daného uzlu dostane k ostatním. Spočítá se *průměrná* délka všech nejkratších cest z uzlu ke všem ostatním uzlům sítě, jinak řečeno kolik „skoků“ je v průměru potřeba, a vezme se *převrácená hodnota*. Právě kvůli tomuto převrácení znamenají vyšší hodnoty vyšší blízkost a jsou „lepší“. Míra se hodí tam, kde jde především o rychlost šíření.

Vlastní centralita (*eigenvector centrality*)

Vlastní centralita uzlu je proporcionální součtu vlastních centralit všech uzlů přímo spojených s uvažovaným uzlem: uzel s vysokou vlastní centralitou je spojen s jinými uzly s vysokou vlastní centralitou. Metrika vychází z myšlenky, že moc a status aktéra jsou *rekurzivně* definovány mocí a statusem jeho sousedů, a měří tedy, jak dobře je aktér spojen s jinými *dobře propojenými* aktéry.

Formálně: pro $G = (V, E)$ s maticí sousednosti $A = (a_{i,j})$ platí

$$x_i = \frac{1}{\lambda} \sum_{j \in M(i)} x_j = \frac{1}{\lambda} \sum_{j \in V} a_{i,j} x_j \iff Ax = \lambda x,$$

kde $M(i)$ je množina sousedů i . Vlastních čísel λ , pro něž nenulové řešení existuje, je mnoho. *Požadavek nezápornosti* všech složek vlastního vektoru však znamená, že žádanou míru centrality dává *jen největší* vlastní číslo. Vlastní vektor je navíc určen až na společný násobek, takže je nutná normalizace, například součtem přes všechny uzly $= 1$ nebo $= |V|$.

Míra blízkce souvisí s Google PageRank, kde odkazy ze silně odkazovaných stránek váží více (odd. 6.14).

Rozdíl mezi čtyřmi mírami je nejlépe vidět na otázce, kterou každá z nich klade, a na tom, co znamená v konkrétní síti:

Míra	Interpretace v sociálních sítích
Stupňová	Kolik lidí může tato osoba dosáhnout <i>přímo</i> ? — <i>Síť hudebních spoluprací</i> : s kolika lidmi spolupracoval?
Mezilehlá	Jak pravděpodobně leží tato osoba na nejpřímější cestě v síti? — <i>Síť špiónů</i> : přes koho nejspíš protéká většina informací?
Blízkostní	Jak rychle může tato osoba dosáhnout všech v síti? — <i>Síť sexuálních vztahů</i> : jak rychle se do zbytku sítě rozšíří přenosná nemoc?
Vlastní	Jak dobře je tato osoba spojena s jinými dobře propojenými lidmi? — <i>Citační síť</i> : který autor je nejvíce citován jinými dobře citovanými autory?

Množiny klíčových hráčů. Jednotlivé míry samy o sobě nestačí. V příkladu ze slidů je uzel 10 nejcentrálnější podle stupňové centrality, ale uzly 3 a 5 *společně* dosáhnou více uzlů. Navíc je vazba mezi 3 a 5 *kritická*: při jejím přerušení se síť rozpadne na dvě izolované podsítě. Za jinak stejných podmínek jsou tedy uzly 3 a 5 *společně* pro síť důležitější než uzel 10 samotný. *Uvažovat množiny klíčových hráčů proto pomáhá.*

Strukturní ekvivalence

Vyjadřuje *podobnost* mezi aktéry v sociální síti a je založena na sousedech, které sdílejí, ekvivalentně na počtu identických vazeb. Dva aktéři jsou *strukturně ekvivalentní*, lze-li jejich pozice vzájemně vyměnit *bez změny celkové struktury* sítě. Přibližnou strukturní ekvivalenci lze použít pro hierarchické shlukování sociálních sítí.

6.5 Koheze: míry na úrovni sítě

Zatímco centralita hodnotí jednotlivý uzel, následující míry charakterizují síť jako celek.

Reciprocita, hustota, klastrovací koeficient

Reciprocita (*reciprocity*) měří vzájemnost a reciproční výměnu v síti a souvisí se sociální kohezí. Je to podíl počtu *reciprokých* vztahů, tedy takových, kde hrana vede v obou směrech, na celkovém počtu vztahů v síti, přičemž dva uzly jsou „ve vztahu“, existuje-li mezi nimi alespoň jedna hrana. **Má smysl jen v orientovaných grafech.**

Hustota (*density*) je podíl počtu hran v síti na celkovém počtu *možných* hran:

$$\text{density} = \frac{|E|}{|V|(|V| - 1)/2} \quad (\text{neorientovaný graf}), \quad \frac{|E|}{|V|(|V| - 1)} \quad (\text{orientovaný}).$$

Je to běžná míra toho, jak dobře je síť propojená, tedy jak „těsně upletená“ je. Perfektně propojená síť se nazývá *klika* a má hustotu 1. Orientované grafy mají poloviční hustotu než jejich neorientované protějšky, protože možných hran je v nich dvakrát více.

Klastrovací koeficient (*clustering coefficient*) uzlu i měří hustotu jeho *sousedství*, tedy sítě tvořené pouze uzly přímo spojenými s i . Pro $G = (V, E)$ je sousedství $N_i = \{v_j : e_{ij} \in E \vee e_{ji} \in E\}$, $k_i = |N_i|$, a

$$C_i = \frac{2|\{e_{jk} : v_j, v_k \in N_i, e_{jk} \in E\}|}{k_i(k_i - 1)} \quad (\text{neorientovaný}), \quad C_i = \frac{|\{e_{jk}\}|}{k_i(k_i - 1)} \quad (\text{orientovaný}).$$

Klastrovací koeficient sítě C je průměr koeficientů všech uzlů, tedy $C = \frac{1}{|V|} \sum_{i=1}^{|V|} C_i$. Indikuje přítomnost (sub)komunit a shlukovací algoritmy se snaží maximalizovat počet hran padnoucích do téhož klastru.

Průměr, průměrná vzdálenost, excentricita, malé světy

Průměr sítě (*diameter*) je nejdelší nejkratší cesta, tedy vzdálenost, mezi libovolnými dvěma uzly. Je to užitečná míra *dosahu* sítě, protože udává, jak dlouho nejdéle trvá dosáhnout libovolného uzlu. Řidší síť mají obecně větší průměr.

Průměrná vzdálenost je průměr všech nejkratších cest v síti a udává, jak daleko od sebe budou dva uzly v průměru.

Excentricita uzlu je největší geodetická vzdálenost mezi daným uzlem a libovolným jiným uzlem grafu.

Malý svět (*small world*) je síť, která vypadá téměř náhodně, ale má *významně vysoký klastrovací koeficient* a *relativně krátkou průměrnou délku cesty*: uzly se lokálně klastrují a zároveň jsou dosažitelné v několika krocích. V sociálních sítích je to velmi častý jev, a to kvůli *tranzitivitě* silných vazeb a *schopnosti slabých vazeb dosáhnout přes klastry*. Síť malého světa mají mnoho klastrů, ale i mnoho mostů mezi nimi, které zkracují průměrnou vzdálenost.

6.6 Model reálných komplexních sítí a jejich vlastnosti

Popsané míry se nyní spojí do obrazu toho, čím se reálné sítě liší od grafů, které bychom vygenerovali náhodně.

Šest charakteristických vlastností reálných komplexních sítí

Reálné komplexní sítě jsou *nenáhodné* a *neregulární* grafy s unikátními rysy; patří mezi ně sociální sítě, informační a znalostní sítě, technologické sítě a biologické sítě. Opakovaně se u nich pozoruje šest vlastností:

1. **Efekt malého světa.** Na existenci malých světů v reálných sociálních sítích poukázal jako první Stanley Milgram experimentem z 60. let, jehož se zúčastnilo asi 300 lidí a v němž byla mediánová délka úspěšných cest 6. Vychází z konceptu „šesti stupňů odloučení“: každý je od každého v průměru šest kroků. Důsledky se týkají rychlosti šíření informací i nemocí.
2. **Tranzitivita / klastrování.** V reálných, a zejména v sociálních sítích je vysoká pravděpo-

dobnost, že uvnitř větších sítí najdeme *úplné podsítě*, v nichž každý zná každého.

3. **Mocninné rozdělení stupňů.** *Rozdělení stupňů* je rozdělení pravděpodobnosti stupňů uzlů přes celou síť, tedy histogram podílu uzlů se stupněm k . Reálné sítě mají rozdělení stupňů zcela odlišné od náhodných grafů.
4. **Odolnost sítě** (*resilience*) popisuje, jak se změní propojenost sítě při odstranění jednoho či více uzlů. Většina sítí je *robustní* proti náhodnému odstranění, ale podstatně méně už proti *cílenému* odstranění uzlů s nejvyšším stupněm. Odstraní-li se „vrátní“ (*gatekeepers*), výrazně se změní komunikační schopnost sítě, protože se některé uzly odpojí. Odstranění jediného uzlu obvykle strukturu neohrozí, a proto je vhodnější testovat odolnost odstraněním určitého procenta uzlů.
5. **Vzorce míchání** (*mixing patterns*). Koexistují-li v síti různé typy uzlů, bývá běžná jistá *selektivita* ve tvorbě spojení. Selektivnímu propojování se říká *asortativní míchání* (*assortative mixing*) čili homofilie a klasickým příkladem je míchání podle rasy. Reálné sítě vykazují vyšší tendenci k asortativnímu míchání.
6. **Komunitní struktura.** Většina reálných sociálních sítí vykazuje komunitní strukturu, která vzniká jako důsledek *globální i lokální heterogenity* rozložení hran: v některých oblastech grafu nacházíme vysoké koncentrace hran (*komunity*) a mezi těmito oblastmi koncentrace nízké.

Bezškálové sítě a preferenční připojování

Bezškálové sítě (Barabási, Albert, 1999) jsou sítě, jejichž rozdělení stupňů následuje *mocninný zákon*; patří sem WWW, citační sítě, biologické sítě a některé sociální sítě. Mocninná rozdělení vznikají typicky tehdy, *závisí-li množství, které něčeho dostanete, na množství, které už máte*. Topologicky se to projeví jako síť s několika silně propojenými uzly (*huby*) a velkým počtem uzlů s nízkým stupněm.

Tentýž mechanismus dostal postupně několik jmen. Nejprve se mu říkalo *rich-get-richer* / *poor-get-poorer* strategie, ekvivalentně *Matoušův efekt*; Price jej nazval *kumulativní výhoda* a Barabási s Albertem **preferenční připojování** (*preferential attachment*). Vysvětlení je prosté: nové uzly vstupující do sítě se připojují k dobře propojeným uzlům, které bývají spojeny s centrálními a prestižními pozicemi (více statusu, popularity, znalostí, peněz).

Dva hlavní mechanismy vzniku SF sítě jsou tedy **růst** a **preferenční připojování**. Výsledná síť má *dlouhý chvost* rozdělení stupňů a *strukturu malého světa*. Tranzitivita a vlastnosti silných a slabých vazeb jsou běžné a mohou ke struktuře malého světa vést, k jejímu vysvětlení ale *nutné* nejsou.

Důvody preferenčního připojování se dělí do tří skupin. *Popularita* („bohatí bohatnou“) znamená, že chceme být spojeni s populárními lidmi, idejemi a věcmi, což jejich popularitu dále zvyšuje, a to bez ohledu na objektivní měřitelné charakteristiky. *Kvalita* („dobří se zlepšují“) předpokládá, že hodnotíme podle objektivních kritérií kvality, takže kvalitnější uzly přirozeně přitáhnou pozornost rychleji. *Smíšený model* shrnuje obojí do věty „nelze předpovědět, kdo se stane hvězdou, i když kvalita hraje roli“: mezi uzly s podobnými atributy se „hvězdami“ stanou ty, které první dosáhnou kritického množství (*halo efekt*).

Struktura jádro–periferie a centralizace

Centralizace (*centralization*) měří, do jaké míry je síť centralizovaná či decentralizovaná. Vychází z *rozdílů ve stupních* mezi uzly a obvykle se normalizuje do $[0, 1]$. Síť, která silně závisí na 1–2 vysoce propojených uzlech, například jako důsledek preferenčního připojování, vykazuje větší rozdíly ve stupňové centralitě. Centralizované struktury mohou být lepší v některých úlohách vyžadujících koordinaci, třeba při týmovém řešení problémů, ale jsou *náchylnější k selhání*, odpojí-li se klíčoví hráči.

Mnoho velkých skupin a komunit má *jádro* hustě propojených uživatelů, které je kritické pro propojení mnohem větší *periferie*. Jádra lze identifikovat vizuálně, nebo zkoumáním polohy uzlů s vysokým stupněm a jejich společných rozdělení stupňů; k rozlišení jádra a periferních elementů

slouží i *bow-tie analýza*, použitá pro analýzu struktury webu.

6.7 Modelování vlivu

Zbývá popsat, jak se síť něco šíří. Modely vlivu se v detailech liší, sdílejí ale několik předpokladů.

Společné vlastnosti metod modelování vlivu

- Sociální síť je reprezentována *orientovaným* grafem a každý aktér je jeden uzel.
- Každý uzel začíná buď jako *aktivní*, nebo jako *neaktivní*.
- Aktivovaný uzel aktivuje své sousedy.
- **Jednou aktivovaný uzel už nelze deaktivovat.** Šíření je tedy nevratné a množina aktivních uzlů může jen růst.

Model lineárního prahu (LTM)

Aktér (uzel) podnikne akci, pokud počet jeho přátel, kteří akci podnikli, překročí jistý *prah*.

- Každý uzel v si zvolí prah θ_v *náhodně* z uniformního rozdělení na $[0, 1]$.
- V každém diskrétním kroku zůstávají aktivní všechny uzly, které byly aktivní v kroku předchozím.
- Nově se aktivují uzly splňující

$$\sum_{u \in N_v^{\text{aktivní}}} w_{u,v} \geq \theta_v \sum_{u \in N_v} w_{u,v}.$$

LTM se soustředí na *příjemce* vlivu.

Model nezávislých kaskád (ICM)

Soustředí se naopak na *vysílající* uzly.

- Uzel v aktivovaný v kroku t má *jedinou* příležitost aktivovat každého ze svých sousedů, a to náhodně.
- Pro sousední uzel u úspěje aktivace s pravděpodobností $p_{v,u}$ (např. $p = 0,5$).
- Úspěje-li, u se stane aktivním v kroku $t + 1$.
- V dalších kolech se už v o aktivaci u *nepokouší*.
- Difuze začíná počáteční aktivovanou množinou uzlů a pokračuje, dokud není možná žádná další aktivace.

Maximalizace vlivu

Úloha: pro danou síť a parametr k vybrat k uzlů do aktivační množiny B tak, aby se maximalizoval vliv ve smyslu počtu aktivních uzlů na konci. Označme $\sigma(B)$ očekávaný počet uzlů ovlivněných množinou B ; pak hledáme

$$\max_{|B| \leq k} \sigma(B).$$

Úloha je **NP-těžká**, lze však dokázat, že *hladový* přístup dává řešení na úrovni **63 %** optima.

Hladový přístup začne s $B = \emptyset$, vyhodnotí $\sigma(\{v\})$ pro každý uzel a vybere uzel s maximálním σ jako první prvek B . Poté opakovaně vybírá uzel, který $\sigma(B)$ zvýší nejvíce, tj. $\arg \max_{v \in V \setminus B} \sigma(B \cup \{v\})$.

6.8 Vliv versus korelace

Atributy a chování uživatelů bývají po síti rozprostřeny nenáhodně: často korelují s tím, kdo je s kým spojen. Uvažujme u každého uzlu binární atribut, například zda je ten člověk kuřák. Koreluje-li

atribut se sítí, budou aktéři sdílet stejné hodnoty atributu pozitivně korelovaně se sociálními spoji, tedy kuřáci budou interagovat spíše s kuřáky.

Zbývá otázka, jak takovou korelaci z dat vůbec poznat. Postup je přímočarý: spočítá se, kolik hran mezi odlišnými hodnotami atributu by vzniklo pouhou náhodou, a toto očekávání se porovná se skutečným grafem.

Test korelace

Jestliže podíl hran spojujících uzly s *různými* hodnotami atributu je významně *nižší* než očekávaná pravděpodobnost, jde o důkaz korelace.

Příklad: předpokládejme nejprve, že spoje jsou na kuřáctví nezávislé. Podíl kuřáků v síti je $p = 4/9$, podíl nekuřáků $1 - p = 5/9$. Jedna hrana pak spojuje dva kuřáky s pravděpodobností $p \cdot p$, dva nekuřáky s pravděpodobností $(1 - p)(1 - p)$ a kuřáka s nekuřákem s pravděpodobností $2p(1 - p) = 2 \cdot \frac{4}{9} \cdot \frac{5}{9} = 49\%$. Ve skutečném grafu je ovšem tento podíl jen $2/14 = 14\% < 49\%$, takže síť **vykazuje určitý stupeň korelace** vzhledem ke kuřáctví.

Nalezená korelace ale sama o sobě neříká nic o tom, proč vznikla. Vysvětlení jsou tři a rozdíl mezi nimi je podstatný.

Tři sociální procesy vysvětlující korelaci

- **Homofilie** je tendence propojovat se s těmi, kdo s námi sdílejí určitou podobnost; lidově řečeno „vrána k vráně sedá“.
- **Zmatení** (*confounding*) připomíná, že korelaci mohou vytvořit i *vnější* vlivy prostředí: „jednotlivci žijící ve stejném městě se spíše stanou přáteli než náhodní jednotlivci“.
- **Vliv** (*influence*) je naopak proces, který korelaci chování mezi sousedními aktéry teprve způsobuje: „přejde-li většina mých přátel k jinému mobilnímu operátorovi, mohu být jimi ovlivněn a přejít také“.

Proč vliv rozlišovat od homofilie a zmatení? Ze dvou důvodů. Při *analýze* nám rozlišení dovolí předpovědět dynamiku systému, tedy zda se nová norma chování, technologie nebo idea může šířit jako epidemie. Při *návruhu* umožňuje zkonstruovat systém, který konkrétní chování vyvolá; sem patří vakcinační strategie (náhodné, cílené na demografickou skupinu, náhodní známí) i virální marketingové kampaně.

Vliv se dá od pouhé korelace oddělit i kvantitativně, máme-li k dispozici časové značky jednotlivých aktivací.

Logistický model vlivu a shuffle test

Ve studiích modelování vlivu se vliv určuje *časovými značkami*. Pro hranu (u, v) může u ovlivnit v v diskretních krocích $t = 0, 1, 2, \dots$, přičemž v každém kroku se každý dosud neaktivní uzel stane aktivním s pravděpodobností $p(a)$, kde a je počet aktivních kontaktů. Pravděpodobnost aktivace se modeluje jako *logistická* funkce počtu aktivních přátel:

$$p(a) = \frac{e^{\alpha \ln(a+1) + \beta}}{1 + e^{\alpha \ln(a+1) + \beta}}, \quad \text{ekvivalentně} \quad \ln \frac{p(a)}{1 - p(a)} = \alpha \ln(a + 1) + \beta,$$

kde a je počet aktivních přátel, α *koefficient sociální korelace*, β konstanta vysvětlující vrozenou tendenci k aktivaci a $\ln(a + 1)$ vysvětlující proměnná.

Věrohodnost aktivace. Nechtě v čase t se $y_{a,t}$ uživatelů s a aktivními přáteli stane aktivními a $n_{a,t}$ uživatelů s a aktivními přáteli zůstane neaktivních. Věrohodnost v čase t je pak $\prod_a p(a)^{y_{a,t}} (1 - p(a))^{n_{a,t}}$ a pro celou posloupnost kroků $\prod_t \prod_a p(a)^{y_{a,t}} (1 - p(a))^{n_{a,t}}$. Z logu aktivit uživatelů se tedy dají spočítat hodnoty α a β , které tuto věrohodnost maximalizují, a právě α **měří korelaci mezi uzly**.

Shuffle test (test na vliv). Klíčová myšlenka testu je následující: *nehraje-li vliv roli, musí být časování aktivace nezávislé na časování ostatních aktérů*. Jinými slovy, i když časové značky

aktivit náhodně zpřeházíme, měli bychom dostat podobnou sociální korelaci jako předtím. Samotný *test* z toho plyne bezprostředně: zpřeházej časové značky aktivit uživatelů, a je-li nový odhad sociální korelace *významně odlišný* od odhadu z původního logu, jde o důkaz **vlivu**.

Postup má pět kroků. (1) Pro všechna a spočítej $y_{a,t}$ a $n_{a,t}$. (2) Spočítej α, β maximalizující věrohodnost. (3) Vyber náhodnou permutaci π časů aktivace a nastav čas aktivace uzlu v_i na $t'_i = t_{\pi(i)}$; z nových časů spočítej $y'_{a,t}$, $n'_{a,t}$. (4) Spočítej α', β' maximalizující novou věrohodnost. (5) Jsou-li α a α' blízko sebe, model *nevykazuje* sociální vliv.

6.9 Homofilie, výběr a sociální vliv

Z trojice vysvětlení se nyní podrobněji podíváme na homofilii, protože právě u ní se dá dobře ukázat, jak se testuje a jak souvisí s tím, zda vazby formují lidi, nebo lidé vazby.

Test homofilie

Motivace: „vrána k vráně sedá“ — vaši přátelé jsou vám podobnější věkem, rasou, zájmy a názory než náhodně vybraná skupina jednotlivců. *Homofilie* je princip, že máme tendenci být podobní svým přátelům.

Test: kdyby byl každému uzlu *náhodně* přiřazen atribut podle poměru v reálné síti, počet „různorodých“ hran by se od reálné sítě významně nelišil. Je-li podíl p mužů a q žen, je pravděpodobnost hrany muž–muž p^2 , žena–žena q^2 a *různopohlavní* hrany $2pq$.

Je-li podíl heterogenních (různopohlavních) hran významně nižší než $2pq$, jde o důkaz homofilie.

Příklad: různopohlavních hran je 5 z 18; $p = 6/9 = 2/3$, $q = 3/9 = 1/3$; bez homofilie by jich mělo být $2pq = 4/9$, tj. 8 z 18 $\Rightarrow$ *důkaz homofilie*.

Poznámky k testu homofilie. Formulace „významně nižší“ si zaslouží upřesnění: posuzuje se odchylka pod střední hodnotou. Co ale dělat, má-li síť *významně více* než $2pq$ různorodých hran? Pak nejde o selhání testu, nýbrž o *inverzní homofilii*, jejímž učebnicovým příkladem jsou partnerské vztahy muž–žena.

Test navíc není vázán na pohlaví. Použít se dá na libovolnou charakteristiku, ať už jde o rasu, věk, rodný jazyk, nebo preference. Má-li charakteristika více než dvě hodnoty, počítá se analogicky: počet heterogenních hran se porovná s tím, jak by vypadal náhodně generovaný graf, v němž jako pravděpodobnosti vystupují podíly z reálných dat.

Výběr vs. sociální vliv

Tranzitivní uzávěr (*triadic closure*) říká, že mají-li dva jednotlivci společného přítele, je přátelství mezi nimi pravděpodobnější. Homofilie k tomu dodává, že dva jednotlivci jsou si podobnější *kvůli* společnému příteli, takže vazba mezi nimi může vzniknout, i když ani jeden o společném příteli neví. Obvykle je proto obtížné přisoudit vznik vazby jedinému faktoru.

Výběr (*selection*) je tendence vybírat si přátele, kteří jsou nám podobní. Působí na různých úrovních intencionality: buď si v malé skupině podobné přátele aktivně vybíráte, *nebo* je populace vaší školy homogenní vůči celkové populaci, takže vás prostředí *nutí* vybírat si podobné přátele, aniž byste to jakkoli zamýšleli.

Mutabilní vs. imutabilní charakteristiky: *imutabilní* charakteristiky se nemění (pohlaví, rasa) nebo se mění konzistentně s populací (věk, generace), zatímco *mutabilní* se v čase mění (chování, přesvědčení, zájmy, názory).

Sociální vliv působí *opačně* než výběr. U *výběru* řídí individuální charakteristiky tvorbu vazeb; u *sociálního vlivu* naopak existující vazby formují mutabilní charakteristiky lidí.

Který z obou směrů v konkrétních datech převládá, z jediného snímku sítě rozhodnout nelze, a proto jsou nutné **longitudinální studie** (studie s dlouhým časovým rozpětím). Ty sledují sociální spoje i individuální chování v čase a odpovídají na otázku, *zda se mění chování po změnách v síti, nebo se*

mění síť po změnách v chování. Použily se například ke zjištění, zda měl na školní výsledky a užívání drog u teenagerů větší vliv výběr, nebo sociální vliv.

Odpověď má přímý dopad na návrh intervencí. Vykazuje-li užívání drog v síti homofilii, může nejlépe fungovat program cílený na *sociální vliv*, kdy přátelé přesvědčí ostatní přestat. Vznikla-li však homofilie *výběrem*, bývalí uživatelé si jednoduše vyberou nové přátele a chování ostatních to silně neovlivní.

Nejnámější studií tohoto typu je práce, v níž *Christakis a Fowler* sledovali obezitu a sociální síť 12 000 lidí po 32 let a našli homofilii podle obezity. Nabízela se tři vysvětlení: *efekty výběru* (vybírají si lidé přátele s podobným stavem?), *zmatení* (koreluje se stavem obezity i jiné faktory?) a *sociální vliv* (změnil-li se stav přátel, ovlivnilo to budoucí stav dané osoby?). **Nalezen byl významný důkaz pro třetí hypotézu:** obezita je typ „náказы“, který se může šířit sociálním vlivem.

6.10 Afiliační síť a uzávěrové procesy

Dosud jsme v síti viděli jen lidi a vazby mezi nimi. Reálné vazby ale vznikají v nějakém kontextu, a ten se dá do sítě zanést jako druhý typ uzlu.

Afiliační a sociálně-afilialní síť

Afilialní síť vkládá do sítě kontext tím, že zobrazuje spojení k aktivitám, společnostem, organizacím, čtvrtím atd. Je to *bipartitní graf*, v němž každá hrana spojuje dva uzly patřící do *různých* množin: osoby na jedné straně a *afilialce* čili *fokusy* na straně druhé.

Sociálně-afilialní síť postupuje o krok dál a má *dva typy hran*. Hrana *osoba–osoba* vyjadřuje přátelství nebo jiný sociální vztah, hrana *osoba–fokus* účast v daném fokusu.

Nad takovou sítí lze pojmenovat tři různé mechanismy, jimiž vznikají nové hrany.

Tři uzávěrové procesy

- **Tranzitivní uzávěr** (*triadic closure*) nastane, když se dva lidé spojí, protože mají společného přítele.
- **Fokální uzávěr** (*focal closure*) nastane, když se dva lidé spojí, protože mají společný fokus; jde tedy o uzávěr *výběrem*.
- **Členský uzávěr** (*membership closure*) nastane, když se osoba zapojí do fokusu, protože je v něm zapojen její přítel; jde o uzávěr *sociálním vlivem*.

Otázky o uzávěru. Zajímá nás, zda by Sue byla pravděpodobněji přítelkyní Alice, kdyby měly více než jednoho společného přítele. Formálněji se ptáme, *jaká je pravděpodobnost, že dva lidé vytvoří vazbu, jako funkce počtu společných přátel*. Zcela analogicky se ptáme u fokálního uzávěru, kde je argumentem počet společných fokusů, a u členského uzávěru, kde je jím počet přátel už v daném fokusu zapojených.

Metodika měření tranzitivního uzávěru

1. Pořiď dva snímky sítě v časech t_1 a t_2 .
2. Pro každé k identifikuj všechny *páry* uzlů, které mají v t_1 přesně k společných přátel, ale *nejsou* přímo spojeny hranou.
3. Definuj $T(k)$ jako *podíl* těchto párů, které do t_2 vytvořily hranu; to je hledaná pravděpodobnost, že mezi dvěma lidmi s k společnými přáteli vznikne vazba.
4. Vynes $T(k)$ jako funkci k .

Bázový (nulový) model: je-li pravděpodobnost vytvoření vazby s jedním společným přítelem p a jednotlivé společné přátele považujeme za *nezávislé* příležitosti, pak

$$T_{\text{base}}(k) = 1 - (1 - p)^k \quad (\text{resp. } 1 - (1 - p)^{k-1} \text{ podle konvence}).$$

Empirické výsledky. První měření provedli na *e-mailové síti* Kossinets a Watts (2006). Sledovali e-mailovou komunikaci 22 000 studentů velké americké univerzity po jeden rok; vazbu mezi dvěma lidmi předpokládali tehdy, byl-li mezi nimi za posledních 60 dní odeslán e-mail. Snímky pořizovali den po dni a $T(k)$ zprůměrovali přes více párů snímků. Výsledek má tři části: při *žádném* společném příteli se e-maily téměř nevyměňují, při přechodu z 1 na 2 společné přátele nastává *významný nárůst* a každý další nárůst je sice stále významný, ale týká se výrazně menší subpopulace. Uzávěr tedy vykazuje *klesající výnosy*.

Tiž autoři změřili i *fokální uzávěr*. Z rozvrhů týchž 22 000 studentů vytvořili sociálně-afiliační síť, v níž fokusy jsou vyučované předměty. Ukázalo se, že *sdílení předmětu má téměř stejný efekt jako sdílení přítele*, a to opět s klesajícími výnosy.

Pro *členský uzávěr* sestrojili Backstrom et al. (2006) sociálně-afiliační síť LiveJournalu, kde přátelství čerpali z profilů a fokusy odpovídají členství v uživatelsky definovaných komunitách. I zde nastal významný nárůst od 1 ke 2 a poté klesající výnosy. Totéž zopakovali Crandall et al. (2008) pro Wikipedii, kde uzly jsou editoři s diskusní stránkou, vazba znamená, že spolu přes diskusní stránku komunikovali, a fokusy jsou editované články; vzorec vyšel stejný.

Výběr a sociální vliv na datech Wikipedie. Podobnost dvou editorů se měřila jako

$$\frac{\text{počet článků editovaných oběma editory}}{\text{počet článků editovaných alespoň jedním editorem}}$$

Jde o obdobu překryvu sousedství pro hrany. Zjištění bylo jednoznačné: *páry editorů, které komunikovaly, jsou si v chování významně podobnější než ty, které nikdy nekomunikovaly*. Otevřená ovšem zůstává otázka, zda homofilie vzniká proto, že editoři mluví s editory téhož článku (*výběr*), nebo proto, že je k článkům dovedou ti, s nimiž mluví (*sociální vliv*). Analýza proto zprůměrovala historie interakcí, přičemž 0 označuje okamžik vzniku vazby, započítali se jen editoři s alespoň 100 akcemi před i po první interakci a baseline tvoří podobnost náhodného páru editorů. Podobnost roste *už před* vznikem vazby, což svědčí pro výběr, a pokračuje v růstu *i po něm*, což svědčí pro vliv.

6.11 Schellingův model segregace

Homofilie působí lokálně, na úrovni jednotlivce a jeho sousedů. Následující model ukazuje, jak překvapivě silné globální důsledky z ní mohou vzejít.

Schellingův model (T. Schelling, 70. léta)

Jednoduchý model, který ukazuje, jak mohou *globální* vzorce dobrovolné segregace vznikat z *lokální* homofilie.

- Populaci tvoří dva typy jednotlivců, agenti X a O ; oba typy reprezentují *imutabilní* charakteristiku (rasa, etnicita, země původu, rodný jazyk).
- Agenti jsou náhodně rozmístěni v mřížce.
- Agent je se svou polohou *spokojen*, je-li alespoň t jeho sousedů agentem téhož typu, kde t je předem nastavený prah.
- V každém kole se nejprve identifikují všichni *nespokojení* agenti a poté se všichni nespokojení přesunou na místo, kde budou spokojeni.

Někdy se stane, že agent žádné nové uspokojující místo nenajde; pak buď zůstane, kde je, nebo se přesune na náhodné místo. Přesuny navíc mohou způsobit, že *dříve spokojený agent přestane být spokojen*, a právě proto proces pokračuje dál.

Výsledek: mřížka je segregovanější než na začátku.

Varianty modelu: agenti se mohou přesouvat v náhodném pořadí nebo „zametacím“ pohybem, mohou mířit na nejbližší uspokojující místo nebo na náhodné, prah může být procentní nebo se lišit mezi skupinami i v rámci nich a svět se může „obtočit“, takže levý okraj navazuje na pravý. Variant existuje ještě celá řada, ale *výsledek zůstává stejný: samovolně vzniklá segregace*.

Závěr: Schellingův model je příkladem, jak se *imutabilní* charakteristiky mohou stát silně kore-

lovanými s *mutabilními*. Volba místa k životu je mutabilní a v čase se přizpůsobí typu agenta, který je imutabilní, čímž vzniká segregace, tedy homofilie.

6.12 Pozitivní a negativní vztahy: strukturní balance

Dosud měl každý vztah v síti pozitivní konotaci. Existují ale sítě, v nichž lze vyjádřit i vztahy negativní, tedy antagonistické či nepřátelské. *Pozitivní* vztahy jsou přátelské, *negativní* nepřátelské, a přirozeně vyvstane otázka, **kdy je síť v balancovaném stavu**.

Uvidíme přitom podobný jev jako u Schellingova modelu: lokální vlastnosti vynucují vlastnosti globální. Pro začátek si úlohu zjednodušíme a budeme uvažovat sítě, v nichž jsou *všechny* páry uzlů spojeny pozitivním nebo negativním vztahem, tedy úplné grafy čili kliky. Takový předpoklad se hodí pro malé skupiny lidí nebo pro státy a později jej uvolníme.

Vlastnost strukturní balance

Pro tři uzly existují (až na symetrii) čtyři konfigurace znamének:

Znaménka	Interpretace	Stav
+, +, +	A, B, C jsou vzájemní přátelé	balancovaný
+, -, -	A a B jsou přátelé se společným nepřítelem C	balancovaný
+, +, -	B a C jsou přátelé A , ale spolu nevycházejí	nebalancovaný
-, -, -	A, B, C jsou vzájemní nepřátelé	nebalancovaný

Vlastnost strukturní balance: pro každý trojúhelník jsou všechny tři hrany pozitivní, nebo je pozitivní právě jedna.

Tato čistě lokální podmínka, formulovaná pro jednotlivé trojúhelníky, má nečekaně silný globální důsledek.

Věta o balanci

Věta. Je-li označovaný *úplný* graf balancovaný, pak buď

- jsou všechny páry uzlů přátelé, *nebo*
- lze uzly rozdělit do dvou skupin X a Y tak, že každý pár uzlů v X se má rád, každý pár v Y se má rád, a každý v X je nepřítelem každého v Y .

Důkaz. Necht' G je balancovaná síť. Jsou-li všechny hrany pozitivní, jsme hotovi. Existuje-li alespoň jedna negativní hrana, vyber uzel A a polož X = všichni přátelé A (včetně A), Y = všichni nepřátelé A . Zbývá ověřit tři věci. (1) Každé dva uzly v X jsou přátelé, protože jinak bychom měli trojúhelník A + dva jeho přátelé s jedinou negativní hranou, tedy dvě kladné a jednu zápornou, což je nebalancované. (2) Každé dva uzly v Y jsou přátelé, protože jinak bychom měli trojúhelník A + dva jeho nepřátelé se třemi negativními hranami, což je rovněž nebalancované. (3) Každý uzel v X je nepřítelem každého uzlu v Y , jinak by trojúhelník A , přítel z X , nepřítel z Y měl dvě kladné a jednu zápornou hranu. $\square$

Aplikace strukturní balance. Klasickou ilustrací je *vývoj aliancí v Evropě 1872–1907*, kde plné tmavé hrany značí přátelství a tečkované červené nepřátelství. Sledujeme Ligu tří císařů 1872–81, Trojspolek 1882, přerušení německo-ruské smlouvy 1890, francouzsko-ruskou alianci 1891–94, Entente Cordiale 1904 a britsko-ruskou alianci 1907. Síť při tom postupně *sklouzává k balancovanému označování*, a tím do první světové války.

Druhým příkladem je *konflikt o odtržení Bangladéše od Pákistánu (1972)*. Poněkud překvapivá podpora Pákistánu ze strany USA je mnohem méně překvapivá, uvědomíme-li si, že SSSR byl nepřítelem Číny, Čína nepřítelem Indie a Indie měla tradičně špatné vztahy s Pákistánem. USA tehdy zlepšovaly vztahy s Čínou, a tak podpořily nepřátele nepřátel Číny.

Předpoklad úplného grafu i samotná definice balance jsou pro reálné sítě příliš přísné, takže se obojí dá oslabit.

Slabá strukturní balance a generalizace

Vlastnost slabé strukturní balance: neexistuje trojice uzlů, jejíž hrany tvoří *právě dvě pozitivní a jednu negativní*. Konfigurace „tři negativní“ je tedy nově dovolena, protože odpovídá situaci, kdy pár vytvoří alianci proti třetímu, zatímco „dvě pozitivní a jedna negativní“ je silnější nebalancovanost, neboť B a C se snaží svou animozitu vyřešit.

Věta. Je-li označovaný úplný graf slabě balancovaný, lze jeho uzly rozdělit do skupin vzájemných přátel tak, že každé dva uzly patřící do různých skupin jsou nepřátelé.

Důkaz. (1) Nechť A je uzel a X množina přátel A včetně A . (2) Všichni přátelé A jsou přátelé mezi sebou. (3) A a všichni jeho přátelé jsou nepřáteli všech ostatních. (4) Odstraň X ze sítě a pokračuj krokem 1. $\square$

Generalizace na neúplné grafy. (a) Nejsou-li všechny páry v síti spojeny, prohlásíme (neúplný) graf za *balancovaný* tehdy, lze-li jej „doplnit“ přidáním hran na označovaný úplný graf, který je balancovaný. (b) Nemusí-li být balancované všechny trojúhelníky, stačí, aby jich byla balancovaná většina; pak lze síť *přibližně* rozdělit na dvě frakce.

6.13 Analýza vazeb: huby, autority a algoritmus HITS

Rok 1998 byl pro modely analýzy vazeb na webu zásadní, protože v něm byly publikovány *oba* algoritmy, PageRank i HITS. Souvislosti mezi nimi jsou nápadné, přesto se dominantním modelem stal PageRank, a to díky své *nezávislosti na dotazu*, schopnosti bránit se spamu a obchodnímu úspěchu Googlu.

Autority a huby

Autorita (*authority*) je hrubě řečeno stránka s mnoha vstupními odkazy. Myšlenka za tím je jednoduchá: stránka může mít k nějakému tématu dobrý či autoritativní obsah, a proto jí mnoho lidí věří a odkazuje na ni.

Hub je naproti tomu stránka s mnoha *výstupními* odkazy. Slouží jako organizátor informací k danému tématu a odkazuje na mnoho dobrých autorit.

Klíčová myšlenka HITS obě role svazuje dohromady: *dobrý hub odkazuje na mnoho dobrých autorit a dobrá autorita je odkazována mnoha dobrými huby*. Autority a huby jsou tedy ve vztahu *vzájemného zesilování*, který se v grafu projeví jako bipartitní podgraf hustě propojených hubů a autorit.

Proč huby oddělovat od autorit? Protože konkurenční firmy na sebe neodkazují, takže je nelze chápat tak, že se přímo vzájemně podporují. V PageRanku naopak podpora přechází přímo z jedné prominentní stránky na druhou, neboť stránka je důležitá, je-li citována jinými důležitými stránkami. To je dominantní způsob podpory u akademických a vládních stránek, mezi blogery, u osobních stránek a ve vědecké literatuře.

Algoritmus HITS (*Hypertext Induced Topic Search*)

Zatímco PageRank je *statický*, HITS je *závislý na dotazu*. Po zadání dotazu proto nejprve rozšíří seznam relevantních stránek vrácených vyhledávačem a teprve pak nad rozšířenou množinou vytvoří *dvě* pořadí: pořadí autorit a pořadí hubů.

Sběr stránek. Pro obecný dotaz q se postupuje takto: pošli q vyhledávači a posbírej t nejvýše hodnocených stránek (v původním článku $t = 200$); této množině se říká *kořenová množina* S . Poté S rozšiř o každou stránku, na kterou odkazuje nějaká stránka z S , a o každou stránku, která odkazuje na stránku z S ; vznikne tak větší *bázová množina* B .

Skóre. Nechť $G = (V, E)$ je hypertextový graf B o n stránkách s maticí sousednosti L , kde $L_{ij} = 1$, existuje-li hrana (i, j) , a 0 jinak. Označme $a(i)$ skóre autority a $h(i)$ skóre hubu stránky i . Vzájemné zesilování se pak zapíše jako

$$a(i) = \sum_{(j,i) \in E} h(j), \quad h(i) = \sum_{(i,j) \in E} a(j),$$

maticově, s $a = (a(1), \dots, a(n))^T$ a $h = (h(1), \dots, h(n))^T$:

$$a = L^T h, \quad h = La.$$

Výpočet mocninnou iterací:

1: $a_0 \leftarrow h_0 \leftarrow (1, 1, \dots, 1)^T$; $k \leftarrow 1$

2: **repeat**

3: $a_k \leftarrow L^T h_{k-1}$; $h_k \leftarrow La_k$

4: $a_k \leftarrow a_k / \|a_k\|_1$; $h_k \leftarrow h_k / \|h_k\|_1$

▷ normalizace

5: $k \leftarrow k + 1$

6: **until** $\|a_k - a_{k-1}\|_1 < \varepsilon$ a $\|h_k - h_{k-1}\|_1 < \varepsilon$

7: **return** a a h

Normalizace je nezbytná, protože velikost hodnot autorit a hubů s každou aktualizací roste; konvergence nastane jen s normalizací, kdy se dělí součtem všech skóre.

Že iterace ke správným hodnotám opravdu konverguje, se ukáže standardní úvahou o vlastních číslech.

Spektrální analýza HITS

Po k krocích platí (spektrum matice = množina jejích vlastních čísel)

$$a_k = (L^T L)^{k-1} a_0, \quad h_k = (L L^T)^k h_0.$$

Vektor a_k tedy musí konvergovat k *vlastnímu vektoru* matice $L^T L$. Existují totiž normalizační konstanty c_k takové, že a_k/c_k konverguje k limitě, a směr se při násobení maticí nezmění právě tehdy, je-li vektor vlastním vektorem.

Matice $L^T L$ je *symetrická*, má tedy n vlastních vektorů $z_1, \dots, z_n$ s vlastními čísly $c_1, \dots, c_n$, kde $|c_1| \geq |c_2| \geq \dots \geq |c_n|$. Rozvineme-li $a_0 = x_1 z_1 + \dots + x_n z_n$, dostaneme

$$a_k = c_1^k x_1 z_1 + \dots + c_n^k x_n z_n, \quad \frac{a_k}{c_1^k} = x_1 z_1 + x_2 \left(\frac{c_2}{c_1}\right)^k z_2 + \dots$$

Za předpokladu $c_1 > c_2 \geq \dots$ jde pro $k \rightarrow \infty$ každý člen kromě prvního k nule, takže a_k/c_1^k konverguje k $x_1 z_1$, tedy k *hlavnímu* vlastnímu vektoru. (Konvergenci nezávisle na počátečním vektoru a uvolnění předpokladu $c_1 > c_2$ zde nedokazujeme.)

Obě matice, které se v HITS objevily, mají v bibliometrii dlouho zavedený význam.

Vztah ke ko-citaci a bibliografickému párování

Ko-citace stránek i a j udává, kolik článků cituje *oba* články i a j :

$$C_{ij} = \sum_{k=1}^n L_{ki} L_{kj} = (L^T L)_{ij}.$$

Matice autorit $L^T L$ algoritmu HITS je tedy *ko-citační matice*.

Bibliografické párování stránek i a j naproti tomu udává, kolik článků je citováno *oběma* články i a j :

$$B_{ij} = \sum_{k=1}^n L_{ik} L_{jk} = (L L^T)_{ij}.$$

Matice hubů $L L^T$ je tedy matice bibliografického párování.

Silné a slabé stránky HITS. *Silnou stránkou* je schopnost řadit stránky *podle tématu dotazu*, což

může poskytnout relevantnější autority a huby. *Slabé stránky* jsou tři. (1) HITS lze snadno spamovat, protože přidat výstupní odkazy na vlastní stránku je velmi snadné. (2) Nastává *drift tématu* (*topic drift*), neboť mnoho stránek v rozšířené množině nemusí být k tématu. (3) Algoritmus je neefektivní v době dotazu, jelikož sběr kořenové množiny, její rozšíření i výpočet vlastních vektorů jsou drahé operace.

6.14 PageRank

Než dojdeme k PageRanku samotnému, hodí se pojmenovat myšlenku, na které stojí: hlasy nemají stejnou váhu.

Prestiž pořadí (*rank prestige*)

U měř prestiže se uvažuje důležitý faktor, totiž *prominence jednotlivých aktérů, kteří „hlasují“*. V reálném světě je osoba vybraná důležitou osobou prestižnější než osoba vybraná méně důležitou, protože hlas generálního ředitele váží více než hlas zaměstnance. Je-li váš okruh vlivu plný prestižních aktérů, je i vaše vlastní prestiž vysoká.

Prestiž pořadí $P(i)$ je proto definována jako lineární kombinace pořadí těch, kteří odkazují na i :

$$P(i) = A_{1i}P(1) + A_{2i}P(2) + \dots + A_{ni}P(n), \quad \text{maticově} \quad P = A^T P,$$

kde $A_{ji} = 1$, odkazuje-li j na i , a 0 jinak. To je *charakteristická rovnice* vlastního systému matice A^T , takže P je vlastní vektor A^T .

Základní PageRank

PageRank interpretuje hyperodkaz ze stránky u na stránku v jako *hlas* stránky u pro stránku v . Nezajímá jej ovšem pouhý počet hlasů: analyzuje i stránku, která hlas dává, neboť *hlasy důležitých stránek váží více a pomáhají činit ostatní stránky důležitějšími*. A protože stránka může odkazovat na mnoho jiných, musí své skóre rozdělit.

Web chápeme jako orientovaný graf $G = (V, E)$ s $n = |V|$. PageRank stránky i je pak

$$P(i) = \sum_{(j,i) \in E} \frac{P(j)}{O_j},$$

kde O_j je počet výstupních odkazů j . Maticově: nechť $A_{ij} = 1/O_i$ pro $(i, j) \in E$ a 0 jinak; pak $P = A^T P$.

Intuice: PageRank se chová jako „kapalina“, která cirkuluje sítí, přechází od uzlu k uzlu po hranách a shromažďuje se v nejdůležitějších uzlech. *Celkový PageRank v síti zůstává konstantní*, protože každá stránka svůj PageRank rozdělí a pošle po odkazech dál. PageRank se nikdy nevytváří ani nezaniká, jen se přemísťuje, a proto na rozdíl od HITS *není nutná normalizace*.

Problém základní definice. V mnoha sítích skončí veškerý PageRank ve „špatných“ uzlech. Odkazují-li si F a G navzájem místo na A , PageRank, který k nim přiteče z C , se už nikdy nemůže vrátit; pro velká k dostaneme $\frac{1}{2}$ pro každý z F , G a 0 pro všechny ostatní. Tomuto jevu se říká „pomalý únik“ (*slow leak*) a způsobují jej malé množiny uzlů dosažitelné z ostatního grafu, ale bez cest zpět. **Řešení:** posílat malý zlomek PageRanku *všem* uzlům.

Škálovaný PageRank a tlumicí faktor

Zvol *škálovací faktor* d mezi 0 a 1 a nahraď základní pravidlo aktualizace takto:

1. nejprve aplikuj základní pravidlo;
2. všechny hodnoty PageRanku sniž faktorem d , čímž se celkový PageRank v síti zmenší z 1 na d ;
3. zbývajících $1 - d$ jednotek rozděl rovnoměrně mezi všechny uzly, tj. $(1 - d)/n$ každému.

Proč to funguje: je to „vodní cyklus“, který v každém kroku $1-d$ jednotek PageRanku „odpaří“ a rovnoměrně je „sprchne“ na všechny uzly.

Opakovaná aplikace škálovaného pravidla *konverguje* k limitním hodnotám PageRanku a ekvilibrium je *jednoznačné*; hodnoty ovšem závisejí na volbě d . V praxi se používá d mezi 0,8 a 0,9 ($d = 0,85$ v původním článku) a d se nazývá *tlumicí faktor* (*damping factor*).

Celý mechanismus se dá popsat i jazykem teorie pravděpodobnosti, což zároveň vysvětlí, odkud se berou podmínky konvergence.

PageRank jako markovský řetězec

Web se modeluje jako *stochastický proces*, v němž každá stránka je *stav* a každý hyperodkaz *přechod* s určitou pravděpodobností. Při *náhodném surfování* je každá přechodová pravděpodobnost $1/O_i$, předpokládáme-li, že surfař klikne na odkazy uniformně náhodně a nepoužívá tlačítko „zpět“ ani nezadáva URL.

Nechť A je matice přechodových pravděpodobností a p_0 počáteční rozdělení pravděpodobnosti. Pak $p_1 = A^T p_0$ a obecně $p_k = A^T p_{k-1}$. Věta o markovských řetězcích nyní říká, že konečný markovský řetězec definovaný stochastickou maticí A má **jednoznačné stacionární rozdělení**, je-li A *ireducibilní* a *aperiodická*. Stacionární rozdělení znamená, že p_k konverguje ke stálému vektoru π nezávisle na volbě p_0 . V ustáleném stavu platí $\pi = A^T \pi$, takže π je hlavní vlastní vektor A^T s vlastním číslem 1, a právě π se použije jako vektor PageRanku. Je to rozumné a intuitivní, protože π odráží *dlouhodobé pravděpodobnosti*, že náhodný surfař stránku navštíví, a stránka má vysokou prestiž, je-li pravděpodobnost její návštěvy vysoká.

Ani jedna podmínka však pro webový graf neplatí:

- A *není stochastická*, protože mnoho stránek nemá výstupní odkazy, což se v matici projeví řádky samých nul (*visící stránky*, *dangling pages*). Nabízejí se dvě řešení: (1) takové stránky během výpočtu odstranit, neboť neovlivňují přímo pořadí žádné jiné stránky; (2) přidat z každé takové stránky úplnou množinu odkazů na všechny stránky, tj. položit $A_{ij} = 1/n$.
- A *není ireducibilní*, neboť ireducibilita znamená, že webový graf je *silně souvislý*, tedy že pro každý pár i, j existuje cesta z i do j ; obecný webový graf silně souvislý není.
- A *není aperiodická*. Stav i je *periodický* s periodou $k > 1$, je-li k nejmenší číslo takové, že všechny cesty ze stavu i zpět do i mají délku, která je násobkem k ; není-li stav periodický ($k = 1$), je aperiodický, a řetězec je aperiodický, jsou-li aperiodické všechny stavy. Příkladem periodického řetězce s $k = 3$ je situace, kdy se z 1 lze vrátit do 1 jen cestou $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$, takže každý návrat trvá 3 přechody.

Obě posledně jmenované vlastnosti se řeší jedinou strategií: přidá se z každé stránky odkaz na každou stránku a každému takovému odkazu se dá malá přechodová pravděpodobnost řízená parametrem d . Takto rozšířená přechodová matice je zjevně ireducibilní i aperiodická.

Finální PageRank

Po augmentaci má náhodný surfař na stránce dvě možnosti: s pravděpodobností d zvolí náhodně odkaz, který bude následovat, a s pravděpodobností $1-d$ skočí na náhodnou stránku. Odtud

$$P = \left(\frac{1-d}{n}\right) \mathbf{E} + d A^T P,$$

kde $\mathbf{E} = \mathbf{e}\mathbf{e}^T$ a $\mathbf{e}$ je sloupcový vektor jedniček, takže $\mathbf{E}$ je matice $n \times n$ samých jedniček. Matice $\left(\frac{1-d}{n}\right)\mathbf{E} + dA^T$ je stochastická (transponovaná), ireducibilní i aperiodická. Přeskálujeme-li ji tak, aby $\mathbf{e}^T P = n$, dostaneme

$$P = (1-d)\mathbf{e} + d A^T P, \quad \text{tj. pro každou stránku } i: \quad P(i) = (1-d) + d \sum_{j=1}^n A_{ji} P(j),$$

což je ekvivalentní formulí z původního článku o PageRanku

$$P(i) = (1 - d) + d \sum_{(j,i) \in E} \frac{P(j)}{O_j}.$$

Výpočet mocninnou iterací:

```
1:  $P_0 \leftarrow \mathbf{e}/n$ ;  $k \leftarrow 1$ 
2: repeat
3:    $P_k \leftarrow (1 - d)\mathbf{e} + dA^\top P_{k-1}$ 
4:    $k \leftarrow k + 1$ 
5: until  $\|P_k - P_{k-1}\|_1 < \varepsilon$ 
6: return  $P_k$ 
```

Výhody PageRanku. První výhodou je *boj se spamem*: stránka je důležitá, jsou-li důležité stránky, které na ni odkazují, a protože pro majitele stránky není snadné přidat *vstupní* odkazy z jiných důležitých stránek, není snadné PageRank ovlivnit. Druhou výhodou je *globální a dotazově nezávislá míra*: hodnoty PageRanku všech stránek se počítají a ukládají *offline*, nikoli v době dotazu. **Kritika** míří právě na tuto dotazovou nezávislost, protože PageRank neumí rozlišit stránky autoritativní *obecně* od stránek autoritativních *k tématu dotazu*.

Souhrn. Analýza vazeb nám ukázala principy *centrality* a *prestiže*, dvojici měr *ko-citace* a *bibliografického párování* a algoritmy *HITS* a *PageRank*, z nichž druhý stojí za Googlem. Yahoo! a MSN mají vlastní nepublikované algoritmy založené na odkazech. **Důležité je nezapomenout**, že hyperodkazové řazení není jediný algoritmus používaný ve vyhledávačích: kombinuje se s mnoha faktory založenými na obsahu, aby vzniklo finální pořadí prezentované uživateli.

6.15 Tematické komunity a jejich objevování

Odkazy se dají využít i k něčemu jinému než k hodnocení důležitosti jednotlivých stránek. Lze z nich vyčíst *tematické komunity*, tedy skupiny tvůrců obsahu nebo lidí, kteří sdílejí nějaký společný zájem. Podle toho, v jakých datech se hledají, se mluví o webových komunitách, o e-mailových komunitách nebo o komunitách pojmenovaných entit.

Blízko k tomu má *cílené procházení* (*focused crawling*), které kombinuje obsah stránek a odkazy mezi nimi, aby prošlo jen tu část webu, jež se týká konkrétního tématu. Prohledávač následuje odkazy a u každé navštívené stránky pomocí učení a klasifikace rozhodne, zda k tématu patří.

Tematická komunita

Mějme danou konečnou množinu entit $E = \{e_1, \dots, e_n\}$ *téhož typu*. *Tematická komunita* je pár $K = (T, M)$, kde T je *téma* komunity a $M \subseteq E$ množina všech entit z E , které téma T sdílejí. Je-li $e \in M$, říkáme, že e je *členem* komunity K . Z takto postavené definice plyne několik vlastností:

- *Téma definuje komunitu.* Pro dané téma T je množina členů určena jednoznačně, takže dvě komunity jsou si rovny, mají-li totéž téma.
- *Téma lze definovat libovolně.* Může jím být událost, třeba sportovní událost nebo skandál, stejně dobře ale i celý koncept, například dolování z webu.
- *Entita může být v libovolném počtu komunit.* Komunity se proto mohou *překrývat* a více komunit může sdílet členy.
- *Entity v E jsou téhož typu.* Lidé a organizace tedy nemohou být v téže komunitě.

Komunity mohou tvořit *hierarchickou strukturu*: tematická komunita (T, M) může obsahovat množinu tematických *subkomunit* $\{(T_1, M_1), \dots, (T_k, M_k)\}$, kde T_i je podtéma T a $M_i \subseteq M$. Původní komunita (T, M) pak vystupuje jako *superkomunita* a každou subkomunitu lze dekomponovat dál.

Cílem objevování je najít v datové sadě obsahující entity skryté komunity, a to u každé z nich jak její *téma* (obvykle reprezentované množinou klíčových slov), tak její *členy*.

Jak se komunita projeví v datech. Členové komunity jsou spolu nějak propojeni a přidružené texty obsahují slova indikativní pro téma komunity. Konkrétní podoba obojího se ale liší podle typu dat. Na *webových stránkách* jsou členové komunity spíše vzájemně propojeni *hyperodkazy* a jejich stránky obsahují *slova* vztažená k tématu. V *e-mailech* spolu členové spíše komunikují a slova vztažená k tématu nesou samotné zprávy. V *textových dokumentech* se členové spíše objevují *spolu* v téže větě či v témže dokumentu a téma indikují slova v těchto větách.

Bipartitní jádra komunit

(i, j) -*bipartitní jádro* je úplný bipartitní podgraf s alespoň i *vějířovými* uzly (*fans*) a alespoň j *středovými* uzly (*centers*), obsahuje tedy všechny možné hrany mezi i vějířovými a j středovými uzly. Obě strany jádra mají svůj protějšek ve slovníku hubů a autorit: vějíře reprezentují *specializované huby*, zatímco středy odpovídají *autoritám*.

Postup hledání bipartitních jader má dva kroky:

- *Krok 1: prořezání.* Nejprve se prořezává podle vstupního stupně, tedy odstraní se všechny stránky, které jsou velmi silně odkazované (počet vstupních odkazů $> k$, např. $k = 50$). Potom se iterativně prořezávají vějíře a středy, aby zbyli kandidáti na (i, j) -jádra: odstraní se všechny vějíře s výstupním stupněm menším než j , odstraní se všechny středy se vstupním stupněm menším než i a z grafu se odstraní hrany spojené s odstraněnými uzly.
- *Krok 2: generování všech (i, j) -jader.* Nejdříve se najdou všechna $(1, j)$ -jádra, tj. množina všech uzlů s výstupním stupněm alespoň j . Všechna $(2, j)$ -jádra se pak zkonstruují kontrolou každého vějíře, který odkazuje také na některý střed v $(1, j)$ -jádre, a stejným způsobem se pokračuje až do i .

Omezení. Algoritmus najde jen *jádrové* stránky komunit, ne všechny členské stránky. Nenažde ani *témata* komunit, ani jejich hierarchickou strukturu.

Překrývající se komunity pojmenovaných entit

Cílem je najít komunity entit z textového korpusu. **Hlavní idea** spočívá v tom, že dvě entity jsou propojeny, *vyskytnou-li se v téže větě*: jsou-li dva lidé zmíněni v jedné větě, obvykle mezi nimi nějaký vztah existuje. Jedna entita se přitom může objevit ve více komunitách a téma komunity reprezentují nejvýznamnější klíčová slova. Metoda přinesla slibné výsledky při hledání komunit politických osobností a celebrit z webových dokumentů.

Algoritmus probíhá ve čtyřech krocích:

1. *Postav graf vazeb.* Každý dokument se parsuje a v každé větě se identifikují obsažené pojmenované entity. Má-li věta více než jednu entitu, propojí se tyto entity *po párech* a klíčová slova z věty se k propojeným párům připojí jako textový obsah. Ostatní věty se zahodí.
2. *Najdi všechny trojúhelníky.* Trojúhelník tvoří tři entity svázané dohromady a reprezentuje základní stavební blok komunit. Vychází se z pozorování, že komunita má tendenci se rozšiřovat trojúhelníky sdílejícími společnou hranu.
3. *Najdi jádra komunit.* Jádro je skupina těsně svázaných trojúhelníků, tedy *uvolněný* úplný podgraf (kliky). Intuitivně jde o množinu těsně propojených členů komunity.
4. *Klastruj okolo jader.* Trojúhelníky a páry entit, které neleží v žádném jádre, se přiřadí k jádrům podle *podobnosti textového obsahu* s nalezenými jádry.

6.16 Detekce komunit

Předchozí metody hledaly komunity i s jejich tématy, tedy současně v obsahu a ve struktuře. Detekce komunit v užším smyslu si klade skromnější otázku: jak se síť rozpadá na skupiny, když se díváme jen na její topologii.

Komunita a detekce komunit

Komunity jsou tvořeny jednotlivci tak, že ti *uvnitř* skupiny interagují mezi sebou *častěji* než s těmi vně skupiny. V různých kontextech se jim říká také *skupina*, *klastr*, *kohezní podgrupa* nebo *modul*.

Detekce komunit je objevování skupin v síti, kde příslušnost jednotlivců ke skupinám *není explicitně dána*.

Definice komunity je subjektivní. Komunitou může být hustě propojený podgraf, stejně tak jí ale může být každá komponenta souvislosti.

V sociálních médiích se rozlišují dva typy skupin: *explicitní* skupiny tvořené předplacením uživatelů a *implicitní* skupiny tvořené sociálními interakcemi.

Nabízí se otázka, proč skupiny vůbec extrahovat z topologie, když některé weby umožňují do skupin přímo vstoupit. Důvody jsou tři: ne všechny weby platformu pro komunity poskytují, ne všichni lidé jsou ochotni vynaložit úsilí na vstup do skupiny a skupiny se mohou *dynamicky měnit*. Interakce v síti navíc samy o sobě poskytují informaci o vztazích mezi uživateli. Doplňují ostatní druhy informací, pomáhají vizualizaci a navigaci a poskytují základ pro další úlohy.

Metody minimálního řezu

Metoda **minimálního řezu** (Ahuja et al., 1993) vychází z prosté úvahy: většina interakcí nastává *uvnitř* skupiny, kdežto mezi skupinami jsou interakce vzácné $\Rightarrow$ detekce komunit $\approx$ úloha minimálního řezu.

Řez je rozdělení uzlů grafu do dvou disjunktních množin. *Úloha minimálního řezu* hledá takové rozdělení, které minimalizuje počet hran mezi oběma množinami, u vážených sítí součet jejich vah; tomuto počtu se říká *velikost řezu*.

Chceme-li $k > 2$ skupin, implementuje se strategie *iterativního bisekce*: ve druhé a dalších iteracích se už nalezené skupiny postupně dělí na dvě podskupiny.

Nevýhodou je, že metoda produkuje skupiny nevyvážených velikostí a jedna z množin často bývá jednoprvková.

Složitost. Jednoduché řešení pomocí *s-t řezu* založeného na maximálním toku má složitost $O(n^5)$; implementace Kargerovým algoritmem klesne na $O(n^4)$, ale nezaručuje vždy nalezení minimálního řezu. Pro jednotkové váhy $w_{ij} = 1$ a bez balančních omezení je úloha 2-cestného řezu polynomiálně řešitelná v $O(n^2 \log n)$ (Kargerův randomizovaný minimální řez). Jakmile se sejdou obecné váhy hran a balanční omezení, stává se úloha **NP-těžkou**.

Poměrový a normalizovaný řez

Minimální řez často vrací nevyvážené rozdělení, kde je jedna množina jednoprvková $\Rightarrow$ změníme účelovou funkci tak, aby brala v úvahu *velikost komunity*. Pro rozdělení $\pi = \{C_1, \dots, C_k\}$ se definují dvě míry:

$$\text{RatioCut}(\pi) = \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{|C_i|}, \quad \text{NCut}(\pi) = \sum_{i=1}^k \frac{\text{cut}(C_i, \bar{C}_i)}{\text{vol}(C_i)},$$

kde $|C_i|$ je počet uzlů v C_i , $\text{vol}(C_i)$ součet stupňů v C_i a $\text{cut}(A, B) = \sum_{i \in A, j \in B} a_{ij}$. **Obě míry preferují vyvážené rozdělení.**

Girvanové–Newmanův algoritmus (2002)

Jde o *divizivní hierarchický* algoritmus, tedy o přístup shora dolů: dekomponuje počáteční úplný graf na postupně menší souvislé části, dokud nezůstanou žádné hrany k odstranění a každý uzel nereprezentuje sám komunitu.

Idea je identifikovat hrany spojující uzly z *různých* komunit, tzv. mosty, a iterativně je z grafu odstraňovat. Slouží k tomu **kritérium mezilehlosti hrany** (*edge betweenness*), což je podíl nejkratších cest procházejících danou hranou. Hrany s vysokou mezilehlostí leží na velkém počtu

nejkratších cest mezi uzly, a jsou tedy mosty.

- 1: spočítej mezilehlost všech hran v síti
- 2: odstraň hranu s *nejvyšší* mezilehlostí
- 3: opakuj předchozí kroky, dokud v grafu zbývají hrany k odstranění

Vstupem je celý graf, **výstupem** hierarchická struktura, reprezentovaná např. dendrogramem.

Volba počtu komunit. Algoritmus vrátí *množinu* možných řešení, ale neurčí, které z nich je nejlepší. Nejlepší rozdělení se proto vybere tak, že se pro každé dělení sítě spočítá *modularita* a vezme se rozdělení s nejvyšší modularitou.

Nevýhodou jsou vysoké výpočetní náklady, kvůli nimž je algoritmus vhodný pouze pro sítě *střední velikosti*: $O(m^2n)$ pro graf o m hranách a n uzlech, resp. $O(n^3)$ pro řídké sítě.

Příklad výpočtu mezilehlosti hrany: mezilehlost hrany $e(1, 2)$ je 4, protože všechny nejkratší cesty z 2 do $\{4, 5, 6, 7, 8, 9\}$ musí procházet buď $e(1, 2)$, nebo $e(2, 3)$, a $e(1, 2)$ je nejkratší cesta mezi 1 a 2. Po odstranění $e(4, 5)$ se mezilehlost $e(4, 6)$ zvýší na 20, což je nejvyšší hodnota; po odstranění $e(4, 6)$ má nejvyšší mezilehlost 4 hrana $e(7, 9)$.

Modularita Q

Modularita kvantifikuje kvalitu daného rozdělení grafu na komunity. **Základní idea** je ta, že síť má smysluplnou komunitní strukturu, je-li počet hran *mezi* komunitami menší, než by se očekávalo při náhodné volbě. Očekávaný počet hran mezi uzly i a j se stupni k_i , k_j je $k_i k_j / 2m$, odkud

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j),$$

kde A_{ij} je prvek matice sousednosti (počet hran mezi i a j), m počet hran, k_i stupeň uzlu i , c_i skupina, do níž i patří, a δ Kroneckerovo delta.

Hodnoty. Modularita nabývá hodnot $Q \in [-1, 1]$ a může být pozitivní i negativní. Je-li pozitivní, existuje možnost nalézt v síti komunitní strukturu; jsou-li hodnoty velké a pozitivní, může odpovídající rozdělení odrážet skutečnou komunitní strukturu. Pro smysluplné komunity by mělo být $Q \geq 0,3$ (Clauset et al., 2004).

Louvainova metoda (Blondel et al., 2008)

Louvainova metoda je *hladová* optimalizační metoda pro *aglomerativní hierarchickou* maximalizaci modularity a běží ve dvou fázích.

Ve **fázi 1** se každý uzel považuje za samostatnou komunitu a postupuje se takto:

1. Spočítej modularitu Q .
2. Přesuň izolovaný uzel z jeho komunity do sousední komunity.
3. Spočítej zisk či ztrátu modularity, kterou tato změna přiřazení způsobila.
4. Zvýší-li se modularita, tedy jde-li o zisk, ponech uzel v „nové“ komunitě.
5. Opakuj, dokud není možné žádné další zlepšení.

Výstupem fáze 1 je rozdělení k -té úrovně, kde k je číslo iterace.

Ve **fázi 2** se vytvoří nová síť (*supergraf*) odvozená z původní. Každý nový uzel (*superuzel*) je agregací uzlů přiřazených dané komunitě nalezené ve fázi 1 a dva superuzly jsou spojeny hranou, existuje-li alespoň jedna hrana spojující dva uzly uvnitř odpovídajících komunit. Síť se tím zmenší a lze na ni znovu pustit fázi 1.

Iterace. Obě fáze se střídají tak dlouho, dokud není dosaženo maximální modularity.

Výhodou je dobrý výkon i na velkých síťových grafech při nízkých výpočetních nákladech $O(n \log n)$, kde n je počet uzlů. **Nevýhodou** je, že výsledky jsou *citlivé na pořadí* zpracování.

Další nevýhody. Metoda může produkovat *nesouvislé komunity*: přesune-li se uzel do jiné komunity, může se původní komunita stát *vnitřně nesouvislou*, a přesto zůstanou její uzly lokálně optimálně přiřazeny, takže v ní zůstanou. Metoda dále *uváže v lokálních optimech* a *selhává v upřesnění komunit* po počátečním slučování.

Leidenův algoritmus (Traag, Waltman, van Eck, 2019)

Leidenův algoritmus je upřesněním Louvainovy metody pro aglomerativní hierarchickou maximalizaci modularity. Probíhá ve třech fázích: (1) lokální přesuny uzlů, obdobně jako u Louvaina; (2) **upřesnění rozdělení**, což je právě ten krok, který Louvainovi chybí; (3) agregace sítě, opět obdobně jako u Louvaina.

Výhodou je vyšší efektivita než u Louvainovy metody. U obrovských grafů ale i tak může docházet k dlouhým dobám zpracování a rychlost pak mohou zvýšit paralelní vícejádrové implementace. **Nevýhodou** je *tvrdé rozdělení*, kdy uzly mohou patřit jen do jedné komunity.

Výzvy detekce komunit. Přes velké množství algoritmů zůstává detekce komunit náročným problémem, a to ze dvou hlavních důvodů. (1) Počet komunit obvykle není znám a je třeba jej určit. (2) Komunity jsou obvykle *heterogenní* co do velikosti a hustoty.

6.16.1 Taxonomie metod detekce komunit

Metody lze rozdělit do čtyř kategorií podle toho, na jak velký kus sítě se jejich kritérium dívá; samotná kritéria se přitom mohou podle úlohy lišit. *Uzlově orientované* metody žádají, aby určité vlastnosti splňoval každý uzel ve skupině. *Skupinově orientované* metody se dívají na spojení uvnitř skupiny jako celku a nezoomují až na úroveň jednotlivých uzlů. *Síťově orientované* metody pracují s celou sítí a rozdělují ji do několika disjunktních množin. A konečně *hierarchicky orientované* metody konstruují hierarchickou strukturu komunit.

Uzlově orientovaná detekce komunit

Po uzlech se může požadovat, aby splňovaly různé vlastnosti používané v tradiční SNA. Následující čtyři rodiny definic se liší právě tím, kterou vlastnost si vyberou.

Úplná vzájemnost: kliky. *Klika* je maximální úplný podgraf tří a více uzlů, z nichž všechny jsou navzájem sousední. Nalezení maximální kliky je **NP-těžké** a přímočará implementace vyžaduje časově velmi drahé. Pomůže *idea rekursivního prořezávání*: v klice velikosti k má každý uzel stupeň $\geq k - 1$, takže uzly se stupněm $< k - 1$ v maximální klice nebudou. Opakovaně se tedy vzorkuje podsít, v ní se najde klika (např. hladově), a je-li velikosti k , odstraní se pro nalezení větší kliky všechny uzly se stupněm $\leq k - 1$; postup se opakuje, dokud není síť dost malá. V sociálních médiích se takto prořeže mnoho uzlů, protože stupně uzlů následují mocninné rozdělení. Kliky se běžně používají jako *jádro* či *semeno* pro prozkoumání větších komunit.

Metoda perkolace klik (*clique percolation method*, CPM) reaguje na to, že klika je striktní definice a v praxi nestabilní. Zároveň je to příklad metody, která umí najít *překrývající se* komunity.

Vstupem je parametr k a síť. *Postup* začne nalezením všech klik velikosti k , z nichž se zkonstruuje *graf klik*: dvě kliky jsou v něm sousední, sdílejí-li $k - 1$ uzlů. Každá komponenta souvislosti grafu klik pak tvoří jednu komunitu. *Příklad*: kliky velikosti 3 jsou $\{1, 2, 3\}$, $\{1, 3, 4\}$, $\{4, 5, 6\}$, $\{5, 6, 7\}$, $\{5, 6, 8\}$, $\{5, 7, 8\}$, $\{6, 7, 8\}$ $\Rightarrow$ komunity $\{1, 2, 3, 4\}$ a $\{4, 5, 6, 7, 8\}$, přičemž uzel 4 je v obou.

Dosažitelnost: k -klika, k -klub, k -klan. Hlavní ideou je, že každý uzel ve skupině má být dosažitelný v k skocích. k -klika je maximální podgraf, v němž je největší geodetická vzdálenost mezi libovolnými dvěma uzly $\leq k$. Pozor na to, že k -klika může mít *uvnitř podgrafu* průměr $\geq k$: v jiném příkladovém grafu ze slidů má 2-klika $\{12, 4, 10, 1, 6\}$ $d(1, 6) = 3$. k -klub je podstruktura s průměrem $\leq k$ a k -klan je k -klika s průměrem $\leq k$. *Příklad*: kliky $\{1, 2, 3\}$; 2-kliky $\{1, 2, 3, 4, 5\}$, $\{2, 3, 4, 5, 6\}$; 2-kluby $\{1, 2, 3, 4\}$, $\{1, 2, 3, 5\}$, $\{2, 3, 4, 5, 6\}$.

Stupně uzlů: k -jádro, k -plex. Zde má mít každý uzel určitý počet spojení na uzly *uvnitř* skupiny. k -jádro (k -core) je podstruktura, kde je každý uzel spojen s alespoň k dalšími členy skupiny. U k -plexu má být ve skupině o n_s uzlech každý uzel sousední s ne méně než $n_s - k$ uzly ve skupině. Obě definice jsou komplementární, protože k -jádro je $(n_s - k)$ -plex. Množina všech k -jader pro dané k však *není* identická s množinou všech $(n_s - k)$ -plexů, protože velikost skupiny n_s se mezi k -jádry může lišit.

Vazby uvnitř vs. vně: LS množiny. *LS množina* (Luccio–Sami) je definována tak, že libovolná její vlastní podmnožina má více vazeb na ostatní uzly *ve* skupině než *mimo* ni. Cílem bylo najít kohezni skupiny uzlů pro návrh obvodů na čipech; pro síťovou analýzu je taková definice *příliš striktní*. Jejím uvolněním je *k-komponenta*, tedy maximální podgraf, který zůstává souvislý i po odstranění méně než k uzlů či hran. Vyžaduje ovšem výpočet hranové souvislosti mezi libovolným párem uzlů algoritmem minimálního řezu či maximálního toku.

Omezení uzlově orientovaných definic. Jsou příliš striktní, byť je lze použít jako jádro komunity. Nejsou škálovatelné, takže se běžně používají pro sítě malé velikosti. A někdy nejsou konzistentní s vlastnostmi rozsáhlých sítí, například se stupni uzlů v bezškálových sítích.

Skupinově orientovaná detekce: kvazikliky

Skupinově orientovaná kritéria vyžadují, aby určitou podmínku splňovala *celá skupina*, například aby hustota skupiny byla dostatečně velká, tedy $\geq$ daný prah. Podgraf $G_S = (V_S, E_S)$ je γ -hustá kvaziklika, je-li

$$|E_S| \geq \gamma \cdot \frac{|V_S|(|V_S| - 1)}{2}.$$

Použít lze strategii podobnou té u klik. *Lokální prohledávání* vzorkuje podgraf a najde v něm maximální γ -hustou kvazikliku velikosti k , *heuristické prořezávání* pak odstraní uzly se stupněm menším než průměrný stupeň, tedy $< \gamma(k - 1)$.

Síťově orientovaná detekce komunit

Kritéria uvažují spojení v síti *globálně* a cílem je rozdělit uzly sítě do *disjunktních* množin. Přístupů je pět: shlukování podle podobnosti uzlů, modely latentního prostoru, aproximace blokovým modelem, spektrální shlukování a maximalizace modularity.

Shlukování podle podobnosti uzlů. Na uzly se aplikuje k -means nebo shlukování založené na podobnosti, přičemž podobnost uzlů je definována podobností jejich *sousedství*. *Strukturní ekvivalence* říká, že dva uzly jsou strukturně ekvivalentní právě tehdy, jsou-li spojeny s *touž* množinou aktérů; pro praktické použití je to příliš restriktivní. Prakticky se proto spojení chápou jako příznaky a měří se

$$\text{Jaccard}(v_i, v_j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}, \quad \cos(v_i, v_j) = \frac{|N_i \cap N_j|}{\sqrt{|N_i| |N_j|}}.$$

Pro rozsáhlé sítě se tedy spočítá podobnost uzlů kosinovou nebo Jaccardovou mírou a aplikuje se klasický k -means, v němž je každý klastr asociován s centroidem a každý uzel je přiřazen ke klastru s nejbližším centroidem.

Modely latentního prostoru. Zobrazí uzly do *nízkorozměrného* prostoru tak, aby byla zachována blízkost uzlů podle propojenosti sítě, a teprve pak aplikují k -means. *Vícerozměrné škálování* (MDS) pro danou síť zkonstruuje matici blízkosti P reprezentující párové vzdálenosti uzlů, například geodetické. Nechť $S \in \mathbb{R}^{n \times l}$ jsou souřadnice uzlů v nízkorozměrném prostoru; řešením je $S = V\Lambda^{1/2}$, kde V je l hlavních vlastních vektorů (a Λ diagonální matice odpovídajících vlastních čísel) dvojité centrované matice blízkosti.

Blokové modely. Matice sousednosti se aproximuje jako $A \approx S\Sigma S^T$, kde Σ je *míchací matice* a S *indikátorová matice komunit*. Uvolní-li se S na reálné hodnoty, odpovídá optimální řešení hlavním vlastním vektorům A .

Spektrální shlukování. Poměrový i normalizovaný řez lze přeformulovat jako minimalizaci $\text{tr}(S^T L S)$, kde $L = D - A$ je *grafový laplacián* (pro poměrový řez), resp. $L = D^{-1/2}(D - A)D^{-1/2}$ *normalizovaný grafový laplacián* (pro normalizovaný řez), a D je diagonální matice stupňů. Po *spektrální relaxaci* je optimálním řešením matice tvořená vlastními vektory s *nejmenšími* vlastními čísly. První vlastní vektor je přitom nepoužitelný, protože říká, že všechny uzly patří do téhož klastru.

Maximalizace modularity. Modularita měří sílu rozdělení tým, že bere v úvahu rozdělení stupňů. *Síla komunity* je $\sum_{i,j \in C} (A_{ij} - \frac{d_i d_j}{2m})$ a modularita $Q = \frac{1}{2m} \sum_C \sum_{i,j \in C} (A_{ij} - \frac{d_i d_j}{2m})$. Analogicky ke spektrálnímu shlukování lze maximalizaci modularity přeformulovat pomocí *matice modularity* $B = A - \frac{dd^T}{2m}$. Optimálním řešením jsou pak *hlavní* vlastní vektory matice modularity, na které se jako postprocessing aplikuje k -means.

Jednotný pohled: maticová faktorizace. Modely latentního prostoru, blokové modely, spektrální shlukování a maximalizaci modularity lze všechny zapsat jako faktorizaci $X \approx S \Sigma S^T$, kde

$$X = \begin{cases} \text{(dvojitě centrovaná) matice blízkosti} & \text{modely latentního prostoru,} \\ \text{sociomatrix } A & \text{aproximace blokovým modelem,} \\ \text{grafový laplacián} & \text{minimalizace řezu,} \\ \text{matice modularity} & \text{maximalizace modularity.} \end{cases}$$

Liší se tedy jen tím, co se za X dosadí. **Omezení** mají všechny stejné: vyžadují, aby uživatel zadal *počet komunit* předem.

Hierarchicky orientovaná detekce komunit

Cílem je vybudovat hierarchickou strukturu komunit na základě topologie sítě, což umožňuje analyzovat síť v různých *rozlišeních*. Reprezentativní přístupy jsou dva.

Divizivní hierarchické shlukování, jehož zástupcem je Girvanové–Newman ($O(m^2 n)$, resp. $O(n^3)$ pro řídké sítě), rozdělí uzly do několika množin a každou množinu dále rozdělí na menší; na rozdělení lze aplikovat síťově orientované metody. Jedním konkrétním příkladem je rekurzivní odstraňování „nejslabší“ vazby: najde se hrana s nejmenší silou, odstraní se a aktualizuje se síla každé hrany, a to rekurzivně, dokud se síť nerozpadne na požadovaný počet komponent souvislosti. Každá komponenta pak tvoří komunitu. Sílu vazby lze měřit *mezilehlostí hrany*, protože hrana s vyšší mezilehlostí bývá mostem mezi dvěma komunitami.

Aglomerativní hierarchické shlukování, jehož zástupcem je Louvain s $O(n \log n)$, postupuje opačně: každý uzel se inicializuje jako komunita a komunity se pak postupně slučují do větších podle konkrétního kritéria, například na základě *přírůstku modularity*.

6.17 Predikce vazeb

Detekce komunit se ptá, jak síť vypadá teď. Predikce vazeb se ptá, jak bude vypadat dál, a opírá se přitom o dva principy: o homofilii a o tranzitivní uzávěr.

Úloha predikce vazeb

Cílem je predikovat *budoucí* vazby mezi páry uzlů v síti. V komerčních sociálních sítích jako Facebook nebo LinkedIn se tak uživatelům doporučují potenciální přátelé. Opřít se lze o podobnost obsahu i o strukturu sítě:

- **Míry založené na obsahu** využívají princip *homofilie*, tedy že uzly s podobným obsahem se spíše propojí. Predikci vazeb zlepšují, jsou ale citlivé na konkrétní síť, jak ukazují krátké a zašuměné tweety s nestandardními akronymy na Twitteru. Platí přitom, že *obsahová homofilie nutně neimplikuje strukturní propojenost!*
- **Strukturní míry** využívají princip *tranzitivního uzávěru*, tedy že dva uzly, které sdílejí sousedství, se spíše propojí. Tranzitivní uzávěr je velmi častý napříč doménami, a proto jsou tyto míry účinné v různých typech sítí a přímo aplikovatelné. Patří k nim *sousedské* míry, *Katzova* míra a míry založené na *náhodných procházkách*. Opačným směrem už implikace platí: *strukturní propojenost obvykle implikuje i obsahovou homofilii*.

Sousedské míry predikce vazeb

Sousedské míry kvantifikují pravděpodobnost budoucí vazby počtem *společných sousedů* mezi párem uzlů i a j . Nechť S_i je množina sousedů uzlu i .

Míra společných sousedů je prostě $CN(i, j) = |S_i \cap S_j|$. Její *slabinou* je, že nezohledňuje *relativní* počet společných sousedů vůči počtu ostatních spojení: kdyby byly Alice a Bob velmi populární veřejné osobnosti spojené s velkým počtem aktérů, mohli by mít mnoho společných sousedů *jen náhodou*.

Jaccardova míra $Jaccard(i, j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}$ měří podobnost sousedství *normalizovanou* vůči stupňům uzlů, které se kvůli mocninnému rozdělení silně liší. Větší stupně Alice nebo Boba tedy jejich Jaccardův koeficient snižuje. *Slabinou* je, že se míra dobře nepřizpůsobuje *prostředním* sousedům: jsou-li společní sousedé Alice a Boba velmi populární, tedy mají vysoký stupeň, pak se vyskytují jako společní sousedé i u mnoha jiných párů uzlů, a jsou proto pro predikci *méně důležití*.

Adamicova–Adarova míra je vážená verze míry společných sousedů, navržená tak, aby zohlednila různou důležitost různých společných sousedů. Váha společného souseda je *klesající* funkcí jeho stupně, např. $1/\log |S_k|$:

$$AA(i, j) = \sum_{k \in S_i \cap S_j} \frac{1}{\log |S_k|}.$$

Příklad: pro čtyři společné sousedy se stupni 4, 2, 2, 4 vyjde $\frac{1}{\log 4} + \frac{1}{\log 2} + \frac{1}{\log 2} + \frac{1}{\log 4} = \frac{3}{\log 2}$.

Katzova míra

Sousedské míry *nejsou účinné*, je-li počet sousedů sdílených párem uzlů malý. Sdílejí-li Alice a Bob jediného společného souseda a Alice a Jim také jediného, sousedské míry mezi těmito případy nerozliší, přestože existuje *nepřímá* propojenost přes delší cesty. V takových případech jsou vhodnější míry založené na *procházkách*, jako je Katzova míra.

Nechť $W_{ij}^{(t)}$ je počet procházek délky t mezi uzly i a j . Pro předem zvolený parametr $\beta < 1$ se definuje

$$Katz(i, j) = \sum_{t=1}^{\infty} \beta^t W_{ij}^{(t)},$$

kde β je *diskontní faktor*, který potlačuje význam delších procházek; pro malá β nekonečná suma konverguje. Je-li A symetrická matice sousednosti neorientované sítě, lze matici Katzových koeficientů K rozměru $n \times n$ spočítat jako

$$K = \sum_{t=1}^{\infty} \beta^t A^t = (I - \beta A)^{-1} - I,$$

neboť vlastní čísla A^t jsou t -té mocniny vlastních čísel A . Silně centrální uzly mají tendenci tvořit vazby s mnoha uzly $\Rightarrow$ součet Katzových koeficientů uzlu i vůči ostatním uzlům se nazývá jeho **Katzova centralita**.

Predikce vazeb jako klasifikační úloha

Přítomnost či absence vazby mezi párem uzlů se dá chápat jako *binární* indikátor třídy. Pro každý pár uzlů se pak zkonstruuje vícerozměrný datový záznam:

- *příznaky* zahrnují všechny různé podobnosti mezi uzly, tedy sousedské, Katzovy i založené na procházkách, plus řadu příznaků preferenčního připojování, například stupně obou uzlů v páru;
- výsledkem je *pozitivně-neoznačkováná* klasifikační úloha, protože páry uzlů s hranou jsou

pozitivní příklady a zbývající páry jsou neoznačované;

- neoznačované příklady lze při trénování přibližně považovat za negativní;
- negativních párů je ale v rozsáhlých a řídkých sítích příliš mnoho, a proto se použije jen jejich vzorek.

Algoritmus. V *trénovací fázi* se vygenerují data s jedním záznamem pro každý pár uzlů s hranou a se vzorkem záznamů z párů bez hrany; příznaky odpovídají extrahovaným podobnostním a strukturním rysům a označením třídy je přítomnost či absence hrany. Nad těmito daty se postaví a natrénuje model klasifikátoru. V *testovací fázi* se každý testovaný pár uzlů převede na vícerozměrný záznam a natrénovaný klasifikátor predikuje jeho označení. Běžnou volbou báze klasifikátoru je *logistická regrese*; kvůli nevyváženosti dat se často používají *cenově citlivé* verze klasifikátorů.

Výhody supervizovaného přístupu. Obsahové příznaky lze extrahovat a použít snadno a klasifikátor se sám naučí jejich relevanci. Pro orientované sítě lze navíc extrahovat příznaky i *asymetricky*, například vstupní a výstupní stupně místo obyčejných stupňů nebo různé míry založené na náhodných procházkách jako personalizovaný PageRank. Celkově přístup nabízí vyšší flexibilitu díky schopnosti učit se vztahy mezi vazbami a příznaky různých typů.

Souhrn predikce vazeb. Jednotlivé míry se v účinnosti liší podle datové sady. *Sousedské* míry lze efektivně spočítat i pro rozsáhlá data a fungují téměř tak dobře jako ostatní nesupervizované míry. Míry založené na *náhodných procházkách* a *Katzovy* míry jsou užitečné pro *velmi řídké* sítě, v nichž nelze počet společných sousedů robustně změřit. *Supervize* poskytuje lepší přesnost a adaptabilitu napříč doménami sociálních sítí, je ale výpočetně drahá. V posledních letech se k vylepšení predikce používá i obsah, avšak strukturní míry jsou *výrazně silnější*, což lze přičíst tranzitivním vlastnostem reálných sítí. Obsahové míry jsou založeny na „obrácené homofilii“, kdy se pro predikci vazeb využívá podobný nebo s vazbami korelovaný obsah.

6.18 Shrnutí ke zkoušce

- **SNA** vyrostla ze sociálních věd, z analýzy sítí a z teorie grafů a jejím předmětem jsou *vztahy*, nikoli entity a jejich atributy. Sít se dá zapsat obrázkem, seznamem hran nebo maticí sousednosti, která je u orientovaných grafů nesymetrická. Rozlišuje se *ego síť* a „celá“ síť, jenže žádná studovaná síť ve skutečnosti celá není: tomu se říká *problém specifikace hranic*.
- **Síla vazeb** dělí hrany na silné a slabé; klasickým výsledkem je *síla slabých vazeb* (Granovetter 1973). Se silou vazeb souvisejí homofilie, tranzitivita a přemostění, přičemž homofilie spolu s tranzitivitou vedou ke vzniku *klik*. Most je hrana, jejíž odstranění rozpojí koncové uzly, a o *zkratkový most* jde tehdy, když po jejím odstranění vzroste vzdálenost nad 2. Sílu vazby kvantifikuje **překryv sousedství** $O(i, j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2}$: čím větší, tím silnější vazba.
- **Čtyři míry centrality** se ptají každá na něco jiného. *Stupňová* říká, kolik lidí uzel přímo dosáhne. *Mezilehlá* počítá, na kolika nejkratších cestách uzel leží, a tím ukazuje, kde se síť rozpadne. *Blízkostní* je převrácená průměrná délka nejkratších cest a odpovídá rychlosti šíření. *Vlastní* se počítá z $Ax = \lambda x$, kde nezáporný vektor dává jen *největší* vlastní číslo, a měří, jak dobře je uzel spojen s dobře propojenými. U zkoušky se chce interpretační tabulka a vysvětlení, proč jedna míra nestačí (uzly 3 a 5 *společně* > uzel 10). **Strukturní ekvivalence** znamená, že si dva uzly mohou vyměnit pozice, aniž se změní struktura.
- **Kohezi** popisuje několik ukazatelů. Recipocita má smysl jen u orientovaných grafů. Hustota je $|E|/\binom{|V|}{2}$, klika má hodnotu 1 a orientované grafy mají hustotu poloviční. **Klastrovací koeficient** uzlu je $C_i = \frac{2|e_{jk}|}{k_i(k_i-1)}$ a koeficient celé sítě je jejich průměr. Vzdálenosti popisuje průměr sítě (*diameter*), tedy nejdelší nejkratší cesta, dále průměrná vzdálenost a excentricita. O **malý svět** jde tam, kde je vysoké C a zároveň krátká průměrná cesta.
- Reálné komplexní sítě mají **šest vlastností**: efekt malého světa (Milgram naměřil medián 6), tranzitivitu neboli klastrování, mocinné rozdělení stupňů, odolnost (síť je robustní k náhodnému, ale zranitelná cíleným odstraněním hubů), vzorec míchání (asortativní míchání odpovídá

homofilii) a komunitní strukturu. **Bezškálové sítě** vznikají růstem a *preferenčním připojováním*, kterému se také říká rich-get-richer, Matoušův efekt nebo kumulativní výhoda; vysvětluje se popularitou, kvalitou a smíšeným modelem (halo efekt). Celkový tvar sítě pak doplňuje **centralizace** a struktura *jádro-periferie* (bow-tie analýza).

- **Modelování vlivu** stojí na společných rysech: pracuje se s orientovaným grafem, uzly jsou aktivní, nebo neaktivní, a *aktivovaný uzel nelze deaktivovat*. **LTM** losuje prah θ_v uniformně z $[0, 1]$ a uzel aktivuje, jakmile $\sum_{\text{aktivní}} w_{u,v} \geq \theta_v \sum w_{u,v}$; soustředí se tedy na *příjemce*. **ICM** naopak dává uzlu *jedinou* příležitost aktivovat každého souseda, a to s pravděpodobností $p_{v,u}$; těžiště je zde na *vysílajícím*. **Maximalizace vlivu** je NP-těžká, hladový přístup ale zaručuje **63 %** optima.
- Odlišit **vliv od korelace** dá několik kroků. *Test korelace* porovná podíl hran mezi různými hodnotami atributu s hodnotou $2p(1-p)$; u kuřáků vyšlo 14 % < 49 %. Za korelací mohou stát tři procesy: homofilie, *zmatení* a vliv. Logistický model $\ln \frac{p(a)}{1-p(a)} = \alpha \ln(a+1) + \beta$ má v α koeficient sociální korelace a **shuffle test** zpřehází časové značky; liší-li se α' významně od α , je to důkaz *vlivu*.
- **Test homofilie** porovnává počet heterogenních hran s hodnotou $2pq$. Je-li jich významně méně, síť vykazuje homofilii; je-li jich významně *více*, jde o *inverzní* homofilii, jakou znají třeba partnerské vztahy. U zkoušky se počítá příklad: 5 z 18 proti $2pq = 8$ z 18.
- **Výběr a sociální vliv** působí opačným směrem: při výběru řídí charakteristiky tvorbu vazeb, kdežto vliv znamená, že vazby formují *mutabilní* charakteristiky. *Imutabilní* je pohlaví, rasa a věk, *mutabilní* je chování a názory. Oba směry lze rozlišit jedině *longitudinálními* studiemi; studie obezity (Christakis & Fowler, 12 000 lidí, 32 let) přinesla důkaz pro *sociální vliv*.
- **Afiliční sítě** jsou bipartitní a spojují osoby $\times$ fokusy, zatímco sociálně-afiliční síť má rovnou dva typy hran. Uzávěry jsou **tři**: tranzitivní přes společného přítele, *fokální* přes společný fokus, což je *výběr*, a *členský*, kdy přítel už ve fokusu je, což je *sociální vliv*. Měří se tak, že se $T(k)$ odečte ze dvou snímků sítě a porovná s bazovým modelem $T_{\text{base}}(k) = 1 - (1-p)^k$; empiricky vychází velký nárůst z 1 na 2 a poté *klesající výnosy*.
- **Schellingův model** rozmístí do mřížky agenty dvou typů. Agent je spokojený, jakmile má alespoň t sousedů téhož typu, a nespokojení se přesouvají, takže vzniká *samovolná segregace*. Model ukazuje dvě věci: jak se *imutabilní* charakteristika stane silně korelovanou s *mutabilní*, tady s volbou místa, a jak lokální homofilie vytvoří globální vzorec.
- **Strukturní balance** pokládá za balancované trojúhelníky +++ a +--, tedy ty, kde jsou všechny tři vazby pozitivní, nebo právě jedna. **Věta o balanci** říká, že v úplném balancovaném grafu jsou buď všichni přátelé, nebo se graf rozdělí na dvě skupiny vnitřních přátel, které jsou si navzájem nepřáteli; u zkoušky se žádá i důkaz, kde X jsou přátelé A , Y nepřátelé A a postupuje se ve třech krocích. **Slabá balance** zakazuje jedinou konfiguraci, totiž *dvě pozitivní a jednu negativní*, a připouští proto rozdělení na *libovolný počet* skupin. Aplikovala se na evropské aliance 1872–1907 a na Bangladěš 1972.
- **HITS** je dotazově *závislý*: z kořenové množiny S ($t = 200$) se vytvoří bazová množina B a pak se mocninou iterací střídá $a = L^T h$ a $h = La$, přičemž normalizace je zde **nutná**. Posloupnost $a_k = (L^T L)^{k-1} a_0$ konverguje k hlavnímu vlastnímu vektoru matice $L^T L$, což je *ko-citační* matice, kdežto LL^T je bibliografické párování. Slabinou je snadné spamování, drift tématu a neefektivnost v době dotazu.
- **PageRank** je naproti tomu dotazově *nezávislý* a počítá se jako $P(i) = \sum_{(j,i) \in E} P(j)/O_j$. Normalizaci *nepotřebuje*, protože celkový PageRank zůstává konstantní, jako by šlo o „kapalinu“. Problém „pomalého úniku“ řeší *škálované* pravidlo s tlumícím faktorem d , který se volí mezi 0,8–0,9 a v článku má hodnotu 0,85; obrazně se tomu říká „vodní cyklus“. Markovský řetězec potřebuje matici *stochastickou, ireducibilní a aperiodickou* a webový graf nesplňuje ani jednu z těchto podmínek: má visící stránky, není silně souvislý a vykazuje periodicitu. Všechny tři nedostatky přitom napraví *jediná* strategie, totiž odkaz z každé stránky na každou s malou pravděpodobností. Finální tvar je $P(i) = (1-d) + d \sum_{(j,i) \in E} P(j)/O_j$ a řeší se mocninou iterací. Výhodou je odolnost proti spamu, protože vstupní odkazy nelze snadno získat, a globální offline

míra; kritizuje se dotazová nezávislost.

- **Tematické komunity** se zapisují jako $K = (T, M)$: komunitu určuje téma, komunity se mohou *překrývat* a tvoří hierarchii subkomunit. *Bipartitní jádra* pracují s (i, j) -jádreem, kde vějíře jsou huby a středy autority; postupuje se prořezáváním a generováním jader, jenže výsledkem jsou pouze jádra, ne témata ani hierarchie. *Komunity pojmenovaných entit* se hledají tak, že se ze společných výskytů ve větě postaví graf, v něm se najdou trojúhelníky a z nich jádra, která se nakonec klastrují podle textové podobnosti.
- **Detekce komunit** naráží hned na začátku na to, že **definice komunity je subjektivní**; rozlišují se skupiny explicitní a implicitní. *Minimální řez* dává nevyvážené skupiny a stojí $O(n^5)$, s Kargerovým algoritmem $O(n^4)$; s obecnými vahami a balančními omezeními je **NP-těžký**, a proto se používá *poměrový* a *normalizovaný řez*, které dělí velikostí $|C_i|$, resp. objemem $\text{vol}(C_i)$.
- **Girvanové–Newman** je divizivní algoritmus: postupně odstraňuje hrany s nejvyšší *mezilehlostí*, což stojí $O(m^2n)$, tedy $O(n^3)$. Počet komunit se vybere podle nejvyšší **modularity** $Q = \frac{1}{2m} \sum_{i,j} (A_{ij} - \frac{k_i k_j}{2m}) \delta(c_i, c_j)$, která nabývá hodnot $Q \in [-1, 1]$; za smysluplné se považují komunity s $Q \geq 0,3$.
- **Louvain** maximalizuje modularitu hladově a aglomerativně ve dvou fázích: nejdřív lokální přesuny, pak stažení do supergrafu, celkem za $O(n \log n)$. Nevýhodou je citlivost na pořadí zpracování, možné *nesouvislé komunity* a uvážnutí v lokálních optimech. **Leiden** proto přidává fázi *upřesnění rozdělení*, pořad ale dává jen tvrdé rozdělení. Otevřenou výzvou zůstává neznámý počet komunit a heterogenní velikosti a hustoty.
- Metody se dělí do **čtyř kategorií**. *Uzlově orientované* stojí na klikách: hledání kliky je NP-těžké, pomáhá rekursivní prořezávání uzlů se stupněm $< k - 1$ a **CPM** najde i *překrývající se* komunity přes graf klik, které sdílejí $k - 1$ uzlů. Sem patří k -klika, k -klub a k -klan, dále k -jádro a k -plex, jež jsou komplementární, a také LS množiny a k -komponenty. *Skupinově orientované* metody hledají γ -husté kvazikliky. *Síťově orientované* vycházejí z podobnosti uzlů: Jaccard nebo kosinus v kombinaci s k -means, dále MDS, blokové modely, spektrální shlukování přes grafový laplacián s *nejmenšími* vlastními čísly a maximalizace modularity přes matici modularity s *největšími*. Všechny se dají sjednotit jako maticová faktorizace $X \approx S \Sigma S^T$, která ovšem vyžaduje zadat počet komunit. *Hierarchicky orientované* jsou divizivní Girvanové–Newman a aglomerativní Louvain.
- **Predikce vazeb** používá buď obsahové míry opřené o homofilii, nebo strukturní míry vycházející z tranzitivního uzávěru, které jsou silnější. Ze sousedských měr počítají *společní sousedé* $|S_i \cap S_j|$, ti ale nezohledňují stupně; *Jaccard* $\frac{|S_i \cap S_j|}{|S_i \cup S_j|}$ normalizuje stupně obou uzlů, nikoli však prostředních sousedů; *Adamic–Adar* $\sum_k \frac{1}{\log |S_k|}$ dává společnému sousedovi tím menší váhu, čím vyšší má stupeň. **Katzova míra** $\sum_t \beta^t W_{ij}^{(t)} = (I - \beta A)^{-1} - I$ se hodí pro *řídke* sítě s málo společnými sousedy a její součet přes ostatní uzly dává *Katzovu centralitu*. V **klasifikačním pojetí** se úloha formuluje jako pozitivně-neoznačkováná: nasbírání se vzorek negativ, bázovým klasifikátorem bývá logistická regrese a existují cenově citlivé varianty. Výhoda je dvojí, protože klasifikátor sám najde relevanci příznaků a u orientovaných sítí lze použít *asymetrické* příznaky.

7 Praktické využití technik

Poslední věta okruhu žádá spojení obou polovin dohromady: k čemu se probrané techniky *skutečně* používají. Odpověď má u zkoušky pokaždé stejnou stavbu, takže se vyplatí ji mít nacvičenou. Ke každé aplikaci se hodí umět říct, *jakou úlohu* řeší (kap. 1), *jakou metodou* se na ni jde (kap. 3) a *jak se její výsledek vyhodnocuje* (kap. 5). Aplikace bez těchto tří údajů zůstává jen historkou.

7.1 Dobývání znalostí

Následující přehled spojuje typické aplikace s úlohou, kterou řeší, a s metodou, která se na ni nasažuje. Třetí sloupec vždy doplňuje, co je na té které aplikaci v praxi podstatné.

Aplikace	Typ úlohy / metoda	Poznámka
Analýza nákupního košíku	asociační pravidla, Apriori / FP-Growth	plánování a rozvržení prodejny, kupony a časově omezené slevy, „balíčkování“ produktů; srovnání nových a existujících prodejen pomocí <i>virtuálních položek</i>
Detekce podvodů	klasifikace, detekce odlehlých hodnot	20 mil. výkonů zdravotní pojišťovny → 214 tis. podivných → 14 tis. předaných k ruční kontrole → 8 tis. potvrzených podvodů; stroj <i>zaostřuje</i> pozornost experta
Kreditní skórování	rozhodovací stromy, naivní Bayes, logistická regrese	průběžný příklad v celém textu; požaduje se <i>srozumitelnost</i> (regulace) a zacházení s chybějícími hodnotami
Segmentace zákazníků	klastrová analýza (<i>k</i> -means, hierarchické)	reprezentace segmentu <i>centroidem</i> („typický zákazník“) je pro byznys velmi intuitivní
Predikce odchodu (churn)	klasifikace na nevyvážených datech	accuracy nestačí ⇒ precision/recall, F_1 , ROC/AUC; <i>lift analýza</i> pro rozhodnutí, kolik zákazníků kontaktovat
Cílené marketingové kampaně	skórování a řazení, lift	ekonomická úvaha nad lift křivkou (příklad s hodinkami: 1. příhrádka +\$550, 10. příhrádka −\$475)
Klasifikace textu	SVM s kernely	patří k nejlepším klasifikátorům pro text (vysokorozměrná data); srozumitelnost se neočekává
Predikce časových řad	regrese, neuronové sítě	řada se transformuje posuvným oknem na tabulku vstupů a výstupů
Klasifikace chemických látek	klasifikace strukturních dat	reprezentace řetězci SMILES nebo fakty v Prologu
Web usage mining	sekvenční vzory (GSP)	navigační vzory uživatelů podle posloupností návštěv stránek
Doporučovací systémy [♦ nad rámec slidů]	klastrová analýza, klasifikace, predikce vazeb	kolaborativní filtrování (podobnost uživatelů či položek) vs. obsahové filtrování; studený start řeší obsah

Poznámka k motivaci rozhodovacích stromů. Přednáška uvádí případ banky Barclays, která zavedla telefonní systém rozdělující volající podle úvěrového hodnocení. Klienti s vysokými úsporami nebo s dostatečným úvěrovým limitem byli spojováni s britským operátorem, všichni ostatní putovali do indického call centra. Cílem bylo „odfiltrovat“ zákazníky, kteří nemají peníze na nákup produktů banky.

Kritika systému mířila na dvě věci. Zaprvé zvýhodňuje bohatší klienty, zadruhé si vůbec neklade za cíl poskytnout volajícímu nejlepší radu. Je to dobrá ilustrace toho, že i *technicky správný* klasifikační model může mít problematické důsledky, a zároveň argument pro to, aby fáze *vyhodnocení* v CRISP-DM zahrnovala i pohled mimo přesnost modelu.

7.2 Analýza sociálních sítí

Aplikace analýzy sociálních sítí sahají od podnikové komunikace přes bezpečnost až po medicínu. Přehled je proto řazený podle oblasti, ve které se metoda nasazuje.

Oblast	Využití
Podniky	analýza a zlepšení komunikačních toků v organizaci, s partnery či zákazníky; marketingové kampaně založené na sociálních sítích (vizualizace komunikačních toků před a po zavedení systému pro správu obsahu)
Bezpečnostní složky a armáda	identifikace kriminálních a teroristických sítí ze zachycené komunikace; identifikace <i>klíčových hráčů</i> v nich (míry centrality, odd. 6)
Sociální sítě (Facebook)	identifikace a doporučení potenciálních přátel na základě „přátel přátel“ — přímo úloha <i>predikce vazeb</i>
Občanská společnost	odhalování konfliktů zájmů ve skrytých vazbách mezi státními orgány, lobbisty a podniky
Síťoví operátoři	optimalizace struktury a kapacity komunikačních sítí
Medicína	nakažlivé nemoci a jejich prevence; sexuálně přenosné nemoci — epidemie syfilis v Rockdale County (Atlanta, 1996): skupina bílých dívek okolo 16 let, skupina bílých chlapců 17–21 let a skupina afroamerických chlapců 17–21 let; syfilis diagnostikována u 6 bílých žen, 2 bílých mužů a 2 afroamerických mužů
Bezškálové sítě	<i>výpočetní technika</i> : sítě s SF architekturou (WWW) jsou extrémně odolné proti náhodným poruchám, ale velmi zranitelné koordinovaným útokem — vymýcení internetových virů je v podstatě nemožné; <i>medicína</i> : vakcinační kampaně cílené na huby (lidé s mnoha kontakty — jejich identifikace je ale obtížná), nové léky mířící na klíčové molekuly (huby) dané nemoci s minimalizací vedlejších účinků pomocí nalezených vazebných sítí v buňkách; <i>business</i> : porozumění vazbám mezi firmami, odvětvím a ekonomikou pro monitoring a eliminaci kaskádových finančních krachů; marketing — zkoumání šíření „nákazy“ v SF sítích a efektivnější reklama nových produktů
Detekce komunit	<i>Belgický mobilní operátor</i> (Louvainova metoda): cca 2 miliony zákazníků; velikost uzlu odpovídá počtu jednotlivců v komunitě, barva hlavnímu jazyku komunity (červená francouzština, zelená nizozemština); zobrazeny jen komunity nad 100 zákazníků. Zoom ve vyšším rozlišení odhalí <i>přechodovou</i> komunitu tvořenou subkomunitami s méně zřetelným jazykovým oddělením

Kdy a proč SNA použít. Důvodů je několik a u zkoušky se hodí mít je srovnané. Nejprostší z nich je studium samotné sociální sítě, ať už offline, nebo online, případně snaha o zvýšení její efektivity. Druhým je vizualizace: data se zobrazí tak, aby z nich vystoupily vzorce ve vztazích či interakcích. Třetím pak sledování cest, kterými v sociálních sítích proudí informace.

Zvláštní místo má síťová perspektiva v kvantitativním výzkumu, cenná je ovšem i pro výzkum kvalitativní. Rozsah akcí a příležitostí dostupných jednotlivci je totiž často funkcí jeho *pozice* v sociální síti, a odhalení těchto pozic (např. jako otců, matek, učitelů, pracovníků) může přinést zajímavé a někdy překvapivé výsledky. Kvantitativní analýza sítě k tomu pomáhá identifikovat různé typy aktérů, především klíčové hráče. Poslední skupinou důvodů je práce se sociálními médii obecně: hledání příčin, proč je komunita nebo síť dysfunkční, a naopak podpora sociální koheze a růstu online komunity.

Úvahy o návrhu (*thoughts on design*)

Teorie SNA stála u zrodu některých z prvních sociálních sítí, například SixDegrees. Při návrhových rozhodnutích se přesto dodnes používá spíš zřídka, a tak zůstává otevřená řada otázek:

- Jak může online platforma, případně její správci, využít metody a poznatky SNA?
- Jak může platforma povzbudit uživatele, aby jednali (*lokálně*) tak, že tím rozšíří komunitu?
- Jaká opatření může online komunita přijmout k optimalizaci své struktury? Nežádoucí mohou

být třeba *kliky*, protože odhánějí nově příchozí.

- Jaké struktury jsou žádoucí pro různé typy online platform?
- Jak mohou online komunity identifikovat a využít klíčové hráče ke svému prospěchu?

7.3 Provozní a etické aspekty nasazení

[♦ nad rámec slidů] Fáze *Deployment* v CRISP-DM (odd. 1.5) projekt nekončí. Model nasazený do provozu žije dál a přináší si s sebou vlastní třídu problémů, technických i etických:

- **Monitoring a drift.** Rozdělení dat se v čase mění (*data drift*) a stejně tak se mění i vztah mezi vstupy a cílem (*concept drift*). Model proto stárne: je nutné pravidelně sledovat pokles jeho kvality na produkčních datech a přeučovat ho.
- **Zpětná vazba modelu do dat.** Nasazený model ovlivňuje tutéž realitu, kterou měří. Skórovací model pro schvalování úvěrů vidí jen ty žadatele, kterým úvěr schválil, takže další trénovací data jsou zkreslená jím samým.
- **Soukromí.** Analýza sociálních sítí pracuje s daty *o vztazích*, které uživatel často nemohl ovlivnit, a i anonymizovaný graf lze podle jeho struktury deanonymizovat. Predikce vazeb a modelování vlivu jsou v podstatě nástroje pro předpovídání a ovlivňování chování konkrétních lidí.
- **Spravedlnost a diskriminace.** Model naučený na historických datech reprodukuje historickou nerovnost, jak ukazuje případ Barclays výše. Vypustit chráněný atribut ze vstupů přitom nestačí, protože jej lze často *rekonstruovat* z ostatních atributů.
- **Právní rámec.** GDPR ve svém čl. 22 zakotvuje právo na vysvětlení automatizovaného rozhodnutí a AI Act klade požadavky na vysoce rizikové systémy. V regulovaných doménách je tedy interpretovatelnost právním požadavkem, nikoli pohodlím.

7.4 Shrnutí ke zkoušce

- U každé aplikace je dobré umět říct **trojici** údajů: o jaký typ úlohy jde (deskriptivní, nebo prediktivní; s učitelem, nebo bez učitele), jakou metodou se řeší a jak se výsledek vyhodnocuje.
- Mezi **vlažkové aplikace dobývání znalostí** patří analýza nákupního košíku (asociační pravidla), detekce podvodů (příklad zdravotní pojišťovny, kde stroj *zaostřuje* pozornost experta, ale nenahrazuje jej), kreditní skórování (stromy a Bayes s požadavkem srozumitelnosti), segmentace zákazníků (klastrování, centroid = typický zákazník), churn a cílený marketing (nevyvážená data → ROC/AUC a *lift*), klasifikace textu (SVM), predikce časových řad (posuvné okno) a web usage mining (sekvenční vzory).
- Za **vlažkové aplikace SNA** se dají označit identifikace klíčových hráčů v kriminálních a teroristických sítích (centralita), doporučování přátel (predikce vazeb), optimalizace komunikačních sítí, epidemiologie (Rockdale County 1996), vakcinace cílená na *huby*, marketing a kaskádové finanční krachy v bezškálových sítích a segmentace zákazníků operátora podle jazykových komunit (Louvain, 2 mil. zákazníků).
- **Klíčová vlastnost SF sítí pro praxi** je jejich dvojaká odolnost: náhodnému výpadku odolají až do 80 % uzlů, ale cílený útok na 5–15 % hubů je rozloží. Z toho plyne jak strategie vakcinace, tak nemožnost vymýtit internetové viry.
- **Nasazením projekt nekončí.** Model je nutné monitorovat kvůli driftu, počítat se zpětnou vazbou jeho rozhodnutí do dat, hlídat soukromí (i anonymizovaný graf lze deanonymizovat) i spravedlnost (chráněný atribut lze rekonstruovat z ostatních) a respektovat GDPR čl. 22 a AI Act.